



Reinforcement Learning

陈科海

计算机科学与技术学院

哈尔滨工业大学（深圳）

chenkehai@hit.edu.cn

目录

01 Probability theory

02 Terminology

03 Rewards and Returns

04 Value Functions

05 Play games using RL

06 Value-based learning

07 Policy-based learning

08 Actor-critic method

09 Value Functions

10 Play games using RL

Random Variable

- **Random variable**: unknown; its values depends on outcomes of a random event.
- Uppercase letter ***X*** for random variable.

*Random
Variable*

*Possible
Values*

*Random
Events*

Probabilities

$$X = \begin{cases} 0 \\ 1 \end{cases}$$



$$\mathbb{P}(X = 0) = 0.5$$

$$\mathbb{P}(X = 1) = 0.5$$

Random Variable

- **Random variable**: unknown; its values depends on outcomes of a random event.
- Uppercase letter **X** for random variable.
- Lowercase letter **x** for an observed value.
- For example, I flipped a coin 4 times and observed:
 - $x_1 = 1$,
 - $x_2 = 1$,
 - $x_3 = 0$,
 - $x_4 = 1$.

Probability Density Function (PDF)

- PDF provides a relative likelihood that the value of the random variable would equal that sample.

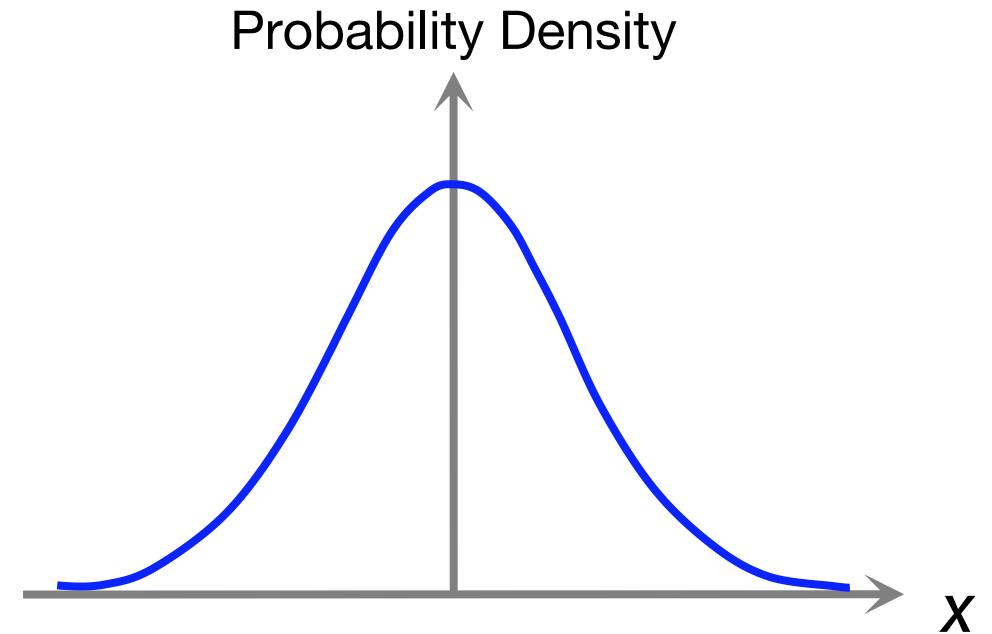
Probability Density Function (PDF)

- PDF provides a relative likelihood that the value of the random variable would equal that sample.

Example: Gaussian distribution

- It is a continuous distribution.
- PDF:

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right).$$



Probability Mass Function (PMF)

- PMF is a function that gives the probability that a discrete random variable is exactly equal to some value

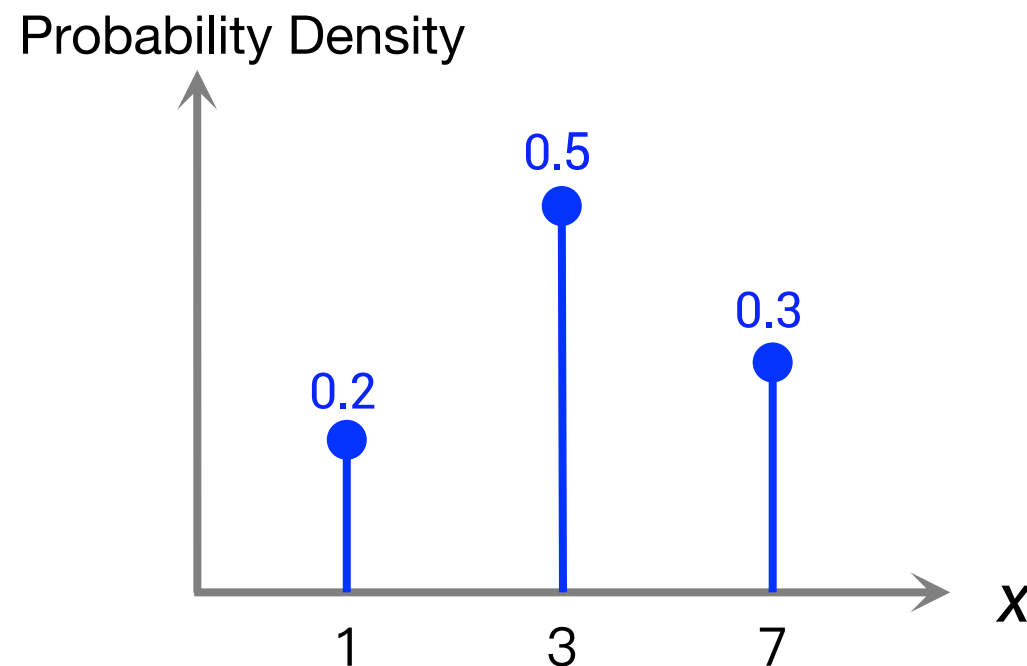
Example

- Discrete random variable: $X \in \{1, 3, 7\}$.
- PDF:

$$p(1) = 0.2,$$

$$p(3) = 0.5,$$

$$p(7) = 0.3.$$



Properties of PDF/PMF

- Random variable X is in the domain $\mathcal{X}$.
- For continuous distribution,

$$\int_{\mathcal{X}} p(x) dx = 1.$$

- For discrete distribution,

$$\sum_{x \in \mathcal{X}} p(x) = 1.$$

Expectation

- Random variable X is in the domain $\mathcal{X}$.
- For continuous distribution, the expectation of $f(X)$ is:

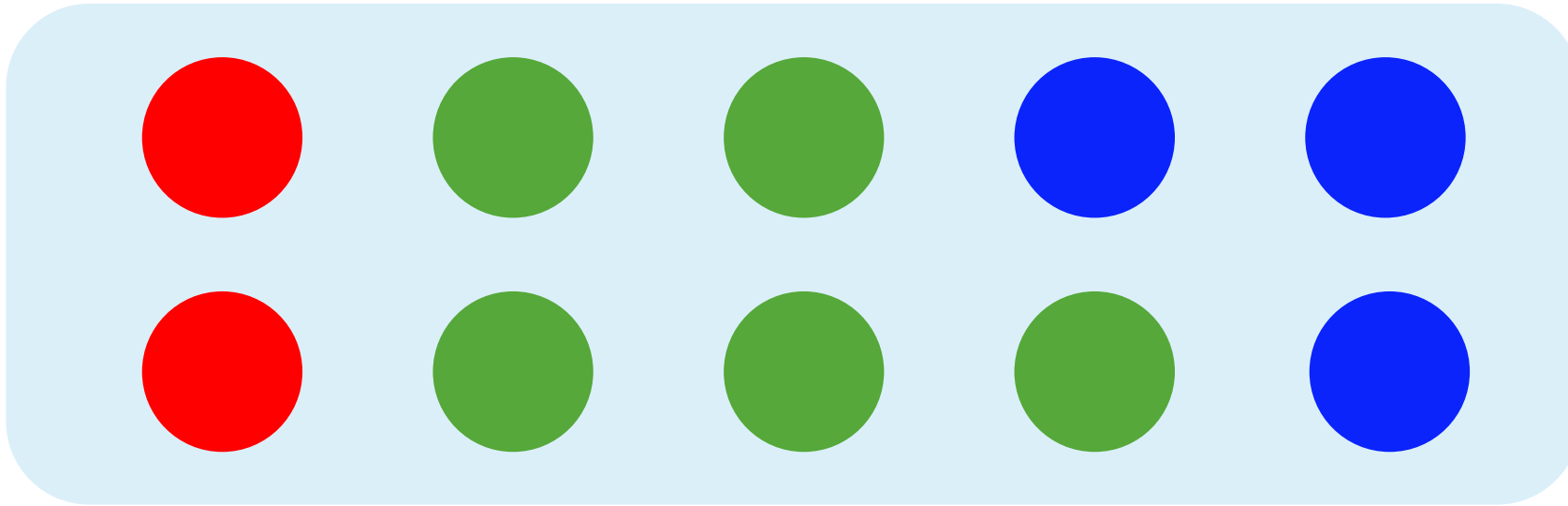
$$\mathbb{E}[f(X)] = \int_{\mathcal{X}} p(x) \cdot f(x) dx.$$

- For discrete distribution, the expectation of $f(X)$ is:

$$\mathbb{E}[f(X)] = \sum_{x \in \mathcal{X}} p(x) \cdot f(x).$$

Random Sampling

- There are 10 balls in the bin: 2 are red, 5 are green, and 3 are blue.
- Randomly sample a ball.
- What will be the color?



Random Sampling

- Sample red ball w.p. 0.2, green ball w.p. 0.5, and blue ball w.p. 0.3.
- Randomly sample a ball.
- What will be the color?

Random Sampling

- Sample red ball w.p. 0.2, green ball w.p. 0.5, and blue ball w.p. 0.3.
- Randomly sample a ball.
- What will be the color?

```
from numpy.random import choice
```

```
samples = choice(['R', 'G', 'B'], size=100, p=[0.2, 0.5, 0.3])  
print(samples)
```

```
['R' 'G' 'R' 'R' 'R' 'R' 'B' 'B' 'B' 'G' 'G' 'B' 'G' 'B' 'B' 'G' 'B' 'G'  
 'B' 'B' 'G' 'B' 'G' 'B' 'B' 'G' 'B' 'B' 'G' 'B' 'G' 'G' 'G' 'G' 'B'  
 'B' 'B' 'B' 'B' 'B' 'G' 'G' 'B' 'R' 'R' 'B' 'R' 'B' 'G' 'R' 'G' 'R' 'G'  
 'R' 'R' 'B' 'G' 'G' 'G' 'B' 'R' 'G' 'B' 'G' 'R' 'G' 'G' 'G' 'B' 'B' 'R'  
 'G' 'G' 'B' 'B' 'R' 'B' 'B' 'B' 'R' 'B' 'G' 'B' 'R' 'B' 'R' 'G' 'B' 'R'  
 'B' 'B' 'G' 'G' 'G' 'R' 'R' 'B' 'R' 'G']
```

目录

- 01 Probability theory
- 02 Terminology
- 03 Rewards and Returns
- 04 Value Functions
- 05 Play games using RL

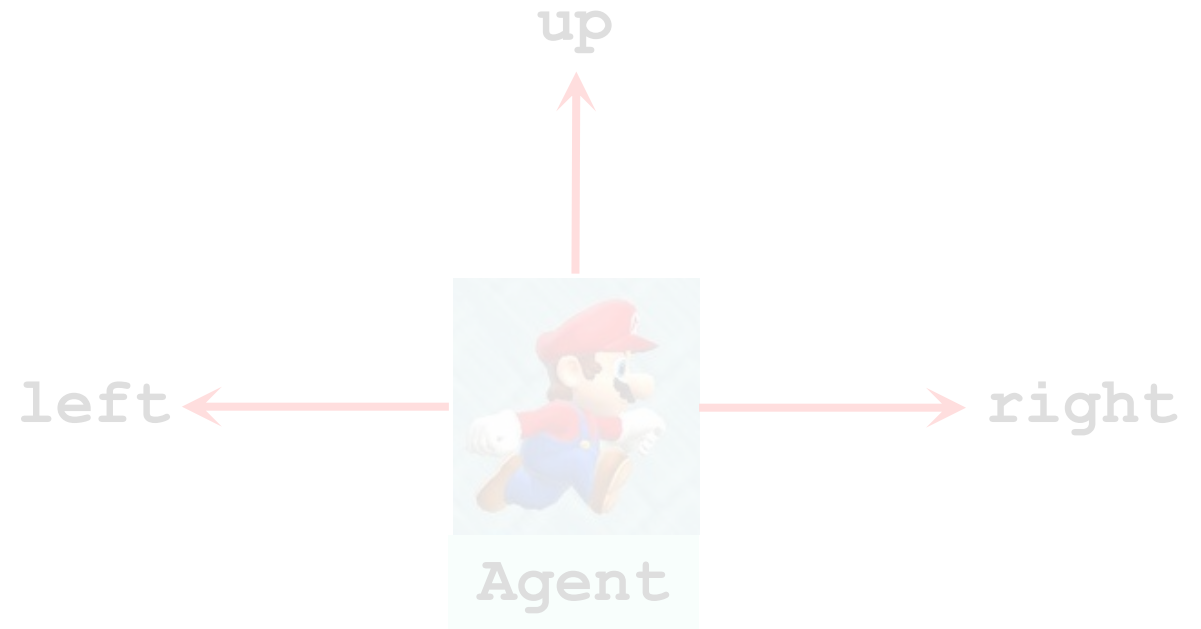
- 06 Value-based learning
- 07 Policy-based learning
- 08 Actor-critic method
- 09 Value Functions
- 10 Play games using RL

Terminology: state and action

state s (this frame)



Action $a \in \{\text{left, right, up}\}$

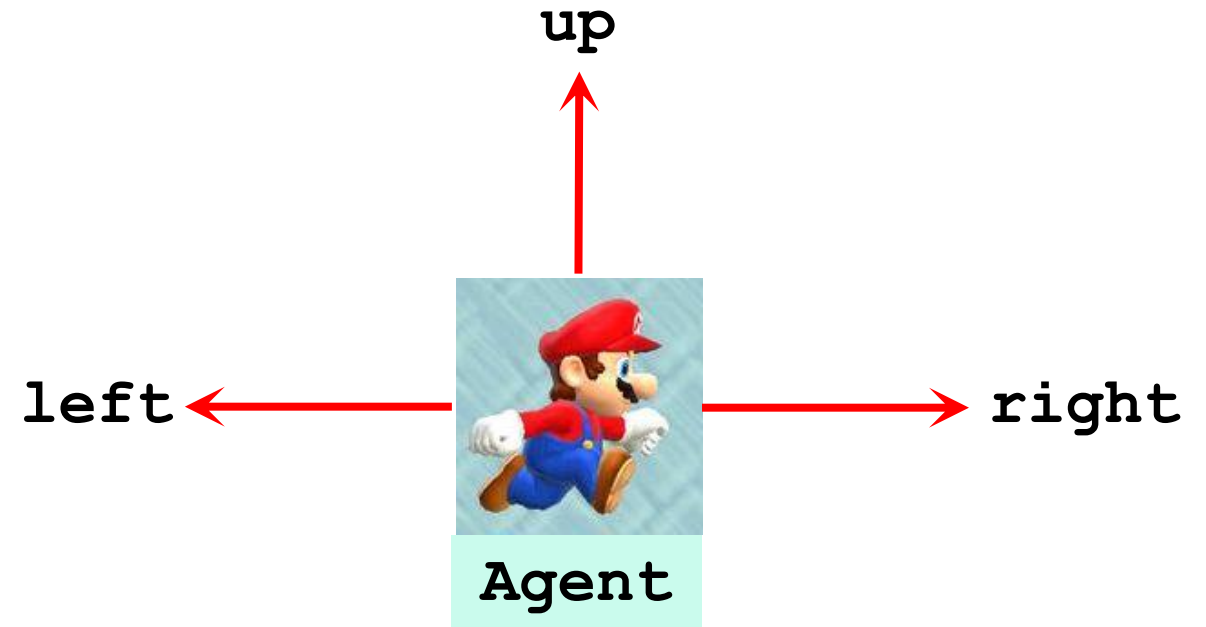


Terminology: state and action

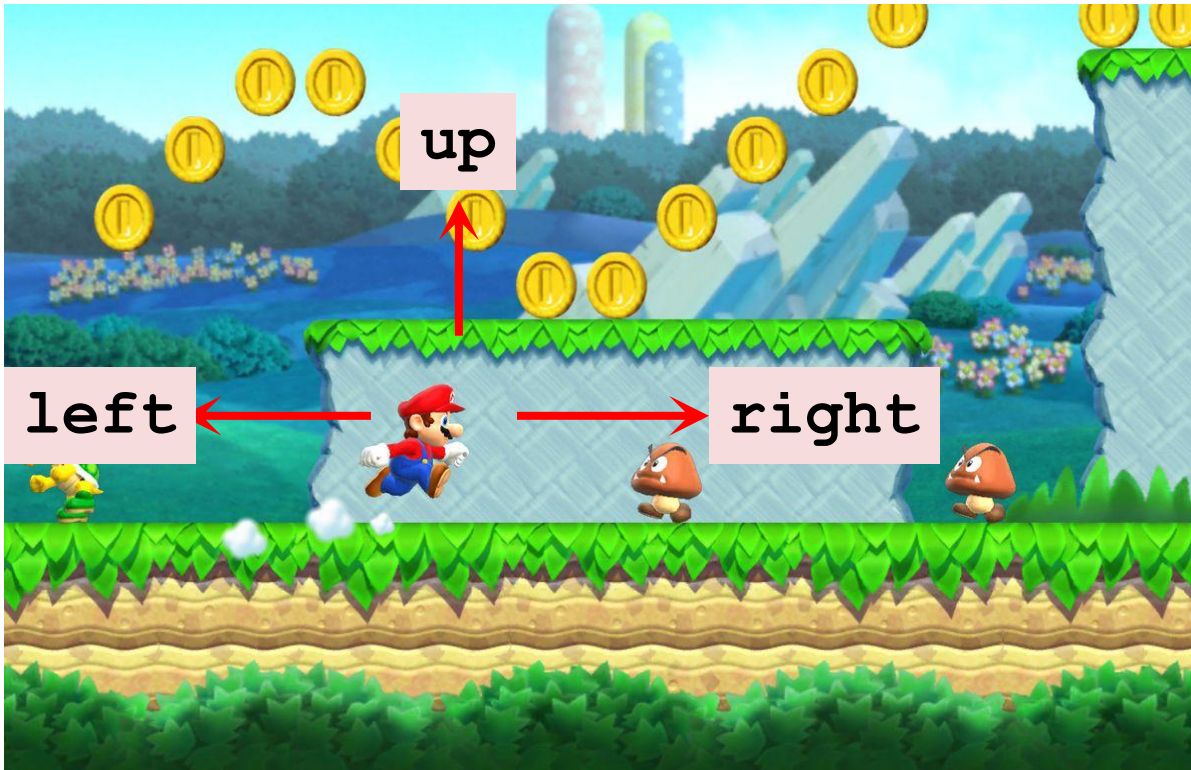
state s (this frame)



Action $a \in \{\text{left, right, up}\}$



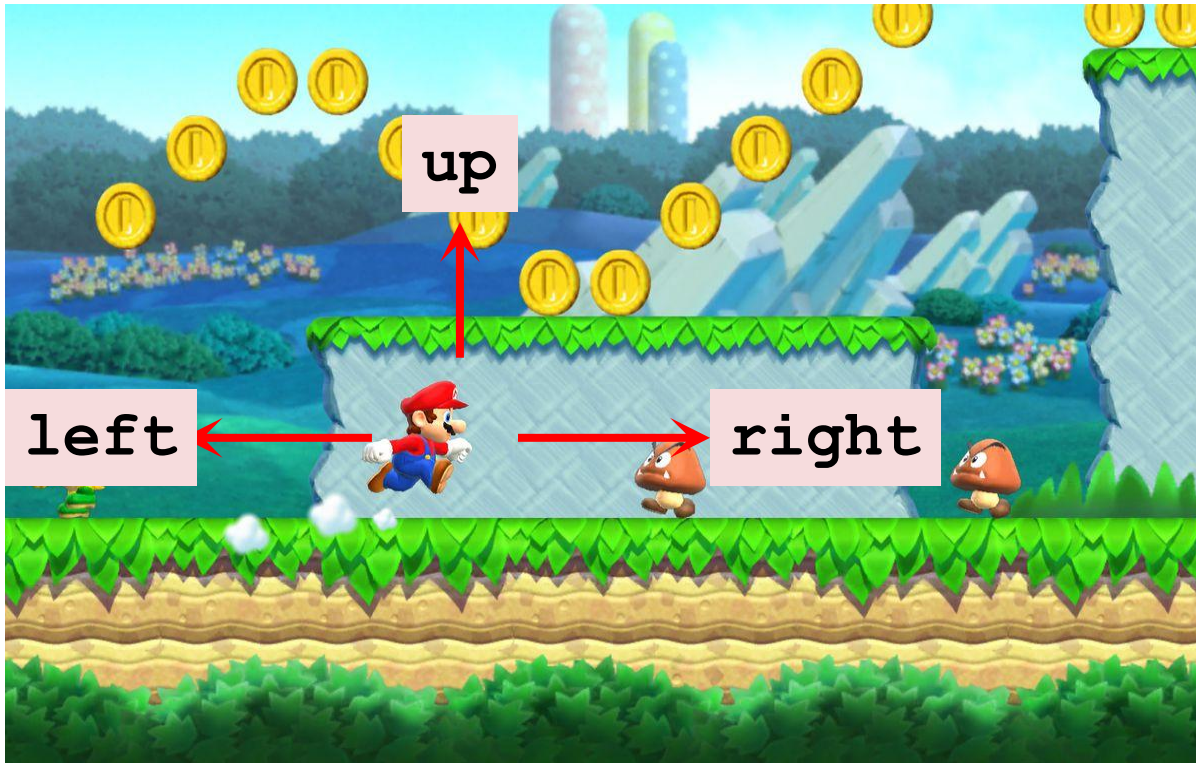
Terminology: policy



policy π

- Policy function $\pi: (s, a) \mapsto [0,1]$:
$$\pi(a | s) = \mathbb{P}(A = a | S = s).$$
- It is the probability of taking action $A = a$ given state s , e.g.,
 - $\pi(\text{left} | s) = 0.2,$
 - $\pi(\text{right} | s) = 0.1,$
 - $\pi(\text{up} | s) = 0.7.$
- Upon observing state $S = s$, the agent's action A can be random.

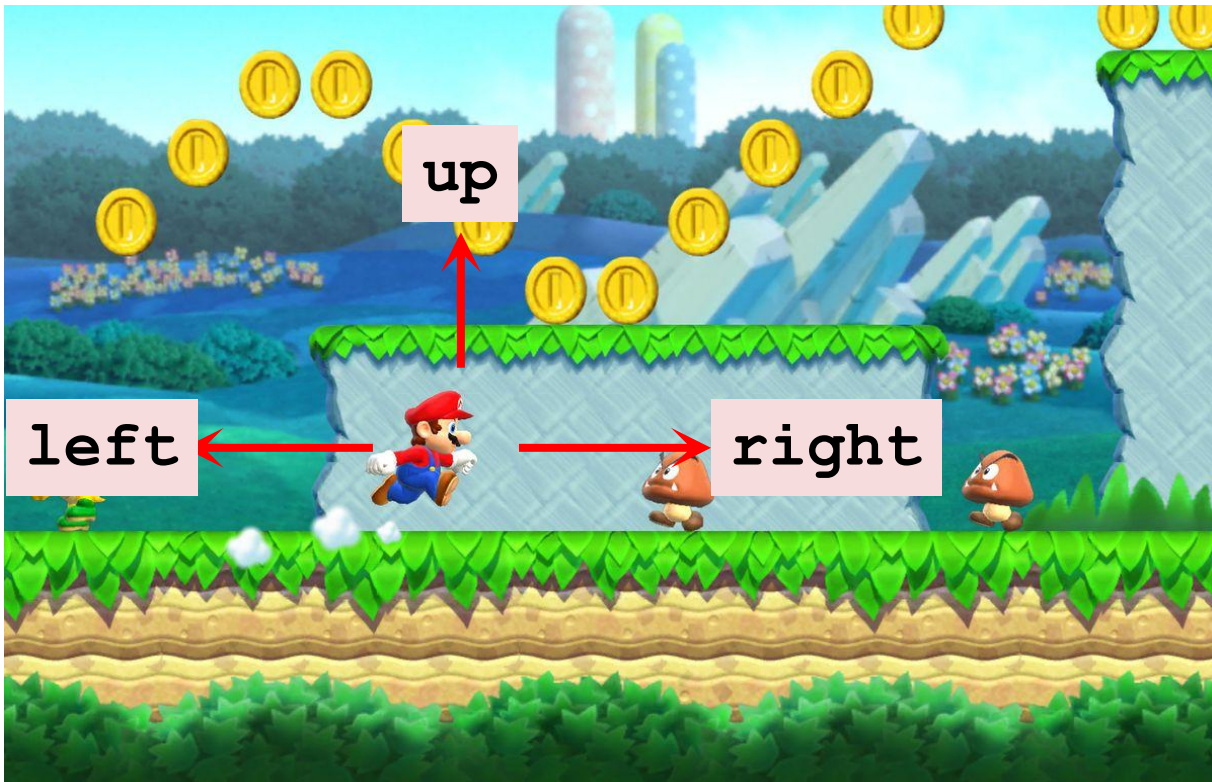
Terminology: policy



policy π

- Policy function $\pi: (s, a) \mapsto [0,1]$:
$$\pi(a | s) = \mathbb{P}(A = a | S = s).$$
- It is the probability of taking action $A = a$ given state s , e.g.,
 - $\pi(\text{left} | s) = 0.2,$
 - $\pi(\text{right} | s) = 0.1,$
 - $\pi(\text{up} | s) = 0.7.$
- Upon observing state $S = s$, the agent's action A can be random.

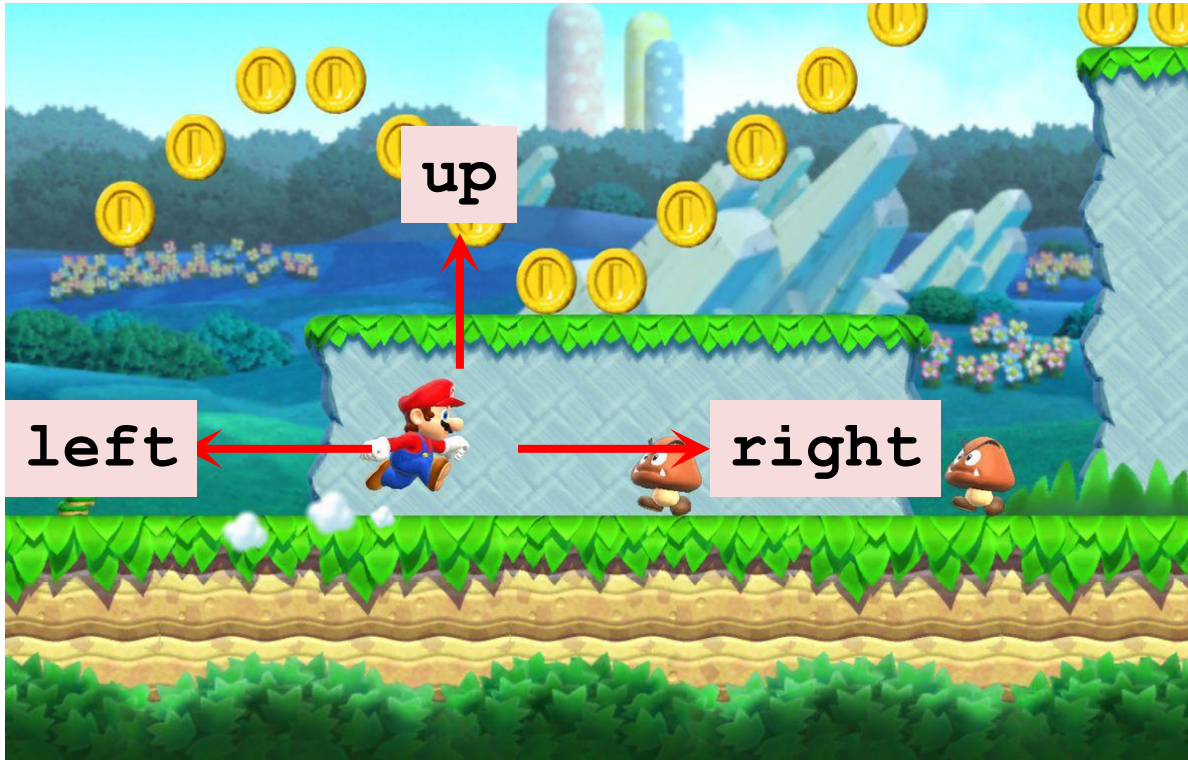
Terminology: policy



policy π

- Policy function $\pi: (s, a) \mapsto [0,1]$:
$$\pi(a | s) = \mathbb{P}(A = a | S = s).$$
- It is the probability of taking action $A = a$ given state s , e.g.,
 - $\pi(\text{left} | s) = 0.2$,
 - $\pi(\text{right} | s) = 0.1$,
 - $\pi(\text{up} | s) = 0.7$.
- Upon observing state $S = s$, the agent's action A can be random.

Terminology: policy



policy π

- Policy function $\pi: (s, a) \mapsto [0,1]$:
$$\pi(a | s) = \mathbb{P}(A = a | S = s).$$
- It is the probability of taking action $A = a$ given state s , e.g.,
 - $\pi(\text{left} | s) = 0.2$,
 - $\pi(\text{right} | s) = 0.1$,
 - $\pi(\text{up} | s) = 0.7$.
- Upon observing state $S = s$, the agent's action A can be random.

Terminology: reward

reward R

- Collect a coin: $R = +1$



Terminology: reward

reward R

- Collect a coin: $R = + 1$
- Win the game: $R = + 10000$



Terminology: reword

reward R

- Collect a coin: $R = +1$
- Win the game: $R = +10000$
- Touch a Goomba: $R = -10000$ (game over).



Terminology: reward



reward R

- Collect a coin: $R = +1$
- Win the game: $R = +10000$
- Touch a Goomba: $R = -10000$ (game over).
- Nothing happens: $R = 0$

Terminology: state transition



state transition

old state $\xrightarrow{\text{action}}$ new state

Terminology: state transition



state transition



- E.g., “up” action leads to a new state.

Terminology: state transition

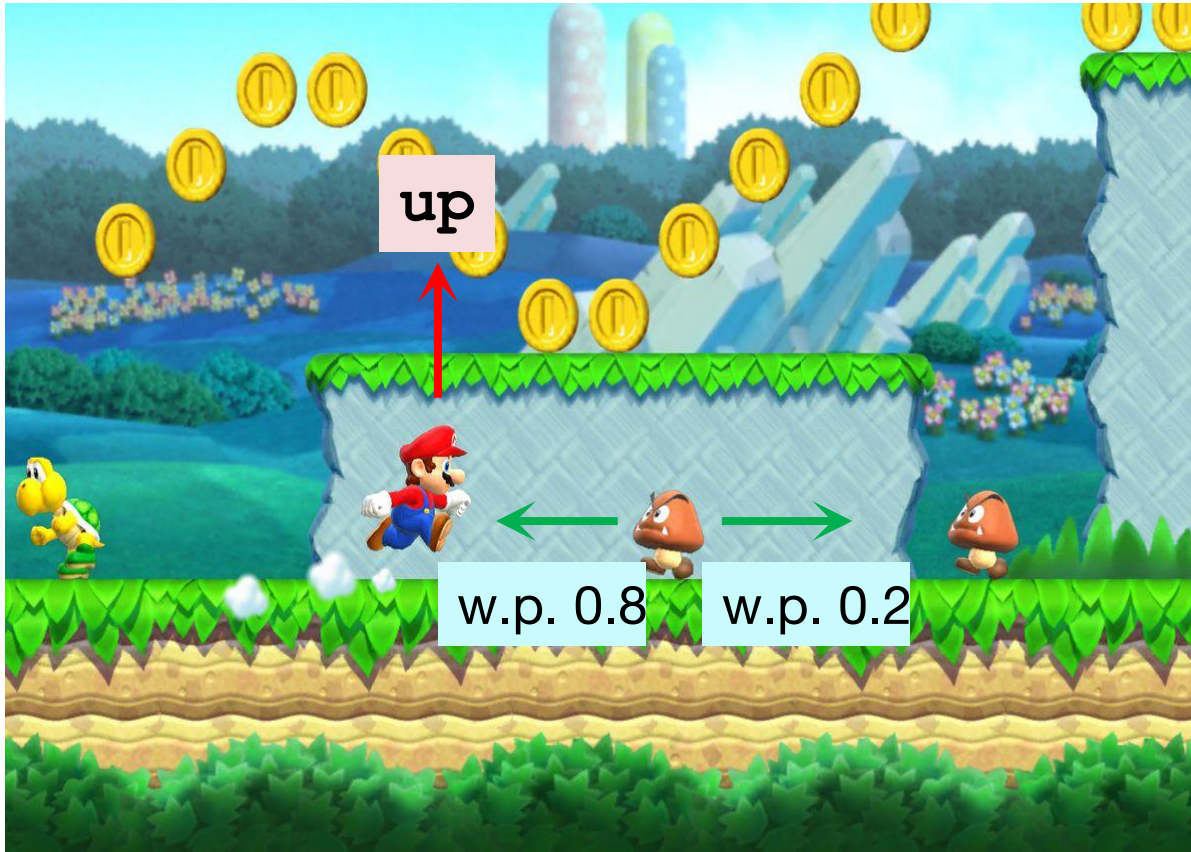


state transition



- E.g., “up” action leads to a new state.
- State transition can be random.
- Randomness is from the environment.

Terminology: state transition

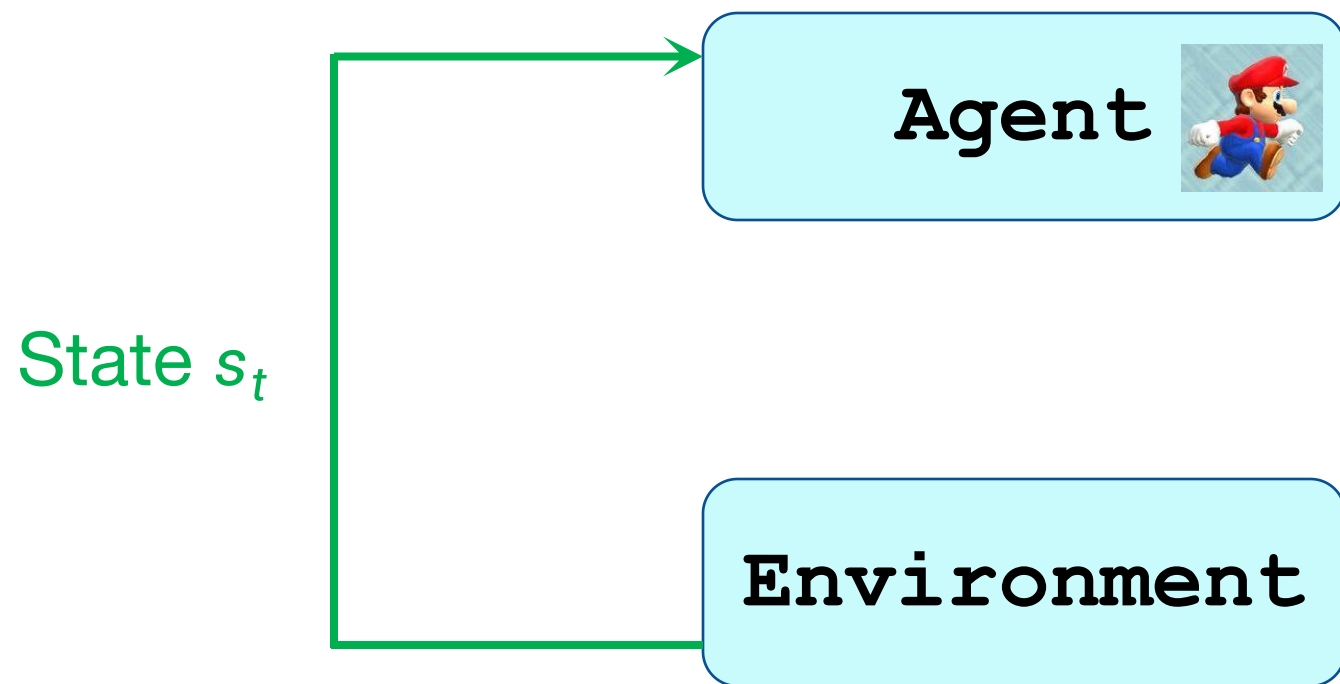


state transition

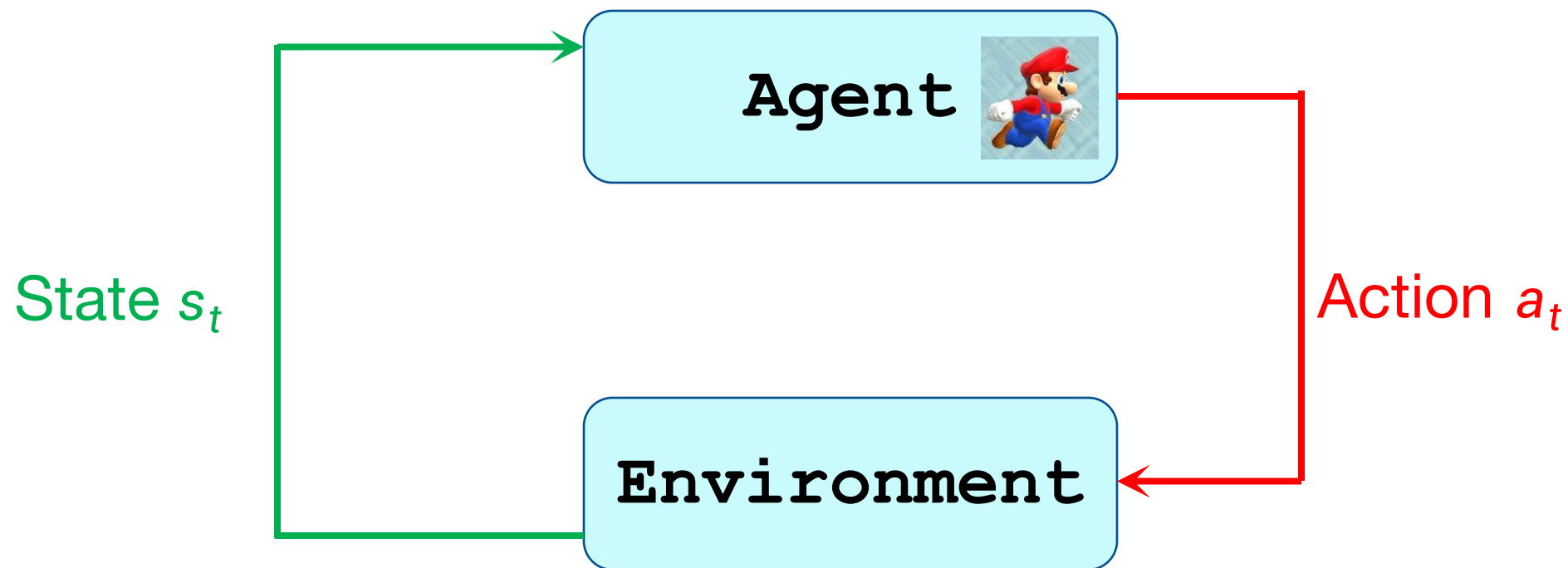


- E.g., “up” action leads to a new state.
- State transition can be random.
- Randomness is from the environment.
- $p(s'|s, a) = \mathbb{P}(S' = s'|S = s, A = a)$.

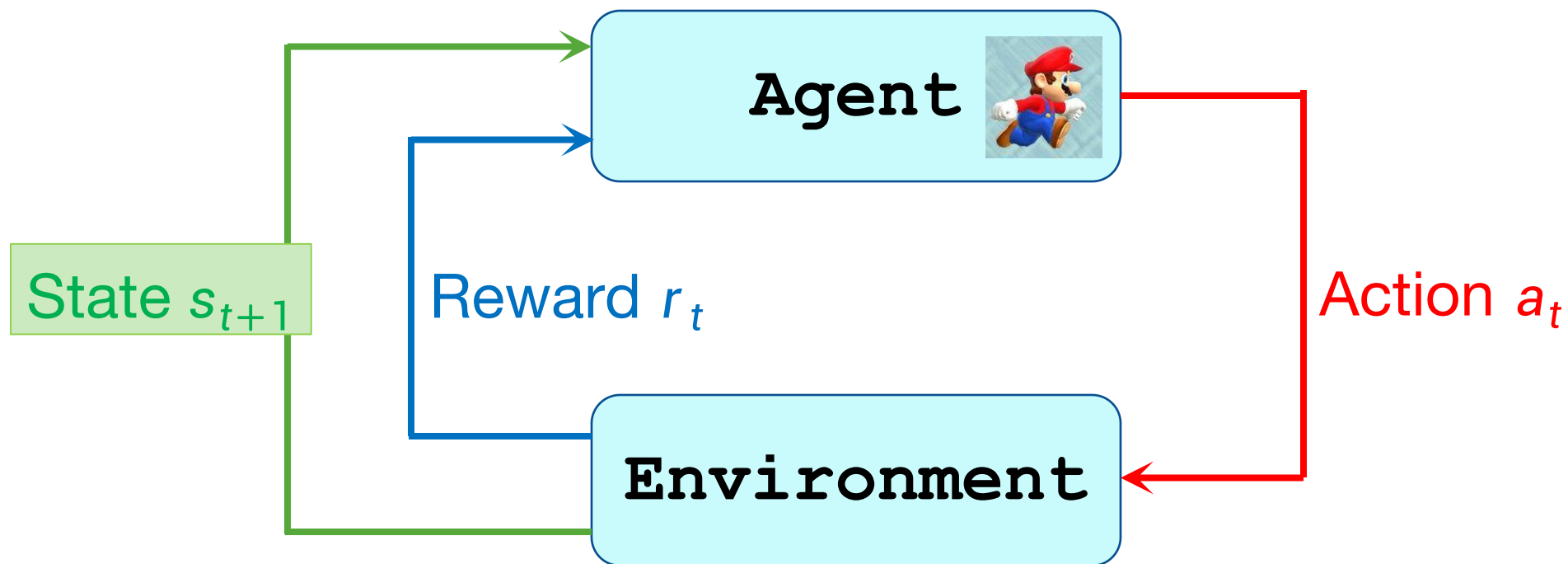
Terminology: agent environment interaction



Terminology: agent environment interaction



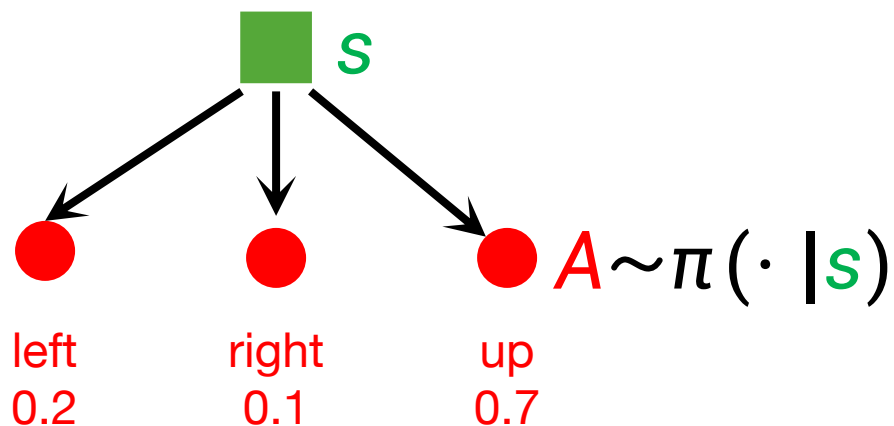
Terminology: agent environment interaction



Randomness in Reinforcement Learning

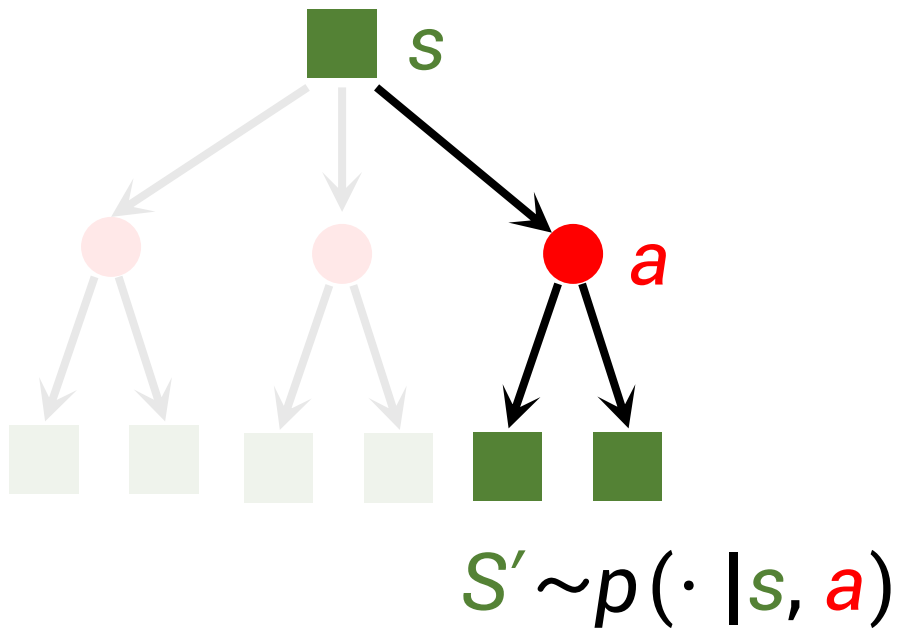
Actions have randomness.

- Given state s , the action can be random, e.g., .



- $\pi(\text{"left"}|s) = 0.2,$
- $\pi(\text{"right"}|s) = 0.1,$
- $\pi(\text{"up"}|s) = 0.7.$

Randomness in Reinforcement Learning



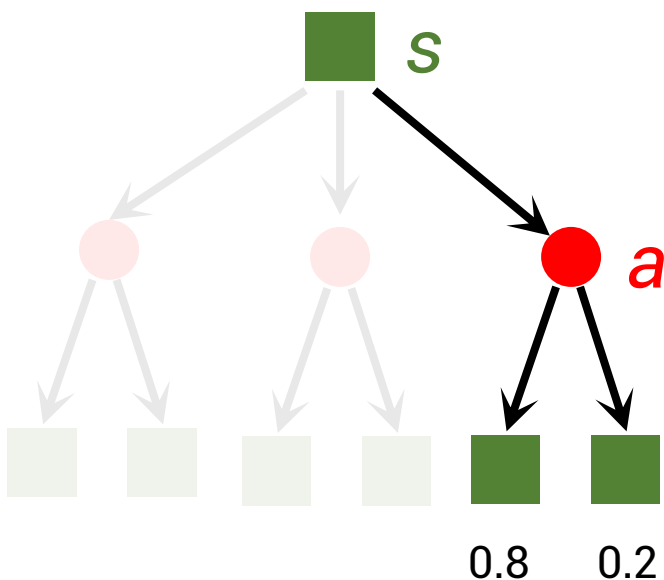
Actions have randomness.

- Given state s , the action can be random, e.g., .
 - $\pi(\text{"left"}|s) = 0.2$,
 - $\pi(\text{"right"}|s) = 0.1$,
 - $\pi(\text{"up"}|s) = 0.7$.

State transitions have randomness.

- Given state $S = s$ and action $A = a$, the environment randomly generates a new state S' .

Randomness in Reinforcement Learning



Actions have randomness.

- Given state s , the action can be random, e.g., .
 - $\pi(\text{"left"}|s) = 0.2$,
 - $\pi(\text{"right"}|s) = 0.1$,
 - $\pi(\text{"up"}|s) = 0.7$.

State transitions have randomness.

- Given state $S = s$ and action $A = a$, the environment randomly generates a new state S' .

Play the game using AI



- Observe a frame (state s_1)
- → Make action a_1 (left, right, or up)
- → Observe a new frame (state s_2) and reward r_1
- → Make action a_2
- → ...

Play the game using AI



- Observe a frame (state s_1)
- $\rightarrow$ Make action a_1 (left, right, or up)
- $\rightarrow$ Observe a new frame (state s_2) and reward r_1
- $\rightarrow$ Make action a_2
- $\rightarrow$...
- (state, action, reward) trajectory:
 $s_1, a_1, r_1, s_2, a_2, r_2, \dots, s_T, a_T, r_T.$

目录

- 01 Probability theory
- 02 Terminology
- 03 **Rewords and Returns**
- 04 Value Functions
- 05 Play games using RL

- 06 Value-based learning
- 07 Policy-based learning
- 08 Actor-critic method
- 09 Value Functions
- 10 Play games using RL

Definition: Return (aka cumulative future reward).

- $U_t = R_t + R_{t+1} + R_{t+2} + R_{t+3} + \dots$

Definition: Return (aka cumulative future reward).

- $U_t = R_t + R_{t+1} + R_{t+2} + R_{t+3} + \dots$

Question: Are R_t and R_{t+1} equally important?

- Which of the followings do you prefer?
 - I give you \$100 right now.
 - I will give you \$100 one year later.

Definition: Return (aka cumulative future reward).

- $U_t = R_t + R_{t+1} + R_{t+2} + R_{t+3} + \dots$

Question: Are R_t and R_{t+1} equally important?

- Which of the followings do you prefer?
 - I give you \$100 right now.
 - I will give you \$100 one year later.
- Future reward is less valuable than present reward.
- R_{t+1} should be given less weight than R_t .

Definition: Return (aka cumulative future reward).

- $U_t = R_t + R_{t+1} + R_{t+2} + R_{t+3} + \dots$

Definition: Discounted return (cumulative discounted future reward)

- γ : discount rate (tuning hyper-parameter).

- $U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \gamma^3 R_{t+3} + \dots$

Randomness in Returns

Definition: Discounted return (at time step t).

$$U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \gamma^3 R_{t+3} + \dots$$

At time step t , the return U_t is **random**.

• Two sources of randomness:

1. Action can be random: $\mathbb{P}[A = a \mid S = s] = \pi(a|s)$.
2. New state can be random: $\mathbb{P}[S' = s' \mid S = s, A = a] = p(s'|s, a)$.

Randomness in Returns

Definition: Discounted return (at time step t).

$$U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \gamma^3 R_{t+3} + \dots$$

At time step t , the return U_t is **random**.

• Two sources of randomness:

1. Action can be random: $\mathbb{P}[A = a | S = s] = \pi(a|s)$.
2. New state can be random: $\mathbb{P}[S' = s' | S = s, A = a] = p(s'|s, a)$.

• For any $i \geq t$, the reward R_i depends on S_i and A_i .

• Thus, given s_t , the return U_t depends on the random variables:

$$A_t, A_{t+1}, A_{t+2}, \dots \text{ and } S_{t+1}, S_{t+2}, \dots$$

目录

- 01 Probability theory
- 02 Terminology
- 03 Rewards and Returns
- 04 **Value Functions**
- 05 Play games using RL

- 06 Value-based learning
- 07 Policy-based learning
- 08 Actor-critic method
- 09 Value Functions
- 10 Play games using RL

Action-Value Function $Q(s, a)$

Definition: Discounted return (cumulative discounted future reward)

- $U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \gamma^3 R_{t+3} + \dots$

Action-Value Function $Q(s, a)$

Definition: Discounted return (cumulative discounted future reward)

- $U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \gamma^3 R_{t+3} + \dots$

Definition: Action-value function for policy π .

- $Q_\pi(s_t, a_t) = \mathbb{E}[U_t | S_t = s_t, A_t = a_t].$



- Return U_t depends on states $s_t, s_{t+1}, s_{t+2}, \dots$ and actions $a_t, a_{t+1}, a_{t+2}, \dots$.

Action-Value Function $Q(s, a)$

Definition: Discounted return (cumulative discounted future reward)

- $U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \gamma^3 R_{t+3} + \dots$

Definition: Action-value function for policy π .

- $Q_\pi(s_t, a_t) = \mathbb{E}[U_t | S_t = s_t, A_t = a_t].$



- Return U_t depends on states $s_t, s_{t+1}, s_{t+2}, \dots$ and actions $a_t, a_{t+1}, a_{t+2}, \dots$.
- Actions are random: $\mathbb{P}[A = a | S = s] = \pi(a|s).$ (Policy function.)
- States are random: $\mathbb{P}[S' = s' | S = s, A = a] = p(s'|s, a).$ (State transition.)

Action-Value Function $Q(s, a)$

Definition: Discounted return (cumulative discounted future reward)

- $U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \gamma^3 R_{t+3} + \dots$

Definition: Action-value function for policy π .

- $Q_\pi(s_t, a_t) = \mathbb{E}[U_t | S_t = s_t, A_t = a_t]$.

Definition: Optimal action-value function.

- $Q^*(s_t, a_t) = \max_{\pi} Q_\pi(s_t, a_t)$.

State-Value Function $V(s)$

Definition: Discounted return (cumulative discounted future reward)

$$\bullet U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \gamma^3 R_{t+3} + \dots$$

Definition: Action-value function for policy π .

$$\bullet Q_\pi(s_t, a_t) = \mathbb{E}[U_t | S_t = s_t, A_t = a_t].$$

Definition: State-value function.

$$\bullet V_\pi(s_t) = \mathbb{E}_A[Q_\pi(s_t, A)]$$

State-Value Function $V(s)$

Definition: Discounted return (cumulative discounted future reward)

- $U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \gamma^3 R_{t+3} + \dots$

Definition: Action-value function for policy π .

- $Q_\pi(s_t, a_t) = \mathbb{E}[U_t | S_t = s_t, A_t = a_t]$.

Definition: State-value function.

- $V_\pi(s_t) = \mathbb{E}_A [Q_\pi(s_t, A)] = \sum_a \pi(a|s_t) \cdot Q_\pi(s_t, a)$. (Actions are discrete.)

Taken w.r.t. the action $A \sim \pi(\cdot|s_t)$.

State-Value Function $V(s)$

Definition: Discounted return (cumulative discounted future reward)

$$\bullet U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \gamma^3 R_{t+3} + \dots$$

Definition: Action-value function for policy π .

$$\bullet Q_\pi(s_t, a_t) = \mathbb{E}[U_t | S_t = s_t, A_t = a_t].$$

Definition: State-value function.

- $\bullet V_\pi(s_t) = \mathbb{E}_A [Q_\pi(s_t, A)] = \sum_a \pi(a|s_t) \cdot Q_\pi(s_t, a).$ (Actions are discrete.)
- $\bullet V_\pi(s_t) = \mathbb{E}_A [Q_\pi(s_t, A)] = \int \pi(a|s_t) \cdot Q_\pi(s_t, a) da.$ (Actions are continuous.)

Understanding the Value Functions

- **Action**-value function: $Q_{\pi}(s_t, a_t) = \mathbb{E}[U_t | S_t = s_t, A_t = a_t]$.
- Given policy π , $Q_{\pi}(s, a)$ evaluates how good it is for an agent to pick action a while being in state s .

Understanding the Value Functions

- **Action**-value function: $Q_{\pi}(s_t, a_t) = \mathbb{E}[U_t | S_t = s_t, A_t = a_t]$.
- Given policy π , $Q_{\pi}(s, a)$ evaluates how good it is for an agent to pick action a while being in state s .
- **State**-value function: $V_{\pi}(s) = \mathbb{E}_A[Q_{\pi}(s, A)]$
- For fixed policy π , $V_{\pi}(s)$ evaluates how good the situation is in state s .
- $\mathbb{E}_s[V_{\pi}(s)]$ evaluates how good the policy π is.

目录

- 01 Probability theory
- 02 Terminology
- 03 Rewards and Returns
- 04 Value Functions
- 05 Play games using RL

- 06 Value-based learning
- 07 Policy-based learning
- 08 Actor-critic method
- 09 Value Functions
- 10 Play games using RL

How does AI control the agent?

Suppose we have a good policy $\pi(a|s)$.

- Upon observing the state s_t ,
- random sampling: $a_t \sim \pi(\cdot|s_t)$.

How does AI control the agent?

Suppose we have a good policy $\pi(a|s)$.

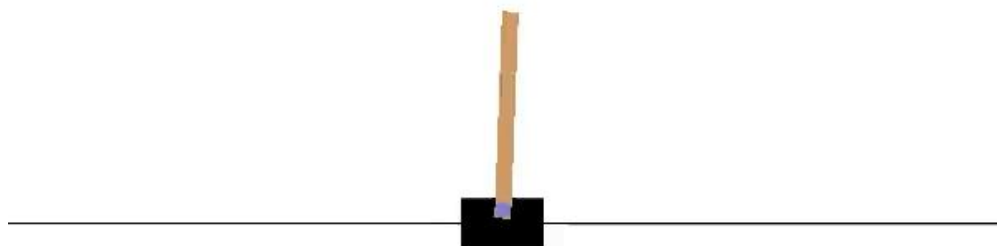
- Upon observing the state s_t ,
- random sampling: $a_t \sim \pi(\cdot|s_t)$.

Suppose we know the optimal action-value function $Q^*(s, a)$.

- Upon observe the state s_t ,
- choose the **action** that maximizes the value: $a_t = \operatorname{argmax}_a Q^*(s_t, a)$.

- Gym is a toolkit for developing and comparing reinforcement learning algorithms.
- <https://gym.openai.com/>

Classical control problems



Cart Pole



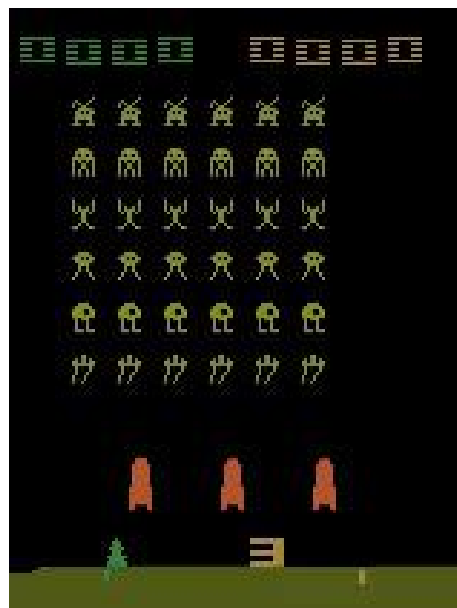
Pendulum

- Gym is a toolkit for developing and comparing reinforcement learning algorithms.
- <https://gym.openai.com/>

Atari Games



Pong



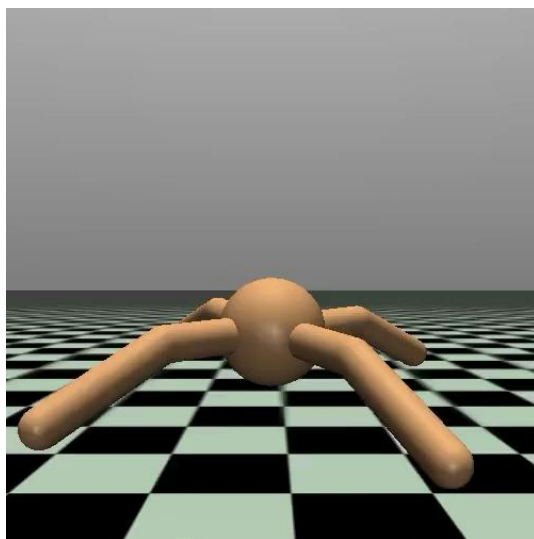
Space Invader



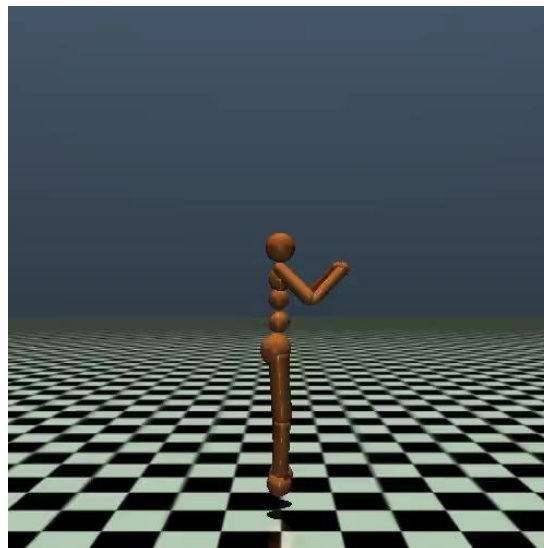
Breakout

- Gym is a toolkit for developing and comparing reinforcement learning algorithms.
- <https://gym.openai.com/>

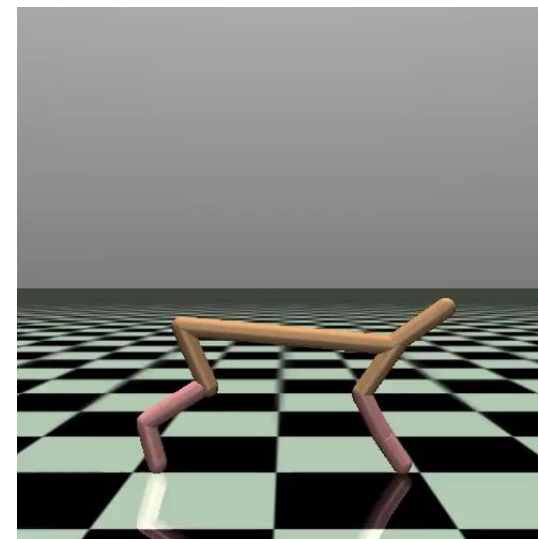
MuJoCo (Continuous control tasks.)



Ant



Humanoid



Half Cheetah

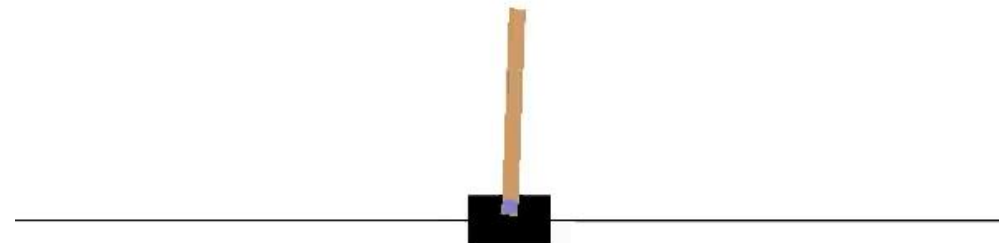


Play CartPole Game

```
import gym
```

```
env = gym.make('CartPole-v0')
```

- Get the environment of CartPole from Gym.
- “env” provides states and reward.



Play CartPole Game

```
state = env.reset()
```

```
for t in range(100):
```

A window pops up rendering CartPole.

```
    env.render()
```

```
    print(state)
```

A random action.

```
    action = env.action_space.sample()
```

```
    state, reward, done, info = env.step(action)
```

```
if done:
```

"done=1" means finished (win or lose the game)

```
    print('Finished')
```

```
    break
```

```
env.close()
```

Terminologies

- Agent 
- Environment
- State s .
- Action a .
- Reward r .
- Policy $\pi(a|s)$
- State transition $p(s'|s, a)$.

Terminologies

- Agent 
- Environment
- State s .
- Action a .
- Reward r .
- Policy $\pi(a|s)$
- State transition $p(s'|s, a)$.

Return and Value

- Return:

$$U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots$$

- Action-value function:

$$Q_\pi(s_t, a_t) = \mathbb{E}[U_t | s_t, a_t].$$

- Optimal action-value function:

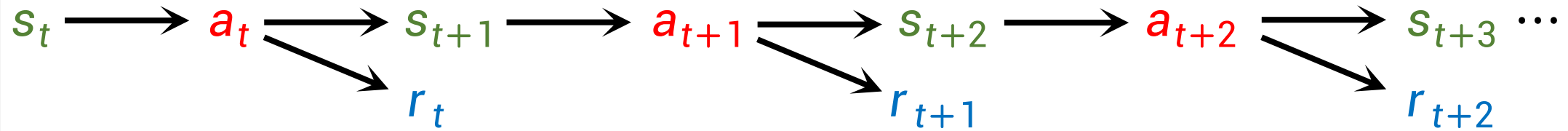
$$Q^*(s_t, a_t) = \max_{\pi} Q_\pi(s_t, a_t).$$

- State-value function:

$$V_\pi(s_t) = \mathbb{E}_A[Q_\pi(s_t, A)].$$

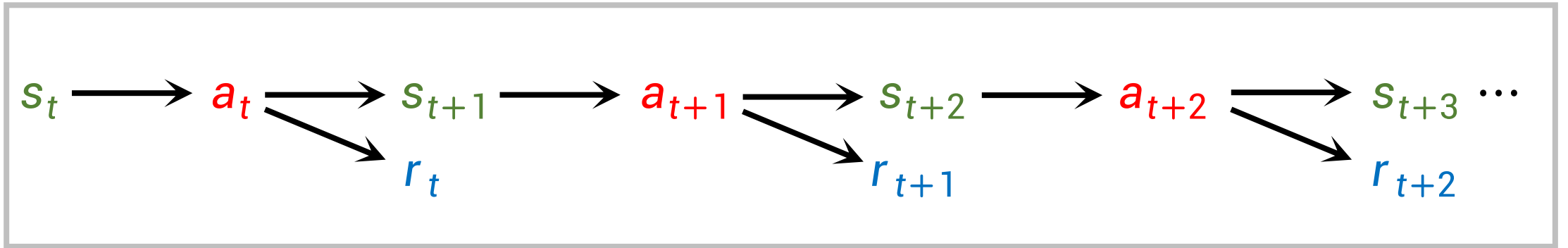
Play game using reinforcement learning

- Observe state s_t , make action a_t , environment gives s_{t+1} and reward r_t .



Play game using reinforcement learning

- Observe state s_t , make action a_t , environment gives s_{t+1} and reward r_t .



- The agent can be controlled by either $\pi(a|s)$ or $Q^*(s, a)$.

目录

- 01 Probability theory
- 02 Terminology
- 03 Rewards and Returns
- 04 Value Functions
- 05 Play games using RL

- 06 Value-based learning
- 07 Policy-based learning
- 08 Actor-critic method
- 09 Value Functions
- 10 Play games using RL

Action-Value Functions

Definition: Discounted return (cumulative discounted future reward)

- $U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \gamma^3 R_{t+3} + \dots$



- The return depends on actions $A_t, A_{t+1}, A_{t+2}, \dots$ and states $S_t, S_{t+1}, S_{t+2}, \dots$
- Actions are random: $\mathbb{P}[A = a \mid S = s] = \pi(a|s)$. (Policy function.)
- States are random: $\mathbb{P}[S' = s' \mid S = s, A = a] = p(s'|s, a)$. (State transition.)

Action-Value Functions $Q(s, a)$

Definition: Discounted return (cumulative discounted future reward)

- $U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \gamma^3 R_{t+3} + \dots$

Definition: Action-value function for policy π .

- $Q_\pi(s_t, a_t) = \mathbb{E}[U_t | S_t = s_t, A_t = a_t].$



- Taken w.r.t. actions $A_{t+1}, A_{t+2}, A_{t+3}, \dots$ and states $S_{t+1}, S_{t+2}, S_{t+3}, \dots$
- Integrate out everything except for the observations: $A_t = a_t$ and $S_t = s_t$.

Action-Value Functions $Q(s, a)$

Definition: Discounted return (cumulative discounted future reward)

$$\bullet U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \gamma^3 R_{t+3} + \dots$$

Definition: Action-value function for policy π .

$$\bullet Q_\pi(s_t, a_t) = \mathbb{E}[U_t | S_t = s_t, A_t = a_t].$$

Definition: Optimal action-value function.

$$\bullet Q^*(s_t, a_t) = \max_{\pi} Q_\pi(s_t, a_t).$$

- Whatever policy function π is used, the result of taking a_t at state s_t cannot be better than $Q^*(s_t, a_t)$.

Approximate the Q Function

Goal: Win the game ($\approx$ maximize the total reward.)

Question: If we know $Q^*(s, a)$, what is the best action?

Approximate the Q Function

Goal: Win the game ($\approx$ maximize the total reward.)

Question: If we know $Q^*(s, a)$, what is the best action?

- Obviously, the best action is $a^* = \operatorname{argmax}_a Q^*(s, a)$.



Q^* is an indicator of how good it is for an agent to pick action a while being in state s .

Approximate the Q Function

Goal: Win the game ($\approx$ maximize the total reward.)

Question: If we know $Q^*(s, a)$, what is the best action?

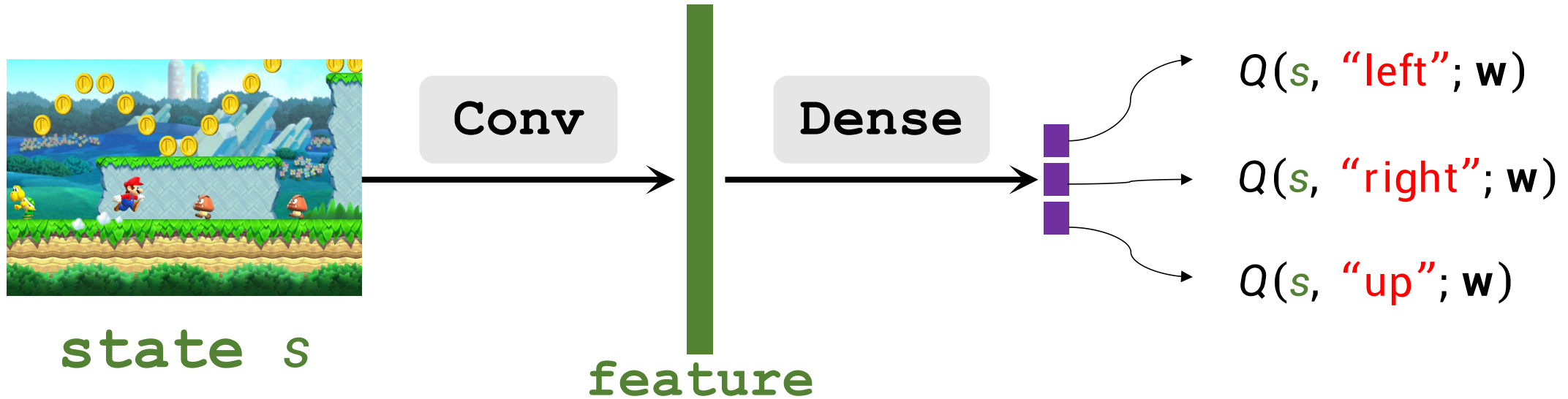
- Obviously, the best action is $a^* = \underset{a}{\operatorname{argmax}} Q^*(s, a)$.

Challenge: We do not know $Q^*(s, a)$.

- Solution: Deep Q Network (DQN)
- Use neural network $Q(s, a; \mathbf{w})$ to approximate $Q^*(s, a)$.

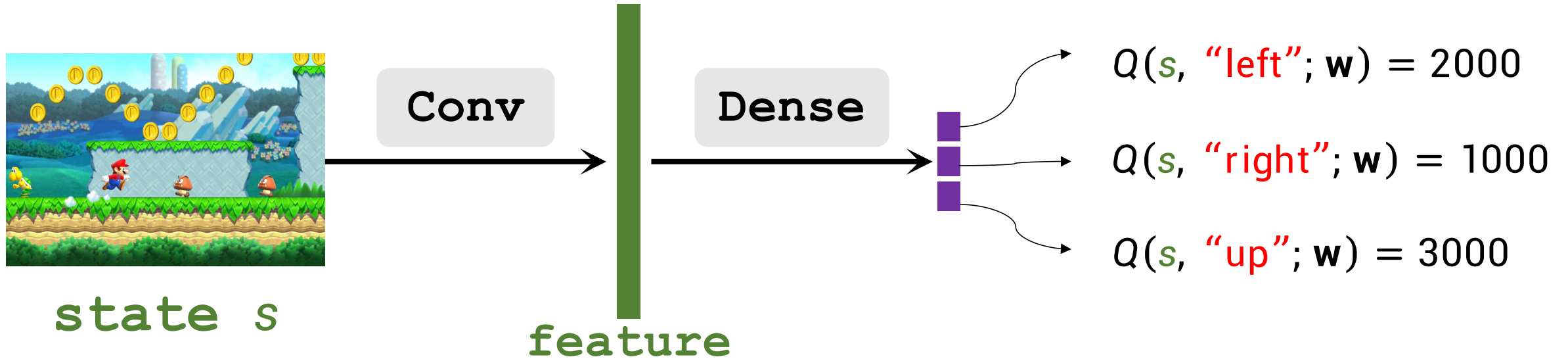
Deep Q Network (DQN)

- Input shape: size of the screenshot.
- Output shape: dimension of action space.



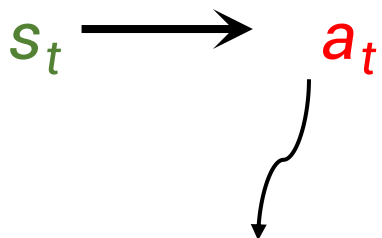
Deep Q Network (DQN)

- Input shape: size of the screenshot.
- Output shape: dimension of action space.



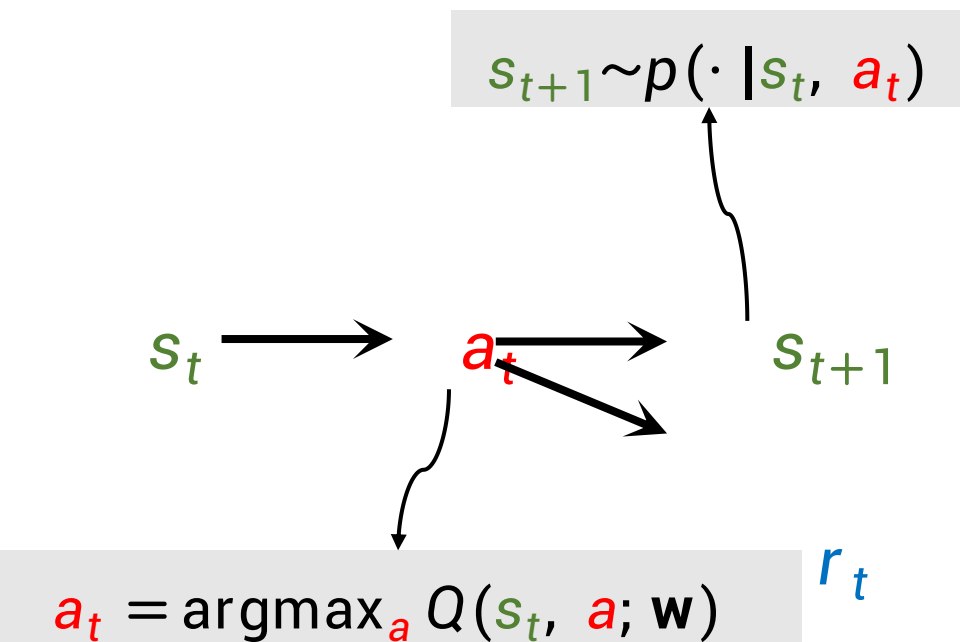
Question: Based on the predictions, what should be the **action**?

Apply DQN to Play Game

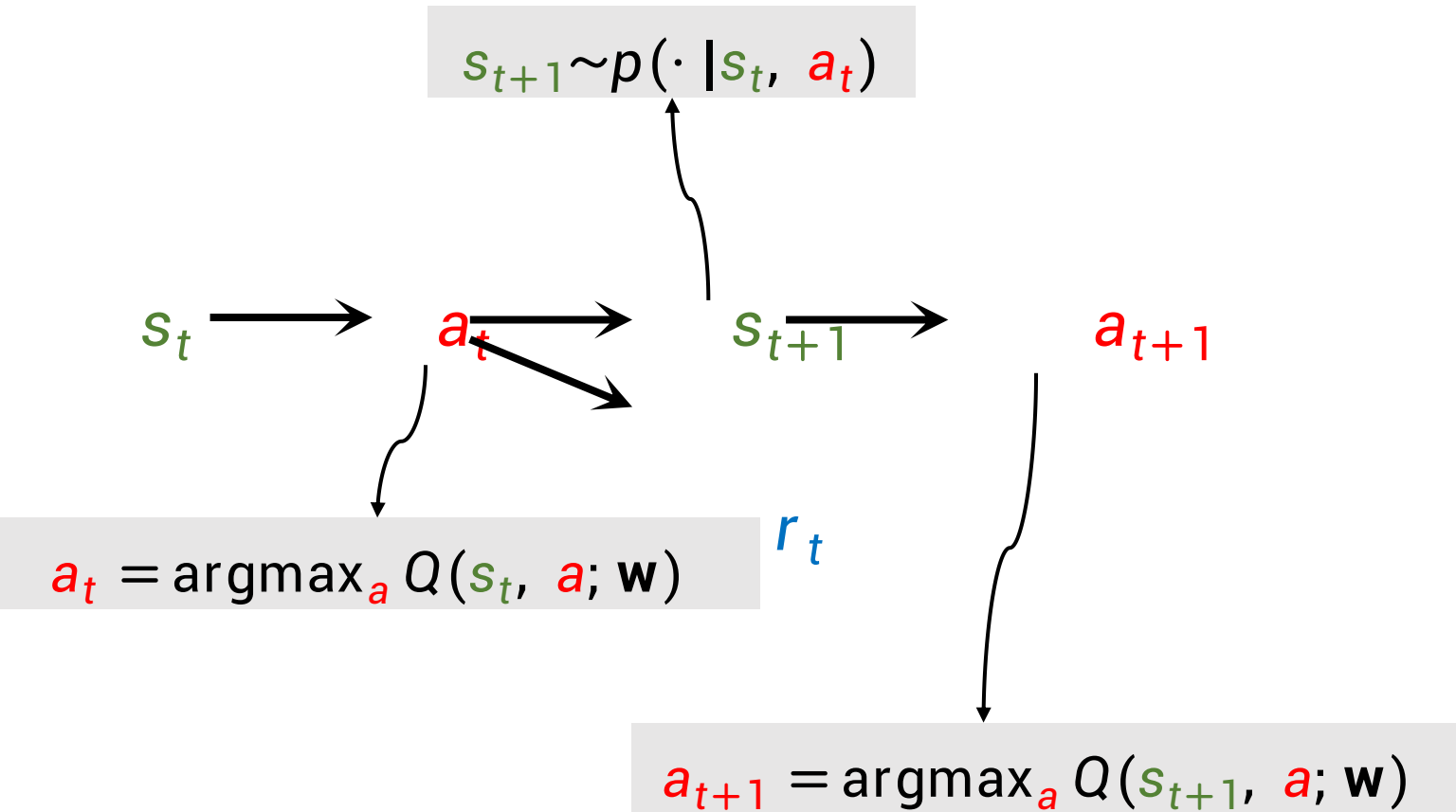


$$a_t = \operatorname{argmax}_a Q(s_t, a; \mathbf{w})$$

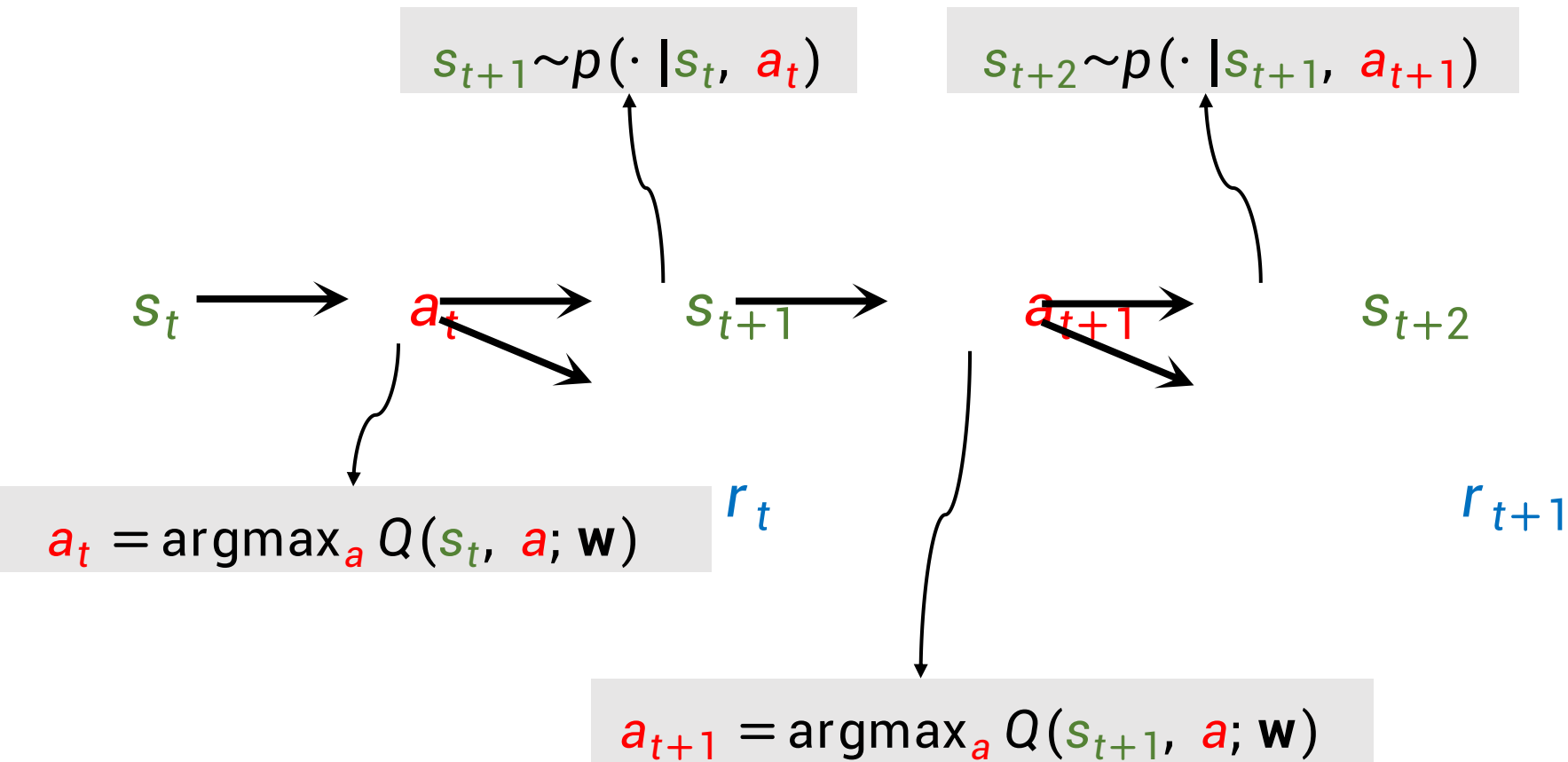
Apply DQN to Play Game



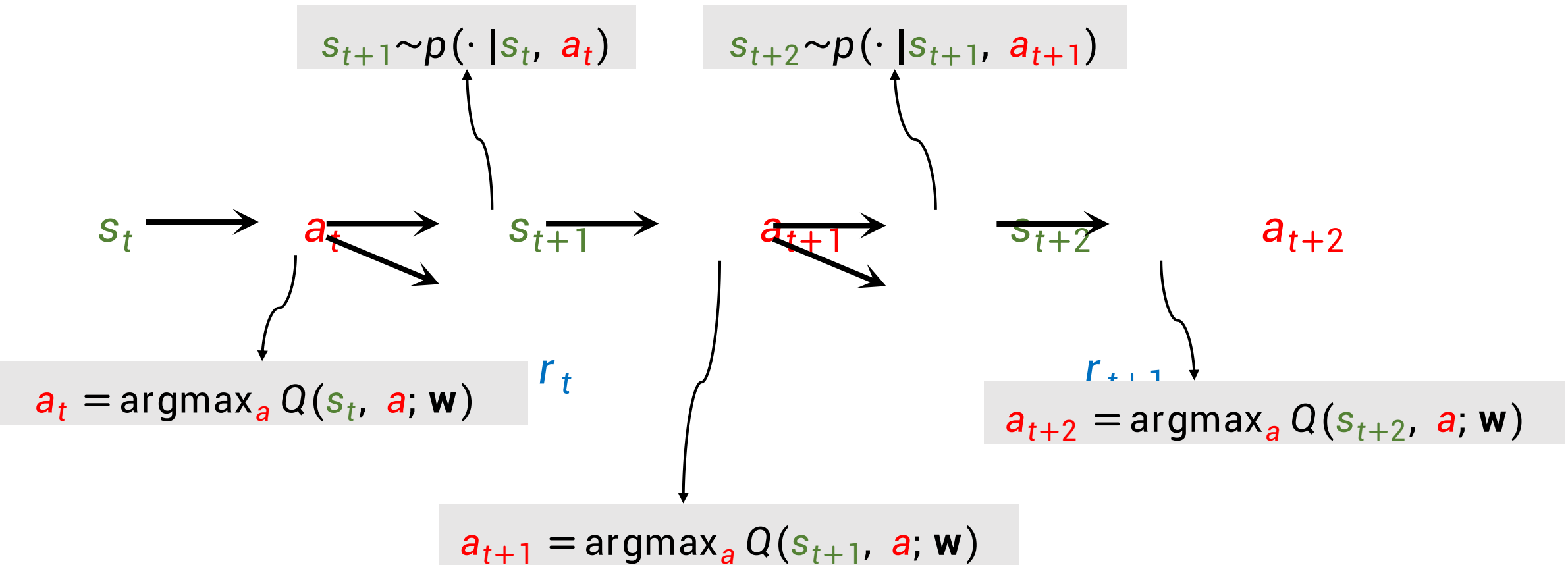
Apply DQN to Play Game



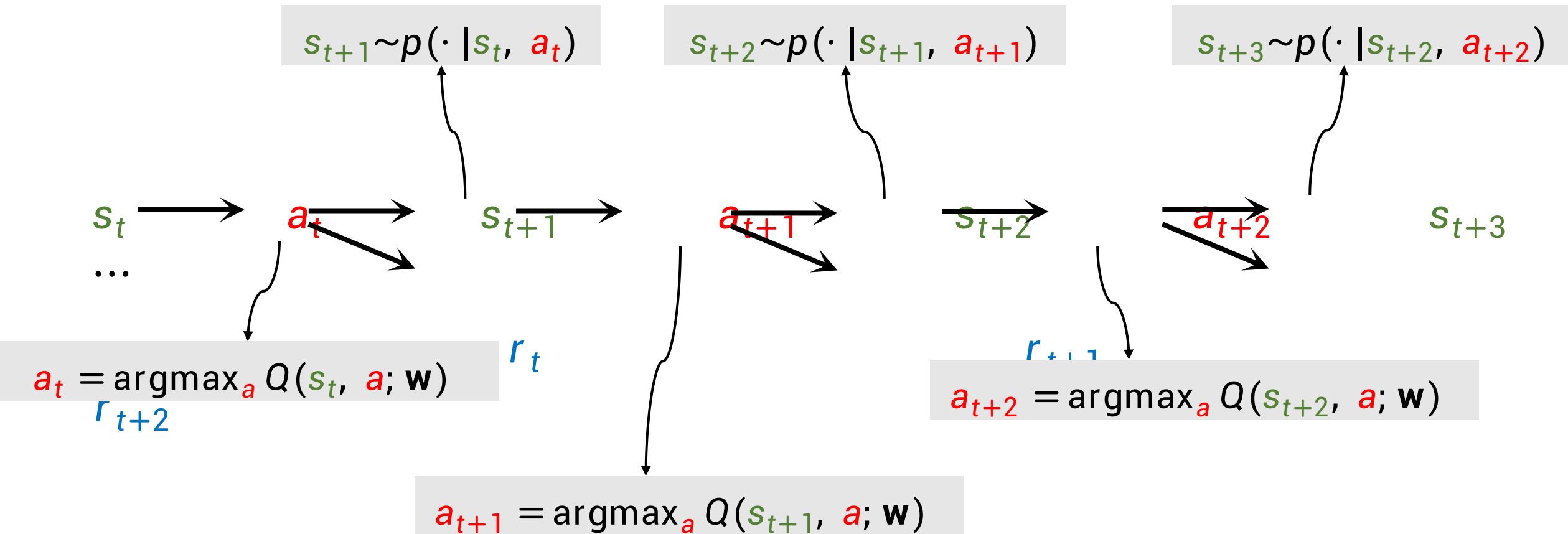
Apply DQN to Play Game



Apply DQN to Play Game



Apply DQN to Play Game



Temporal Difference (TD) Learning

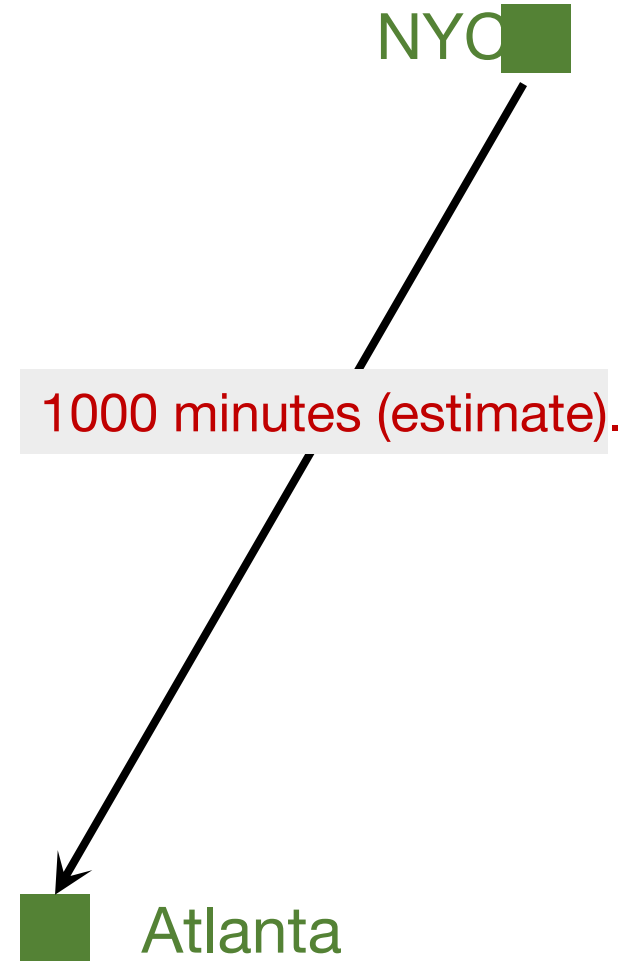
Reference

1. Sutton and others: [A convergent \$O\(n\)\$ algorithm for off-policy temporal-difference learning with linear function approximation](#). In *NIPS*, 2008.
2. Sutton and others: [Fast gradient-descent methods for temporal-difference learning with linear function approximation](#). In *ICML*, 2009.

Example

- I want to drive from NYC to Atlanta.
- Model $Q(\mathbf{w})$ estimates the time cost, e.g., 1000 minutes.

Question: How do I update the model?

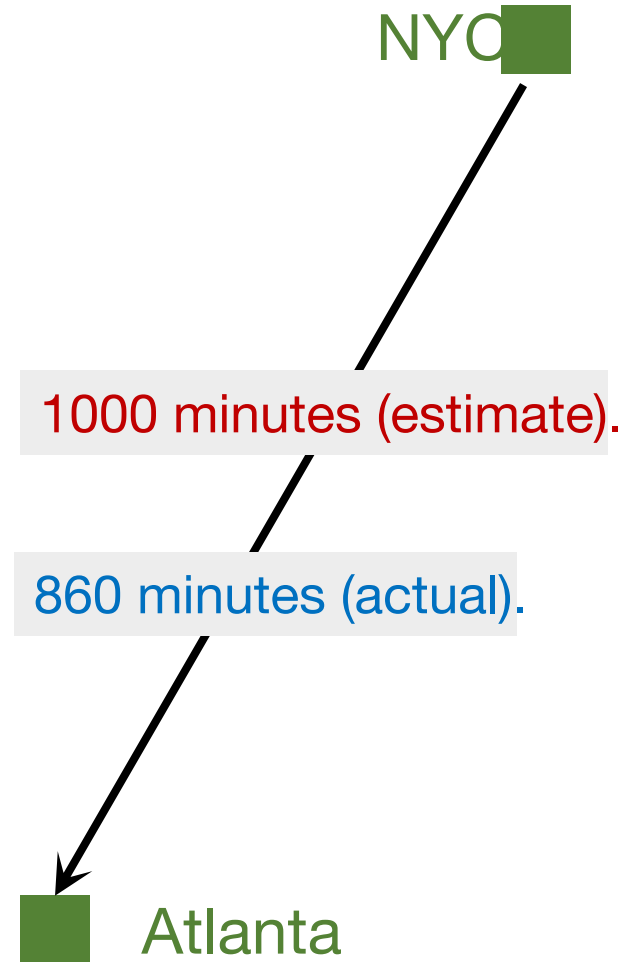


Example

- I want to drive from NYC to Atlanta.
- Model $Q(\mathbf{w})$ estimates the time cost, e.g., 1000 minutes.

Question: How do I update the model?

- Make a prediction: $q = Q(\mathbf{w})$, e.g., $q = 1000$.
- Finish the trip and get the target y , e.g., $y = 860$.

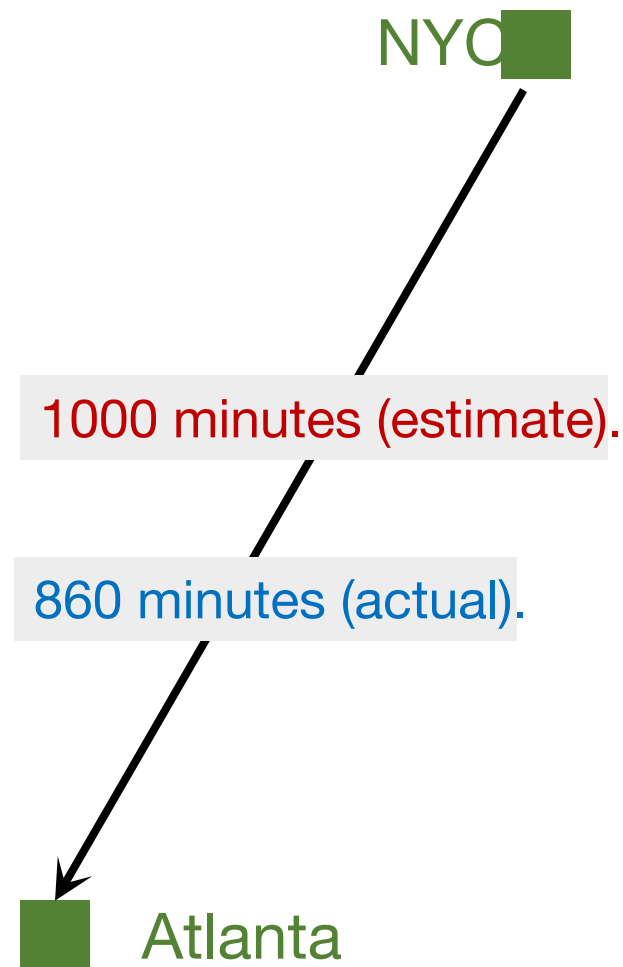


Example

- I want to drive from NYC to Atlanta.
- Model $Q(\mathbf{w})$ estimates the time cost, e.g., 1000 minutes.

Question: How do I update the model?

- Make a prediction: $q = Q(\mathbf{w})$, e.g., $q = 1000$.
- Finish the trip and get the target y , e.g., $y = 860$.
- Loss: $L = \frac{1}{2}(q - y)^2$.
- Gradient: $\frac{\partial L}{\partial \mathbf{w}} = \frac{\partial q}{\partial \mathbf{w}} \cdot \frac{\partial L}{\partial q} = (q - y) \cdot \frac{\partial Q(\mathbf{w})}{\partial \mathbf{w}}$.
- Gradient descent: $\mathbf{w}_{t+1} = \mathbf{w}_t - a \cdot \frac{\partial L}{\partial \mathbf{w}} \Big|_{\mathbf{w}=\mathbf{w}_t}$.

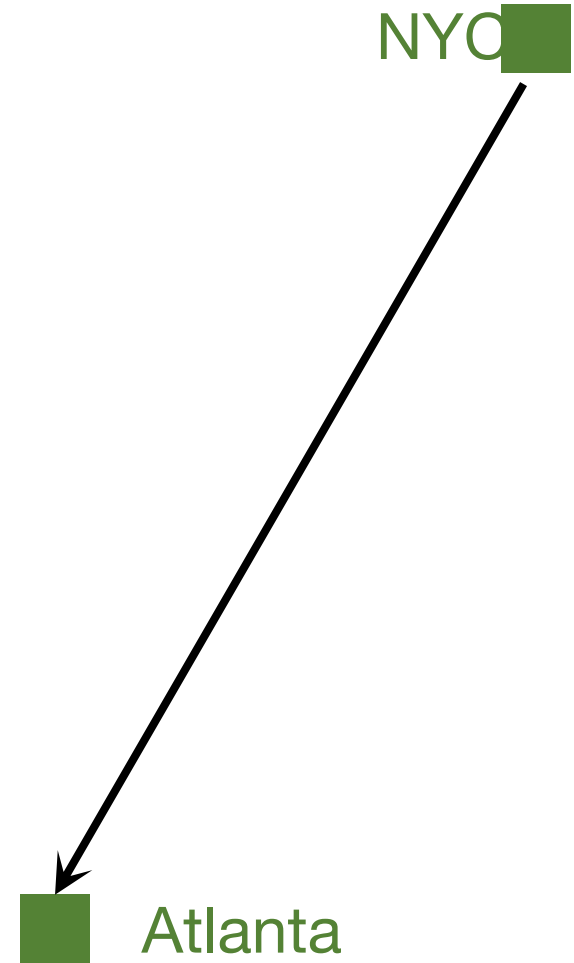


Example

- I want to drive from NYC to Atlanta.
- Model $Q(\mathbf{w})$ estimates the time cost, e.g., 1000 minutes

Question: How do I update the model?

- Can I update the model **before finishing the trip?**

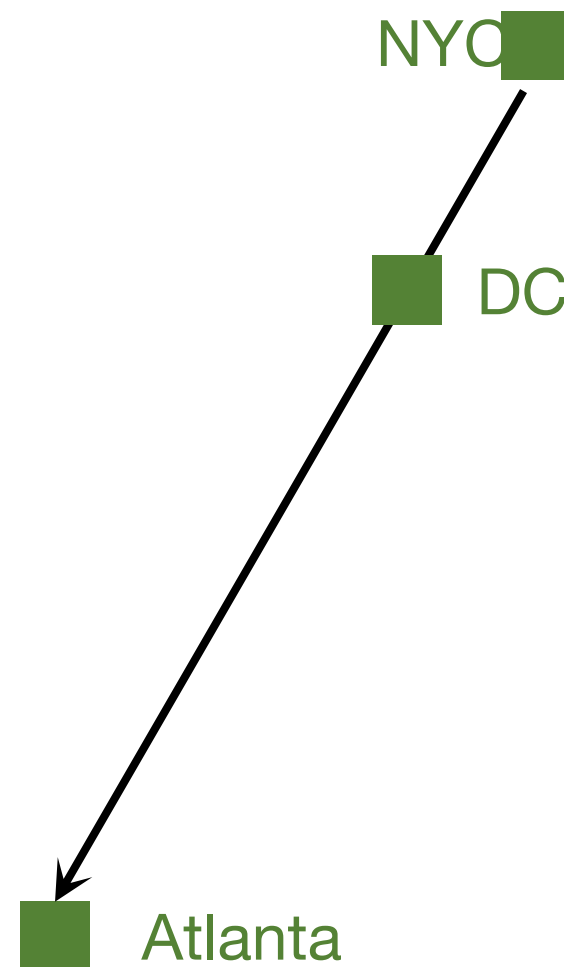


Example

- I want to drive from NYC to Atlanta (via DC).
- Model $Q(\mathbf{w})$ estimates the time cost, e.g., 1000 minutes.

Question: How do I update the model?

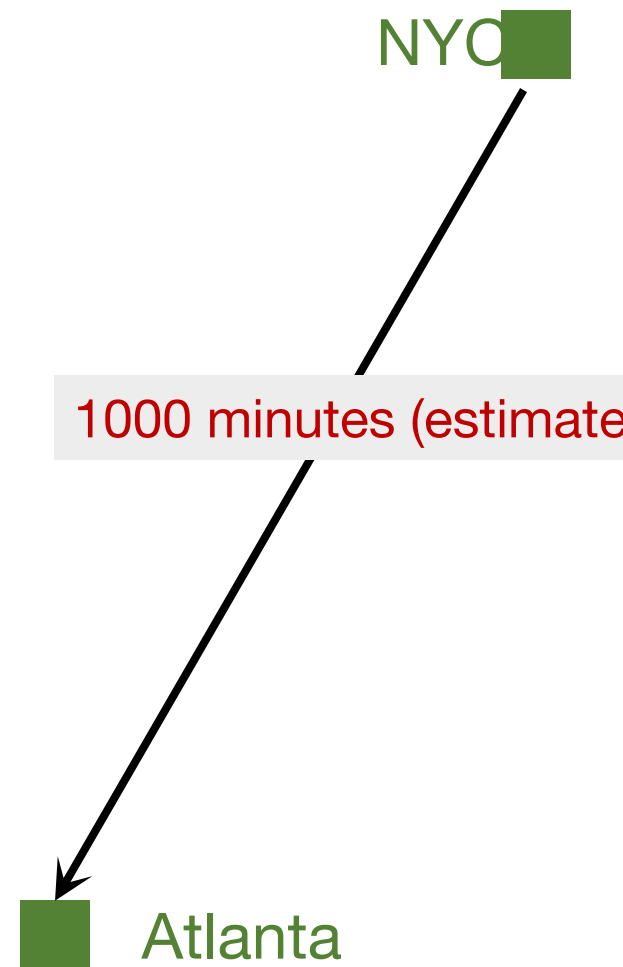
- Can I update the model before finishing the trip?
- Can I get a better $\mathbf{w}$ as soon as I arrived at DC?



Temporal Difference (TD) Learning

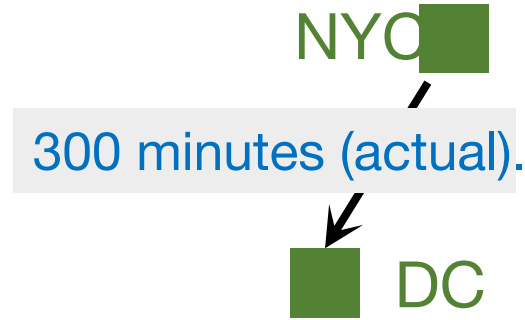
- Model's estimate:

NYC to Atlanta: 1000 minutes (estimate).



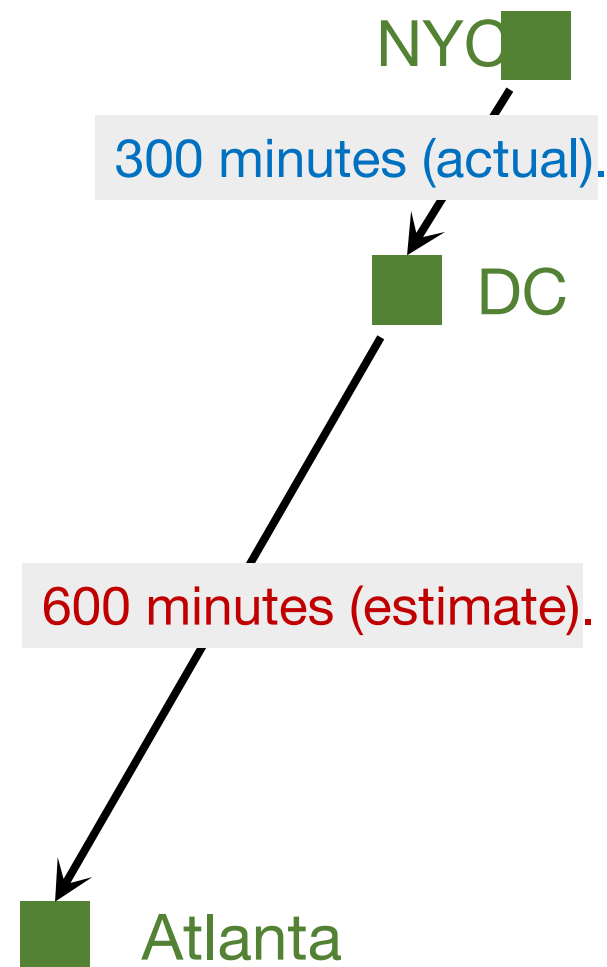
Temporal Difference (TD) Learning

- Model's estimate:
 NYC to Atlanta: 1000 minutes (estimate).
- I arrived at DC; actual time cost:
 NYC to DC: 300 minutes (actual).



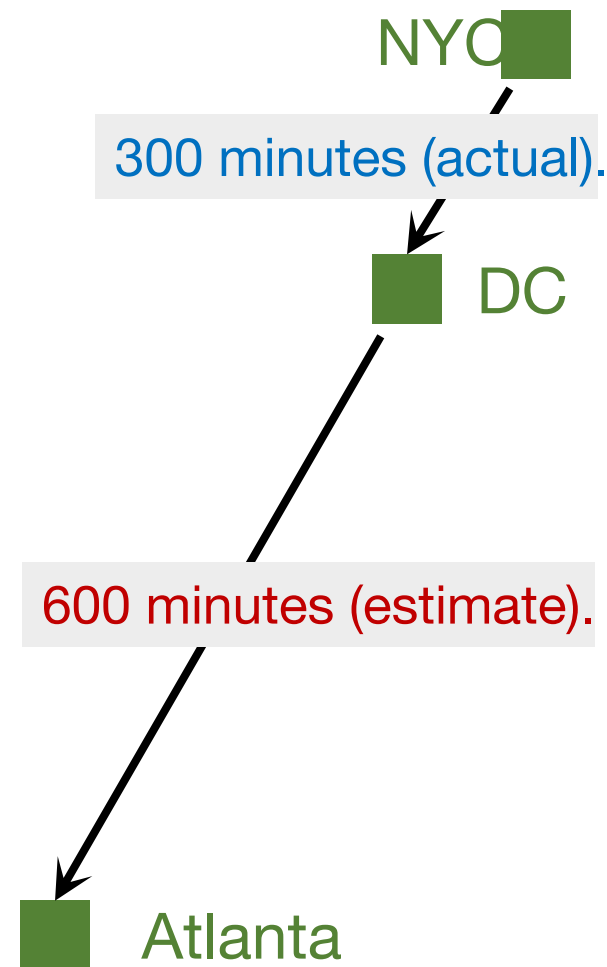
Temporal Difference (TD) Learning

- Model's estimate:
 NYC to Atlanta: 1000 minutes (estimate).
- I arrived at DC; actual time cost:
 NYC to DC: 300 minutes (actual).
- Model now updates its estimate:
 DC to Atlanta: 600 minutes (estimate).



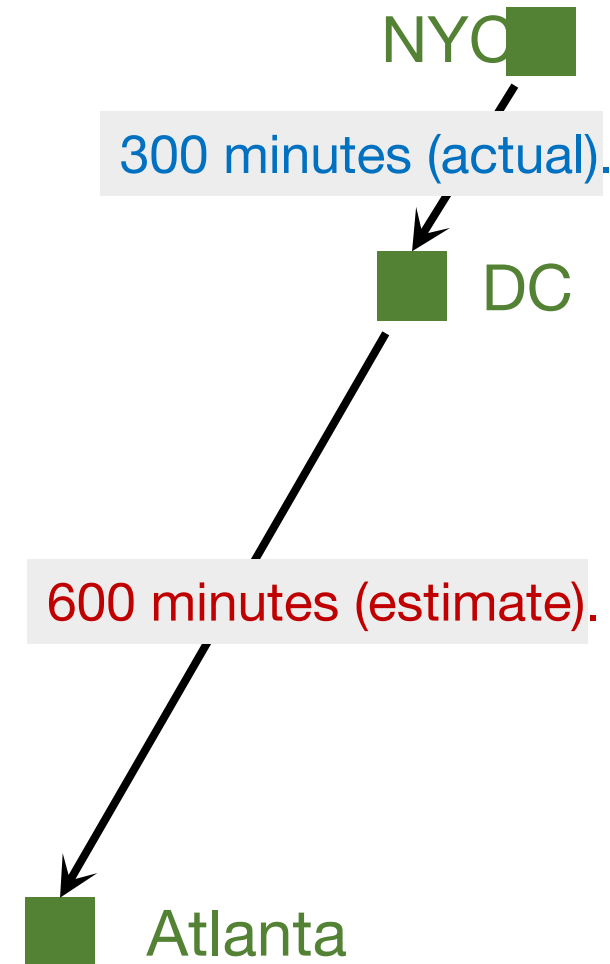
Temporal Difference (TD) Learning

- Model's estimate: $Q(\mathbf{w}) = 1000$ minutes.
- Updated estimate: $300 + 600 = 900$ minutes.
TD target.



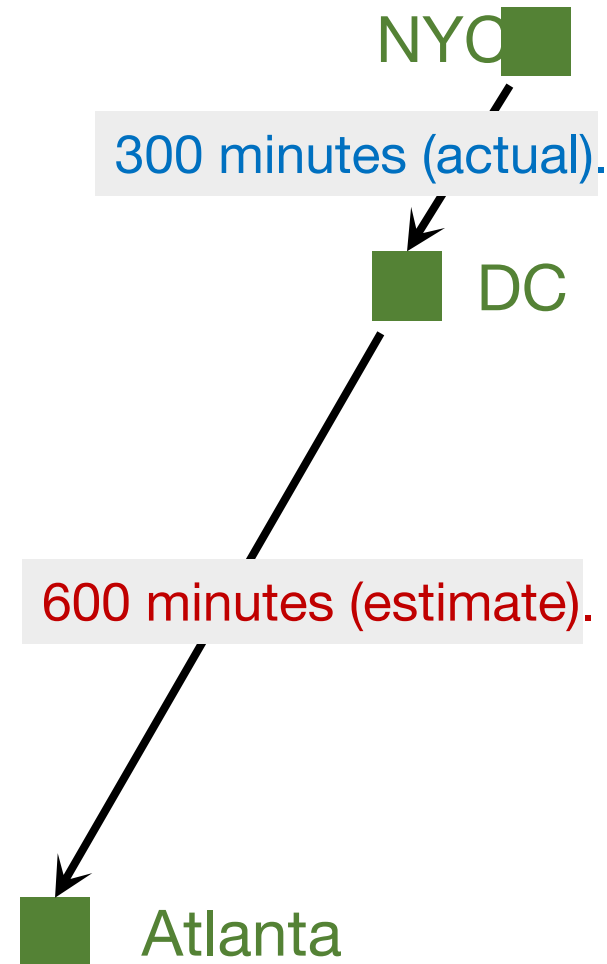
Temporal Difference (TD) Learning

- Model's estimate: $Q(\mathbf{w}) = 1000$ minutes.
- Updated estimate: $300 + 600 = 900$ minutes.
TD target.
- TD target $y = 900$ is a more reliable estimate than 1000.



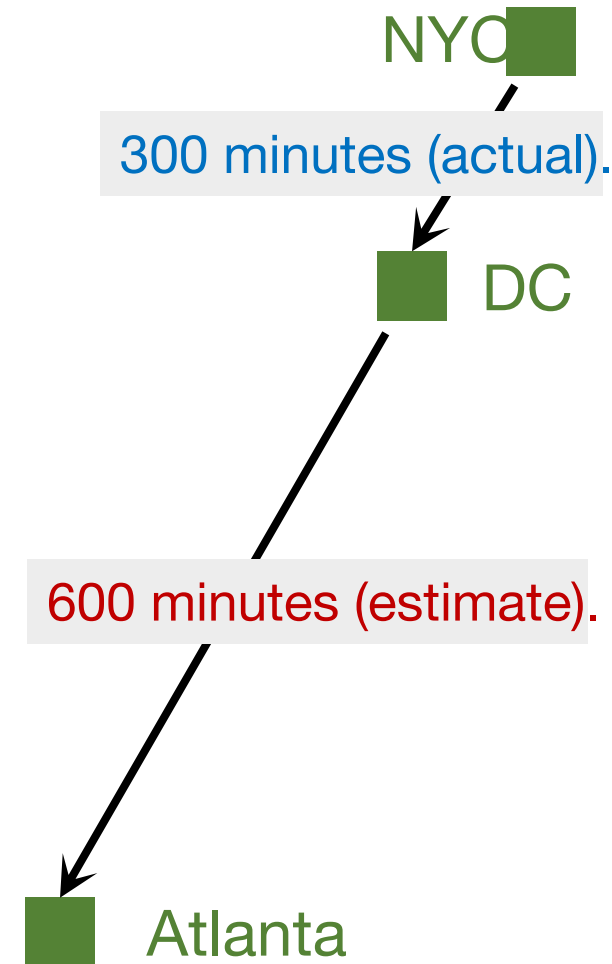
Temporal Difference (TD) Learning

- Model's estimate: $Q(\mathbf{w}) = 1000$ minutes.
- Updated estimate: $300 + 600 = 900$ minutes.
TD target.
- TD target $y = 900$ is a more reliable estimate than 1000.
- Loss: $L = \frac{1}{2} (\underbrace{Q(\mathbf{w}) - y}_{\text{TD error}})^2$.



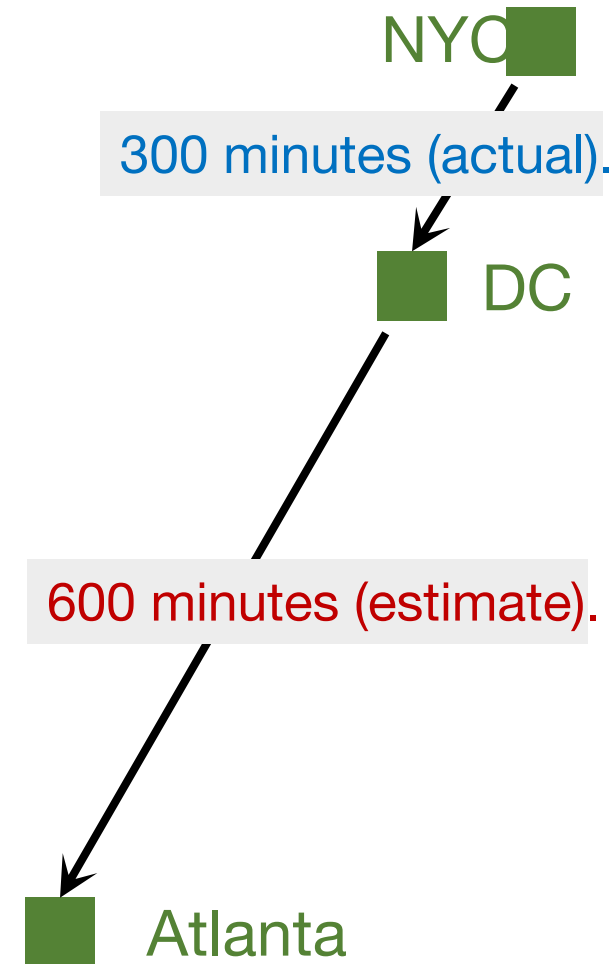
Temporal Difference (TD) Learning

- Model's estimate: $Q(\mathbf{w}) = 1000$ minutes.
- Updated estimate: $300 + 600 = 900$ minutes.
TD target.
- TD target $y = 900$ is a more reliable estimate than 1000 .
- Loss: $L = \frac{1}{2} (Q(\mathbf{w}) - y)^2$.
- Gradient: $\frac{\partial L}{\partial \mathbf{w}} = \underbrace{(1000 - 900)}_{\text{TD error}} \cdot \frac{\partial Q(\mathbf{w})}{\partial \mathbf{w}}$.



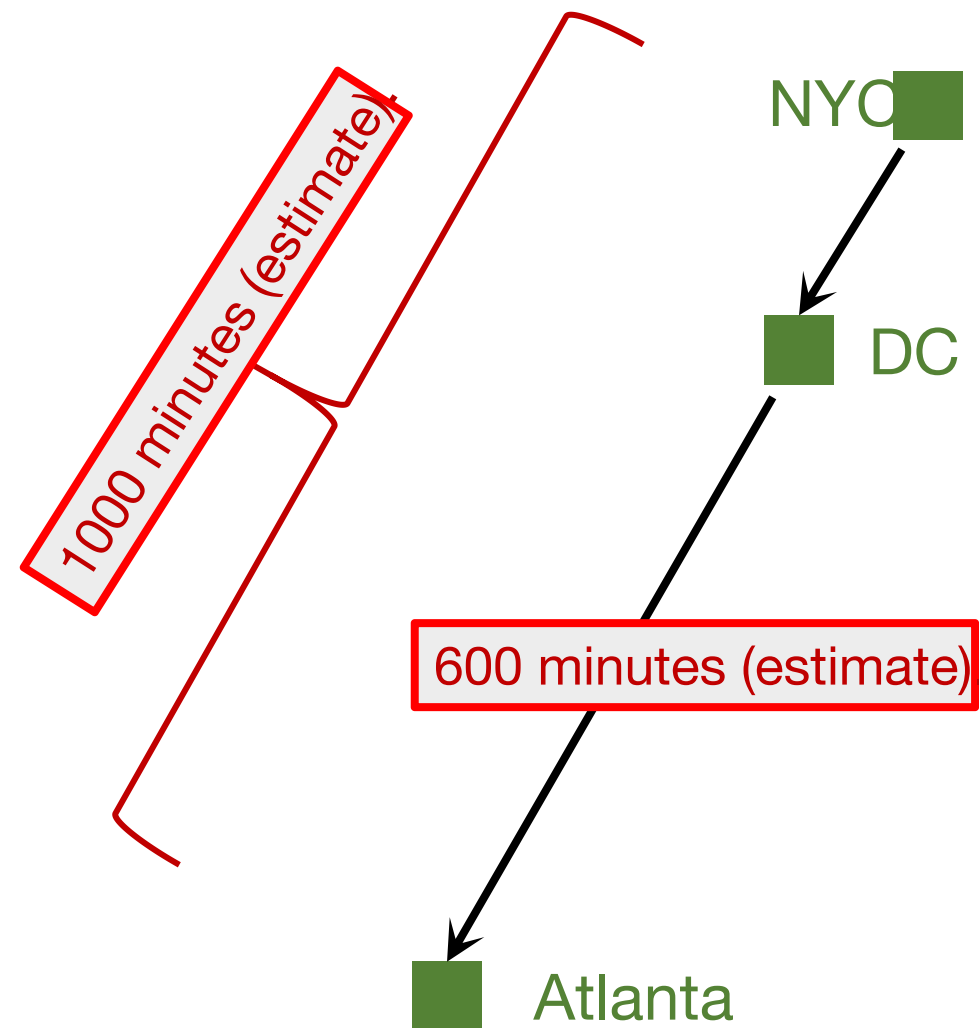
Temporal Difference (TD) Learning

- Model's estimate: $Q(\mathbf{w}) = 1000$ minutes.
- Updated estimate: $300 + 600 = 900$ minutes.
TD target.
- TD target $y = 900$ is a more reliable estimate than 1000 .
- Loss: $L = \frac{1}{2} (Q(\mathbf{w}) - y)^2$.
- Gradient: $\frac{\partial L}{\partial \mathbf{w}} = (1000 - 900) \cdot \frac{\partial Q(\mathbf{w})}{\partial \mathbf{w}}$.
- Gradient descent: $\mathbf{w}_{t+1} = \mathbf{w}_t - a \cdot \frac{\partial L}{\partial \mathbf{w}} \Big|_{\mathbf{w}=\mathbf{w}_t}$.



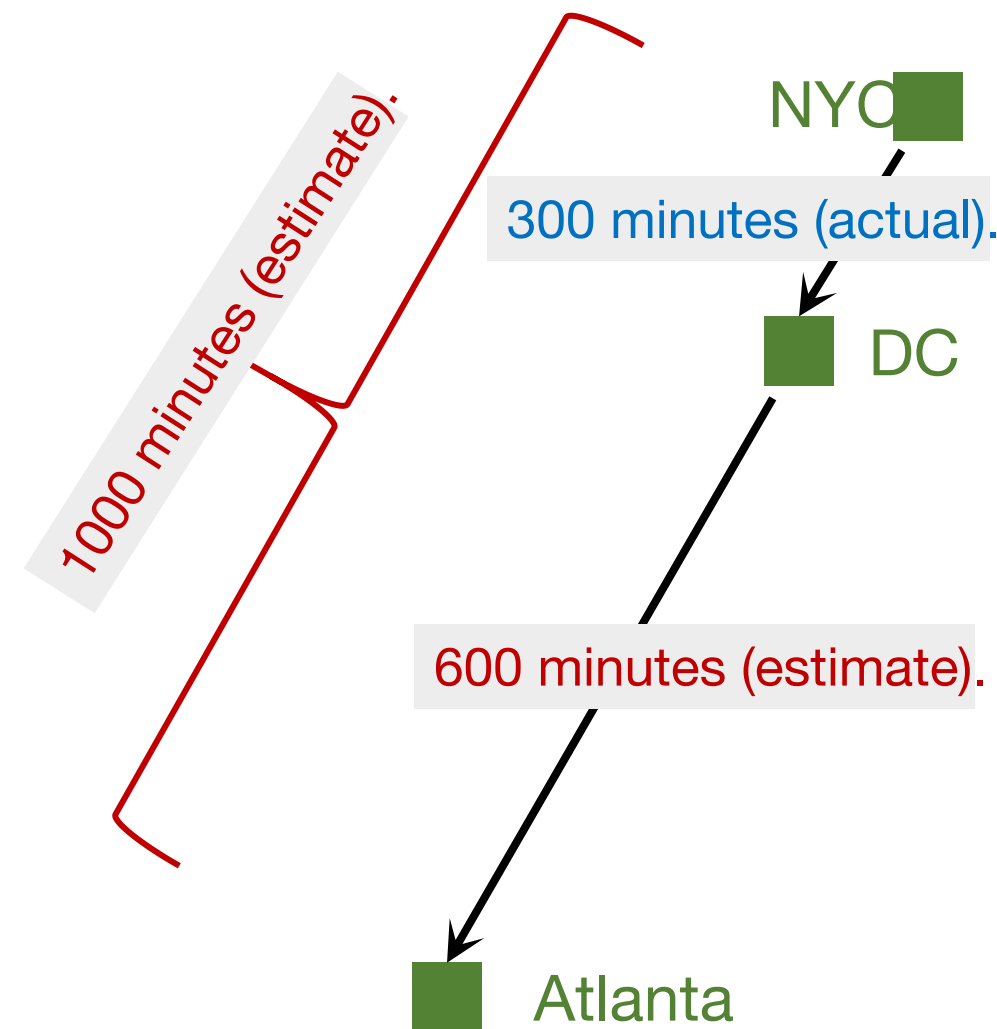
Why does TD learning work?

- Model's estimates:
 - NYC to Atlanta: 1000 minutes.
 - DC to Atlanta: 600 minutes.
 - → NYC to DC: 400 minutes.



Why does TD learning work?

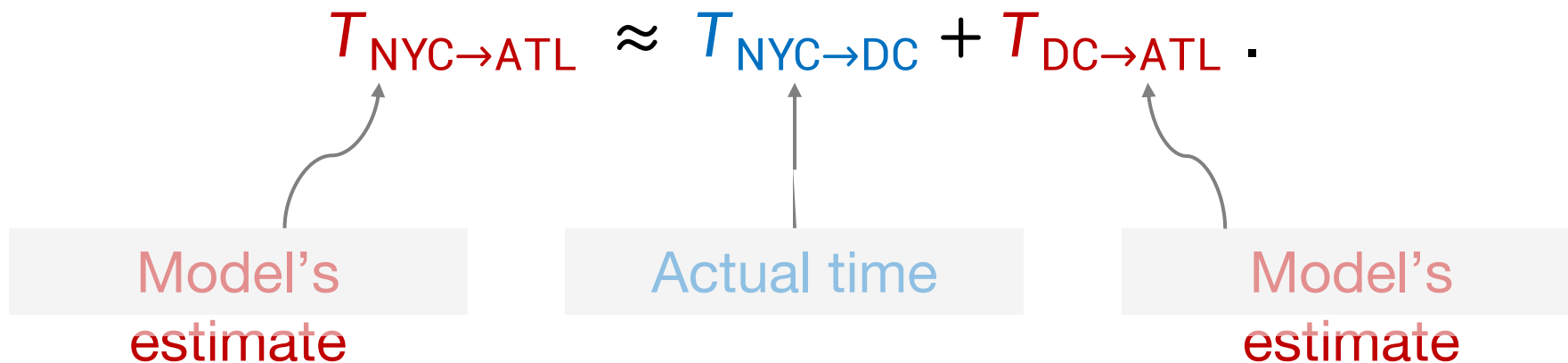
- Model's estimates:
 - NYC to Atlanta: 1000 minutes.
 - DC to Atlanta: 600 minutes.
 - → NYC to DC: 400 minutes.
- Ground truth:
 - NYC to DC: 300 minutes.
- TD error: $\delta = 400 - 300 = 100$



TD Learning for DQN

How to apply TD learning to DQN?

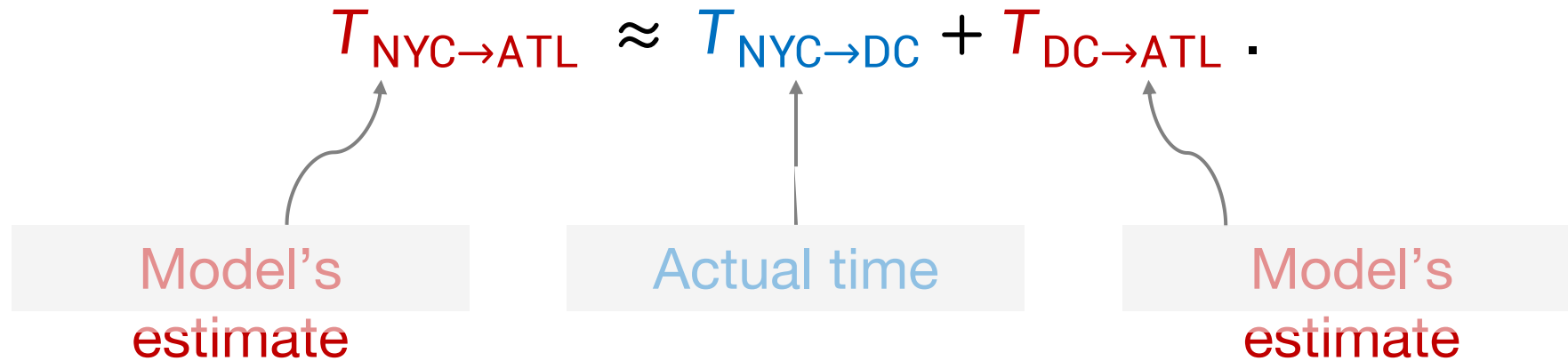
- In the “driving time” example, we have the equation:

$$T_{\text{NYC} \rightarrow \text{ATL}} \approx T_{\text{NYC} \rightarrow \text{DC}} + T_{\text{DC} \rightarrow \text{ATL}} .$$


The diagram illustrates the relationship between model estimates and actual time in a driving time example. It features three light gray rectangular boxes arranged horizontally. The leftmost box contains the text "Model's estimate" in red. The middle box contains the text "Actual time" in blue. The rightmost box contains the text "Model's estimate" in red. Above these boxes is the equation $T_{\text{NYC} \rightarrow \text{ATL}} \approx T_{\text{NYC} \rightarrow \text{DC}} + T_{\text{DC} \rightarrow \text{ATL}} .$. A curved gray arrow points from the left box to the red term $T_{\text{NYC} \rightarrow \text{ATL}}$ in the equation. A straight gray arrow points from the middle box to the blue term $T_{\text{NYC} \rightarrow \text{DC}}$. A curved gray arrow points from the right box to the red term $T_{\text{DC} \rightarrow \text{ATL}}$.

How to apply TD learning to DQN?

- In the “driving time” example, we have the equation:

$$T_{\text{NYC} \rightarrow \text{ATL}} \approx T_{\text{NYC} \rightarrow \text{DC}} + T_{\text{DC} \rightarrow \text{ATL}} .$$


The diagram illustrates the relationship between model estimates and actual time in the driving time example. It features three boxes at the bottom: "Model's estimate" (red text), "Actual time" (blue text), and "Model's estimate" (red text). Arrows point from these boxes to the corresponding terms in the equation above: the first "Model's estimate" points to $T_{\text{NYC} \rightarrow \text{ATL}}$, "Actual time" points to $T_{\text{NYC} \rightarrow \text{DC}}$, and the second "Model's estimate" points to $T_{\text{DC} \rightarrow \text{ATL}}$.

- In deep reinforcement learning:

$$Q(s_t, a_t; \mathbf{w}) \approx r_t + \gamma \cdot Q(s_{t+1}, a_{t+1}; \mathbf{w}) .$$

How to apply TD learning to DQN?

Definition of discounted return:

$$\begin{aligned} \bullet U_t &= R_t + \gamma \cdot R_{t+1} + \gamma^2 \cdot R_{t+2} + \gamma^3 \cdot R_{t+3} + \gamma^4 \cdot R_{t+4} + \dots \\ &\quad \underbrace{\hspace{15em}} \\ &= \gamma \cdot (R_{t+1} + \gamma \cdot R_{t+2} + \gamma^2 \cdot R_{t+3} + \gamma^3 \cdot R_{t+4} + \dots) \end{aligned}$$

How to apply TD learning to DQN?

- $$\begin{aligned} U_t &= R_t + \gamma \cdot R_{t+1} + \gamma^2 \cdot R_{t+2} + \gamma^3 \cdot R_{t+3} + \gamma^4 \cdot R_{t+4} + \dots \\ &= R_t + \gamma \cdot \left(R_{t+1} + \gamma \cdot R_{t+2} + \gamma^2 \cdot R_{t+3} + \gamma^3 \cdot R_{t+4} + \dots \right) \end{aligned}$$

$= U_{t+1}$

How to apply TD learning to DQN?

Identity: $U_t = R_t + \gamma \cdot U_{t+1}$.

$$\begin{aligned} \bullet U_t &= R_t + \gamma \cdot R_{t+1} + \gamma^2 \cdot R_{t+2} + \gamma^3 \cdot R_{t+3} + \gamma^4 \cdot R_{t+4} + \dots \\ &= R_t + \gamma \cdot \left(R_{t+1} + \gamma \cdot R_{t+2} + \gamma^2 \cdot R_{t+3} + \gamma^3 \cdot R_{t+4} + \dots \right) \\ &\quad \underbrace{\hspace{15em}}_{= U_{t+1}} \end{aligned}$$

How to apply TD learning to DQN?

Identity: $U_t = R_t + \gamma \cdot U_{t+1}$.

TD learning for DQN:

- DQN's output, $Q(s_t, a_t; \mathbf{w})$, is an estimate of U_t .
- DQN's output, $Q(s_{t+1}, a_{t+1}; \mathbf{w})$, is an estimate of U_{t+1} .

How to apply TD learning to DQN?

Identity: $U_t = R_t + \gamma \cdot U_{t+1}$.

TD learning for DQN:

- DQN's output, $Q(s_t, a_t; \mathbf{w})$, is an estimate of U_t .
- DQN's output, $Q(s_{t+1}, a_{t+1}; \mathbf{w})$, is an estimate of U_{t+1} .

• Thus,
$$\underbrace{Q(s_t, a_t; \mathbf{w})}_{\text{estimate of } U_t} \approx \mathbb{E}[\underbrace{R_t + \gamma \cdot Q(s_{t+1}, a_{t+1}; \mathbf{w})}_{\text{estimate of } U_{t+1}}].$$

How to apply TD learning to DQN?

Identity: $U_t = R_t + \gamma \cdot U_{t+1}$.

TD learning for DQN:

- DQN's output, $Q(s_t, a_t; \mathbf{w})$, is an estimate of U_t .
- DQN's output, $Q(s_{t+1}, a_{t+1}; \mathbf{w})$, is an estimate of U_{t+1} .

• Thus, $\underbrace{Q(s_t, a_t; \mathbf{w})}_{\text{Prediction}} \approx \underbrace{r_t + \gamma \cdot Q(s_{t+1}, a_{t+1}; \mathbf{w})}_{\text{TD target}}.$

Train DQN using TD learning

- Prediction: $Q(s_t, a_t; \mathbf{w}_t)$.
- TD target:

$$y_t = r_t + \gamma \cdot Q(s_{t+1}, a_{t+1}; \mathbf{w}_t)$$

Train DQN using TD learning

- Prediction: $Q(s_t, a_t; \mathbf{w}_t)$.
- TD target:

$$\begin{aligned} y_t &= r_t + \gamma \cdot Q(s_{t+1}, a_{t+1}; \mathbf{w}_t) \\ &= r_t + \gamma \cdot \max_a Q(s_{t+1}, a; \mathbf{w}_t). \end{aligned}$$

Train DQN using TD learning

- Prediction: $Q(s_t, a_t; \mathbf{w}_t)$.

- TD target:

$$\begin{aligned} y_t &= r_t + \gamma \cdot Q(s_{t+1}, a_{t+1}; \mathbf{w}_t) \\ &= r_t + \gamma \cdot \max_a Q(s_{t+1}, a; \mathbf{w}_t). \end{aligned}$$

- Loss: $L_t = \frac{1}{2} [Q(s_t, a_t; \mathbf{w}) - y_t]^2$.

- Gradient descent: $\mathbf{w}_{t+1} = \mathbf{w}_t - a \cdot \frac{\partial L_t}{\partial \mathbf{w}} \Big|_{\mathbf{w}=\mathbf{w}_t}$.

Summary

Value-Based Reinforcement Learning

Definition: Optimal action-value function.

- $Q^*(s_t, a_t) = \max_{\pi} \mathbb{E} [U_t | S_t = s_t, A_t = a_t].$

Value-Based Reinforcement Learning

Definition: Optimal action-value function.

- $Q^*(s_t, a_t) = \max_{\pi} \mathbb{E} [U_t | S_t = s_t, A_t = a_t].$

DQN: Approximate $Q^*(s, a)$ using a neural network (DQN).

- $Q(s, a; \mathbf{w})$ is a neural network parameterized by $\mathbf{w}$.
- Input: observed state s .
- Output: scores for all the action $a \in \mathcal{A}$.

Temporal Difference (TD) Learning

Algorithm: One iteration of TD learning.

1. Observe state $S_t = s_t$ and perform action $A_t = a_t$.
2. Predict the value: $q_t = Q(s_t, a_t; \mathbf{w}_t)$.
3. Differentiate the value network: $\mathbf{d}_t = \left. \frac{\partial Q(s_t, a_t; \mathbf{w})}{\partial \mathbf{w}} \right|_{\mathbf{w}=\mathbf{w}_t}$.

Temporal Difference (TD) Learning

Algorithm: One iteration of TD learning.

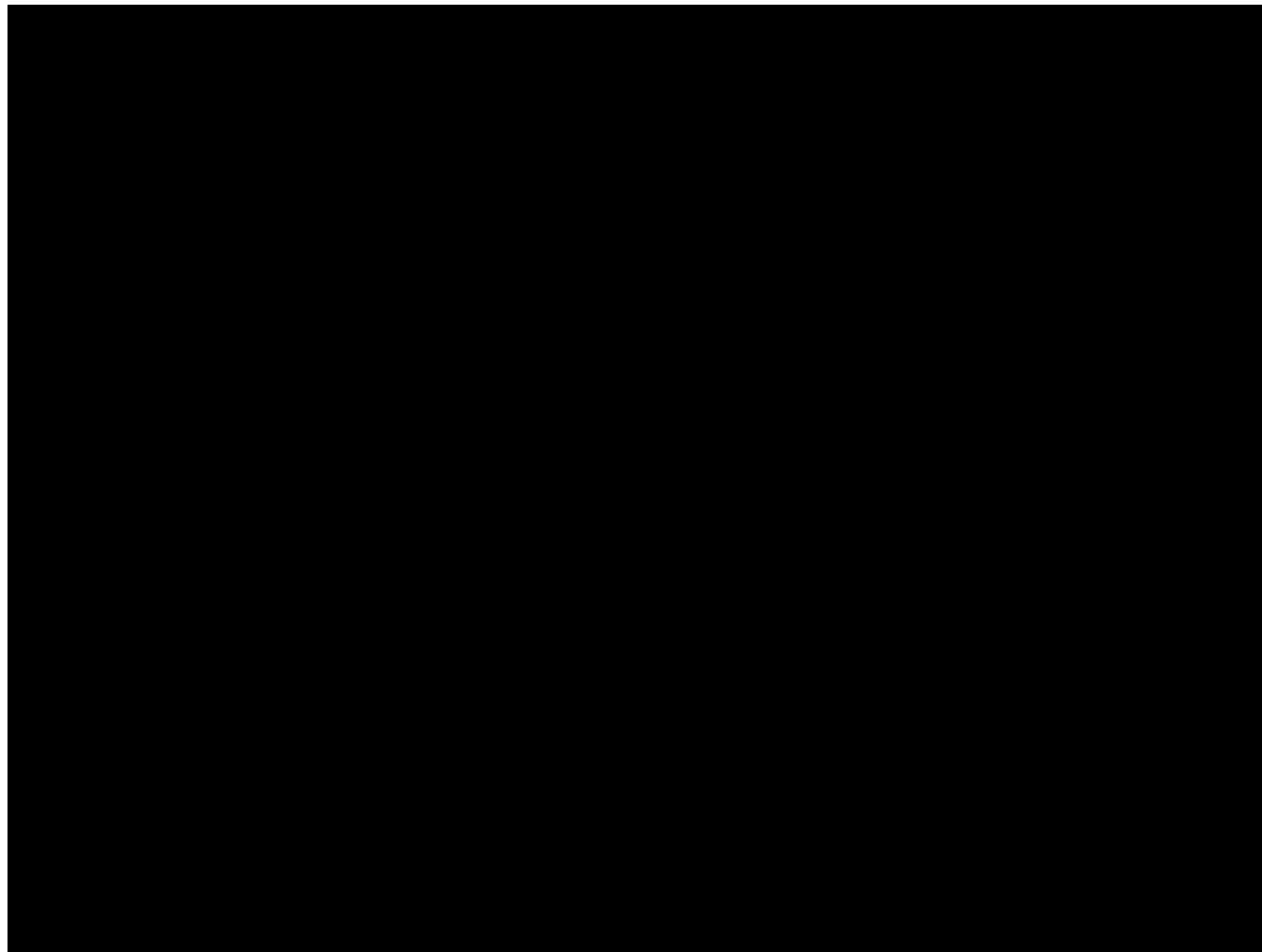
1. Observe state $S_t = s_t$ and perform action $A_t = a_t$.
2. Predict the value: $q_t = Q(s_t, a_t; \mathbf{w}_t)$.
3. Differentiate the value network: $\mathbf{d}_t = \left. \frac{\partial Q(s_t, a_t; \mathbf{w})}{\partial \mathbf{w}} \right|_{\mathbf{w}=\mathbf{w}_t}$.
4. Environment provides new state s_{t+1} and reward r_t .
5. Compute TD target: $y_t = r_t + \gamma \cdot \max_a Q(s_{t+1}, a; \mathbf{w}_t)$.

Temporal Difference (TD) Learning

Algorithm: One iteration of TD learning.

1. Observe state $S_t = s_t$ and perform action $A_t = a_t$.
2. Predict the value: $q_t = Q(s_t, a_t; \mathbf{w}_t)$.
3. Differentiate the value network: $\mathbf{d}_t = \left. \frac{\partial Q(s_t, a_t; \mathbf{w})}{\partial \mathbf{w}} \right|_{\mathbf{w}=\mathbf{w}_t}$.
4. Environment provides new state s_{t+1} and reward r_t .
5. Compute TD target: $y_t = r_t + \gamma \cdot \max_a Q(s_{t+1}, a; \mathbf{w}_t)$.
6. Gradient descent: $\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \cdot (q_t - y_t) \cdot \mathbf{d}_t$.

Play Breakout using DQN



(The video was posted on YouTube by DeepMind)

Policy-Based Reinforcement Learning

Policy Function Approximation

Policy Function $\pi(a|s)$

- Policy function $\pi(a|s)$ is a probability density function (PDF).
- It takes state s as input.
- It output the probabilities for all the actions, e.g.,

$$\pi(\text{left}|s) = 0.2,$$

$$\pi(\text{right}|s) = 0.1,$$

$$\pi(\text{up}|s) = 0.7.$$

- Randomly sample action a random drawn from the distribution.

Can we directly learn a policy function $\pi(a|s)$?

- If there are only a few states and actions, then yes, we can.
- Draw a table (matrix) and learn the entries.

	Action a_1	Action a_2	Action a_3	Action a_4	...
State s_1					
State s_2					
State s_3					
⋮					

Can we directly learn a policy function $\pi(a|s)$?

- If there are only a few states and actions, then yes, we can.
- Draw a table (matrix) and learn the entries.
- What if there are too many (or infinite) states or actions?

	Action a_1	Action a_2	Action a_3	Action a_4	...
State s_1					
State s_2					
State s_3					
⋮					

Policy Network $\pi(a|s; \theta)$

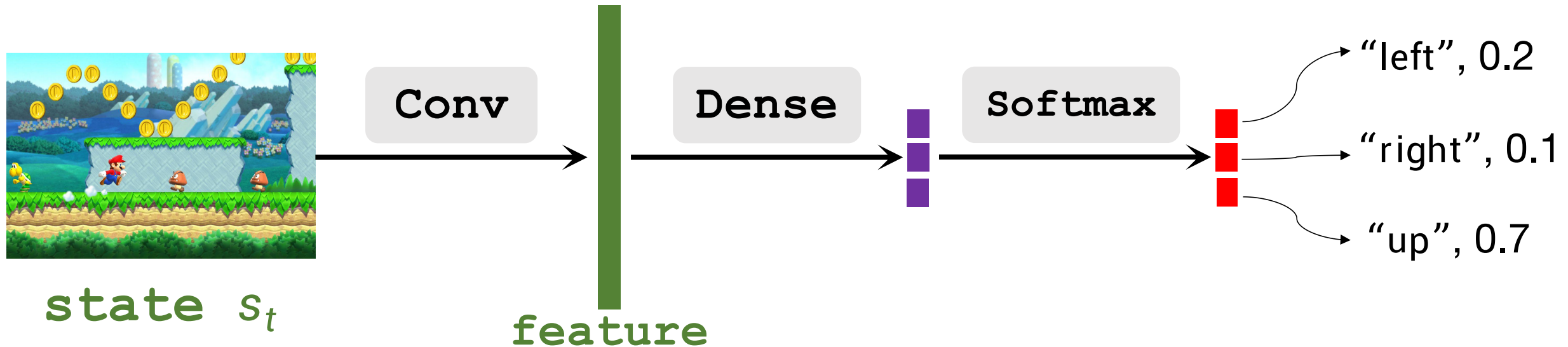
Policy network: Use a neural net to approximate $\pi(a|s)$.

- Use policy network $\pi(a|s; \theta)$ to approximate $\pi(a|s)$.
- θ : trainable parameters of the neural net.

Policy Network $\pi(a|s; \theta)$

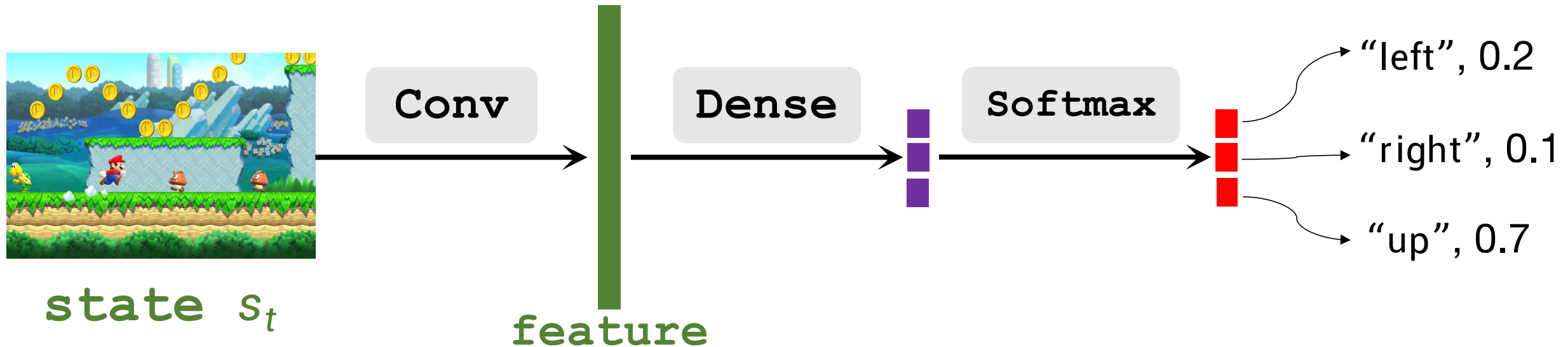
Policy network: Use a neural net to approximate $\pi(a|s)$.

- Use policy network $\pi(a|s; \theta)$ to approximate $\pi(a|s)$.
- θ : trainable parameters of the neural net.



Policy Network $\pi(a|s; \theta)$

- $\sum_{a \in \mathcal{A}} \pi(a|s; \theta) = 1.$
- Here, $\mathcal{A} = \{\text{“left”}, \text{“right”}, \text{“up”}\}$ is the set all actions.
- That is why we use softmax activation.



State-Value Function Approximation

Action-Value Function

Definition: Discounted return.

- $U_t = R_t + \gamma \cdot R_{t+1} + \gamma^2 \cdot R_{t+2} + \gamma^3 \cdot R_{t+3} + \dots$



- The return depends on actions $A_t, A_{t+1}, A_{t+2}, \dots$ and states $S_t, S_{t+1}, S_{t+2}, \dots$

- Actions are random: $\mathbb{P}[A = a \mid S = s] = \pi(a|s)$. (Policy function.)

- States are random: $\mathbb{P}[S' = s' \mid S = s, A = a] = p(s'|s, a)$. (State transition.)

Action-Value Function

Definition: Discounted return.

- $U_t = R_t + \gamma \cdot R_{t+1} + \gamma^2 \cdot R_{t+2} + \gamma^3 \cdot R_{t+3} + \dots$

Definition: Action-value function.

- $Q_\pi(s_t, a_t) = \mathbb{E}[U_t | S_t = s_t, A_t = a_t].$

The expectation is taken
w.r.t. actions

$A_{t+1}, A_{t+2}, A_{t+3}, \dots$ and

states $S_{t+1}, S_{t+2}, S_{t+3}, \dots$

State-Value Function

Definition: Discounted return.

- $U_t = R_t + \gamma \cdot R_{t+1} + \gamma^2 \cdot R_{t+2} + \gamma^3 \cdot R_{t+3} + \dots$

Definition: Action-value function.

- $Q_\pi(s_t, a_t) = \mathbb{E}[U_t | S_t = s_t, A_t = a_t].$

Definition: State-value function.

- $V_\pi(s_t) = \mathbb{E}_A [Q_\pi(s_t, A)]$



Integrate out action $A \sim \pi(\cdot | s_t).$

State-Value Function

Definition: Discounted return.

- $U_t = R_t + \gamma \cdot R_{t+1} + \gamma^2 \cdot R_{t+2} + \gamma^3 \cdot R_{t+3} + \dots$

Definition: Action-value function.

- $Q_\pi(s_t, a_t) = \mathbb{E}[U_t | S_t = s_t, A_t = a_t].$

Definition: State-value function.

- $V_\pi(s_t) = \mathbb{E}_A [Q_\pi(s_t, A)] = \sum_a \pi(a|s_t) \cdot Q_\pi(s_t, a).$



Integrate out action $A \sim \pi(\cdot | s_t).$

Policy-Based Reinforcement Learning

Definition: State-value function.

- $V_{\pi}(s_t) = \mathbb{E}_A [Q_{\pi}(s_t, A)] = \sum_a \pi(a|s_t) \cdot Q_{\pi}(s_t, a).$

Policy-Based Reinforcement Learning

Definition: State-value function.

- $V_{\pi}(s_t) = \mathbb{E}_A [Q_{\pi}(s_t, A)] = \sum_a \pi(a|s_t) \cdot Q_{\pi}(s_t, a).$

Approximate state-value function.

- Approximate policy function $\pi(a|s_t)$ by policy network $\pi(a|s_t; \theta).$

Policy-Based Reinforcement Learning

Definition: State-value function.

- $V_{\pi}(s_t) = \mathbb{E}_A [Q_{\pi}(s_t, A)] = \sum_a \pi(a|s_t) \cdot Q_{\pi}(s_t, a).$

Approximate state-value function.

- Approximate policy function $\pi(a|s_t)$ by policy network $\pi(a|s_t; \theta).$
- Approximate value function $V_{\pi}(s_t)$ by:

$$V(s_t; \theta) = \sum_a \pi(a|s_t; \theta) \cdot Q_{\pi}(s_t, a).$$

Policy-Based Reinforcement Learning

Definition: Approximate state-value function.

- $V(\mathbf{s}; \boldsymbol{\theta}) = \sum_a \pi(\mathbf{a}|\mathbf{s}; \boldsymbol{\theta}) \cdot Q_{\pi}(\mathbf{s}, \mathbf{a}).$

Policy-based learning: Learn $\boldsymbol{\theta}$ that maximizes $J(\boldsymbol{\theta}) = \mathbb{E}_{\mathbf{S}}[V(\mathbf{S}; \boldsymbol{\theta})].$

Policy-Based Reinforcement Learning

Definition: Approximate state-value function.

$$\bullet V(\mathbf{s}; \boldsymbol{\theta}) = \sum_a \pi(\mathbf{a}|\mathbf{s}; \boldsymbol{\theta}) \cdot Q_{\pi}(\mathbf{s}, \mathbf{a}).$$

Policy-based learning: Learn $\boldsymbol{\theta}$ that maximizes $J(\boldsymbol{\theta}) = \mathbb{E}_{\mathbf{s}}[V(\mathbf{s}; \boldsymbol{\theta})]$.

How to improve $\boldsymbol{\theta}$? Policy gradient ascent!

- Observe state $\mathbf{s}$.
- Update policy by: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \beta \cdot \frac{\partial V(\mathbf{s}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}}$.

Policy gradient

Reference

- Sutton and others: Policy gradient methods for reinforcement learning with function approximation. In *NIPS*, 2000.

Policy Gradient

Definition: Approximate state-value function.

- $V(\mathbf{s}; \boldsymbol{\theta}) = \sum_a \pi(a|\mathbf{s}; \boldsymbol{\theta}) \cdot Q_{\pi}(\mathbf{s}, a).$

Policy gradient: Derivative of $V(\mathbf{s}; \boldsymbol{\theta})$ w.r.t. $\boldsymbol{\theta}$.

- $\frac{\partial V(\mathbf{s}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}}$

Policy Gradient

Definition: Approximate state-value function.

- $V(s; \theta) = \sum_a \pi(a|s; \theta) \cdot Q_\pi(s, a).$

Policy gradient: Derivative of $V(s; \theta)$ w.r.t. θ .

- $$\frac{\partial V(s; \theta)}{\partial \theta} = \frac{\partial \sum_a \pi(a|s; \theta) \cdot Q_\pi(s, a)}{\partial \theta}$$

Policy Gradient

Definition: Approximate state-value function.

- $V(\mathbf{s}; \boldsymbol{\theta}) = \sum_{\mathbf{a}} \pi(\mathbf{a}|\mathbf{s}; \boldsymbol{\theta}) \cdot Q_{\pi}(\mathbf{s}, \mathbf{a}).$

Policy gradient: Derivative of $V(\mathbf{s}; \boldsymbol{\theta})$ w.r.t. $\boldsymbol{\theta}$.

- $$\begin{aligned} \frac{\partial V(\mathbf{s}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} &= \frac{\partial \sum_{\mathbf{a}} \pi(\mathbf{a}|\mathbf{s}; \boldsymbol{\theta}) \cdot Q_{\pi}(\mathbf{s}, \mathbf{a})}{\partial \boldsymbol{\theta}} \\ &= \sum_{\mathbf{a}} \frac{\partial \pi(\mathbf{a}|\mathbf{s}; \boldsymbol{\theta}) \cdot Q_{\pi}(\mathbf{s}, \mathbf{a})}{\partial \boldsymbol{\theta}} \end{aligned} \quad \leftarrow \text{Push derivative inside the summation}$$

Definition: Approximate state-value function.

- $V(\mathbf{s}; \boldsymbol{\theta}) = \sum_a \pi(\mathbf{a}|\mathbf{s}; \boldsymbol{\theta}) \cdot Q_{\pi}(\mathbf{s}, \mathbf{a}).$

Policy gradient: Derivative of $V(\mathbf{s}; \boldsymbol{\theta})$ w.r.t. $\boldsymbol{\theta}$.

- $$\begin{aligned} \frac{\partial V(\mathbf{s}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} &= \frac{\partial \sum_a \pi(\mathbf{a}|\mathbf{s}; \boldsymbol{\theta}) \cdot Q_{\pi}(\mathbf{s}, \mathbf{a})}{\partial \boldsymbol{\theta}} \\ &= \sum_a \frac{\partial \pi(\mathbf{a}|\mathbf{s}; \boldsymbol{\theta}) \cdot Q_{\pi}(\mathbf{s}, \mathbf{a})}{\partial \boldsymbol{\theta}} \\ &= \sum_a \frac{\partial \pi(\mathbf{a}|\mathbf{s}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} \cdot Q_{\pi}(\mathbf{s}, \mathbf{a}) \end{aligned}$$

← Pretend Q_{π} is independent of $\boldsymbol{\theta}$.
(It may not be true.)

Policy Gradient

Definition: Approximate state-value function.

- $V(\mathbf{s}; \boldsymbol{\theta}) = \sum_a \pi(\mathbf{a}|\mathbf{s}; \boldsymbol{\theta}) \cdot Q_{\pi}(\mathbf{s}, \mathbf{a}).$

Policy gradient: Derivative of $V(\mathbf{s}; \boldsymbol{\theta})$ w.r.t. $\boldsymbol{\theta}$.

- $\frac{\partial V(\mathbf{s}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} = \sum_a \frac{\partial \pi(\mathbf{a}|\mathbf{s}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} \cdot Q_{\pi}(\mathbf{s}, \mathbf{a})$

Policy Gradient

Definition: Approximate state-value function.

- $V(\mathbf{s}; \boldsymbol{\theta}) = \sum_{\mathbf{a}} \pi(\mathbf{a}|\mathbf{s}; \boldsymbol{\theta}) \cdot Q_{\pi}(\mathbf{s}, \mathbf{a}).$

Policy gradient: Derivative of $V(\mathbf{s}; \boldsymbol{\theta})$ w.r.t. $\boldsymbol{\theta}$.

- $\frac{\partial V(\mathbf{s}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} = \sum_{\mathbf{a}} \frac{\partial \pi(\mathbf{a}|\mathbf{s}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} \cdot Q_{\pi}(\mathbf{s}, \mathbf{a})$

Policy Gradient

Note: This derivation is over-simplified and not rigorous.

Policy Gradient

Definition: Approximate state-value function.

- $V(\mathbf{s}; \boldsymbol{\theta}) = \sum_a \pi(\mathbf{a}|\mathbf{s}; \boldsymbol{\theta}) \cdot Q_\pi(\mathbf{s}, \mathbf{a}).$

Policy gradient: Derivative of $V(\mathbf{s}; \boldsymbol{\theta})$ w.r.t. $\boldsymbol{\theta}$.

- $$\begin{aligned} \frac{\partial V(\mathbf{s}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} &= \sum_a \frac{\partial \pi(\mathbf{a}|\mathbf{s}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} \cdot Q_\pi(\mathbf{s}, \mathbf{a}) \\ &= \sum_a \pi(\mathbf{a}|\mathbf{s}; \boldsymbol{\theta}) \cdot \frac{\partial \log \pi(\mathbf{a}|\mathbf{s}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} \cdot Q_\pi(\mathbf{s}, \mathbf{a}) \end{aligned}$$

Policy Gradient

Definition: Approximate state-value function.

- $V(\mathbf{s}; \boldsymbol{\theta}) = \sum_{\mathbf{a}} \pi(\mathbf{a}|\mathbf{s}; \boldsymbol{\theta}) \cdot Q_{\pi}(\mathbf{s}, \mathbf{a}).$

Policy gradient: Derivative of $V(\mathbf{s}; \boldsymbol{\theta})$ w.r.t. $\boldsymbol{\theta}$.

- $$\begin{aligned} \frac{\partial V(\mathbf{s}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} &= \sum_{\mathbf{a}} \frac{\partial \pi(\mathbf{a}|\mathbf{s}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} \cdot Q_{\pi}(\mathbf{s}, \mathbf{a}) \\ &= \sum_{\mathbf{a}} \pi(\mathbf{a}|\mathbf{s}; \boldsymbol{\theta}) \cdot \boxed{\frac{\partial \log \pi(\mathbf{a}|\mathbf{s}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}}} \cdot Q_{\pi}(\mathbf{s}, \mathbf{a}) \end{aligned}$$

- Chain rule: $\frac{\partial \log [\pi(\boldsymbol{\theta})]}{\partial \boldsymbol{\theta}} = \frac{1}{\pi(\boldsymbol{\theta})} \cdot \frac{\partial \pi(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}}.$

Policy Gradient

Definition: Approximate state-value function.

- $V(\mathbf{s}; \boldsymbol{\theta}) = \sum_{\mathbf{a}} \pi(\mathbf{a}|\mathbf{s}; \boldsymbol{\theta}) \cdot Q_{\pi}(\mathbf{s}, \mathbf{a}).$

Policy gradient: Derivative of $V(\mathbf{s}; \boldsymbol{\theta})$ w.r.t. $\boldsymbol{\theta}$.

- $$\begin{aligned} \frac{\partial V(\mathbf{s}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} &= \sum_{\mathbf{a}} \frac{\partial \pi(\mathbf{a}|\mathbf{s}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} \cdot Q_{\pi}(\mathbf{s}, \mathbf{a}) \\ &= \sum_{\mathbf{a}} \pi(\mathbf{a}|\mathbf{s}; \boldsymbol{\theta}) \cdot \frac{\partial \log \pi(\mathbf{a}|\mathbf{s}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} \cdot Q_{\pi}(\mathbf{s}, \mathbf{a}) \end{aligned}$$

- Chain rule: $\frac{\partial \log [\pi(\boldsymbol{\theta})]}{\partial \boldsymbol{\theta}} = \frac{1}{\pi(\boldsymbol{\theta})} \cdot \frac{\partial \pi(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}}.$
- $\rightarrow \pi(\boldsymbol{\theta}) \cdot \frac{\partial \log [\pi(\boldsymbol{\theta})]}{\partial \boldsymbol{\theta}} = \pi(\boldsymbol{\theta}) \cdot \frac{1}{\pi(\boldsymbol{\theta})} \cdot \frac{\partial \pi(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}}$

Policy Gradient

Definition: Approximate state-value function.

- $V(\mathbf{s}; \boldsymbol{\theta}) = \sum_{\mathbf{a}} \pi(\mathbf{a}|\mathbf{s}; \boldsymbol{\theta}) \cdot Q_{\pi}(\mathbf{s}, \mathbf{a}).$

Policy gradient: Derivative of $V(\mathbf{s}; \boldsymbol{\theta})$ w.r.t. $\boldsymbol{\theta}$.

- $$\begin{aligned} \frac{\partial V(\mathbf{s}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} &= \sum_{\mathbf{a}} \frac{\partial \pi(\mathbf{a}|\mathbf{s}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} \cdot Q_{\pi}(\mathbf{s}, \mathbf{a}) \\ &= \sum_{\mathbf{a}} \pi(\mathbf{a}|\mathbf{s}; \boldsymbol{\theta}) \cdot \frac{\partial \log \pi(\mathbf{a}|\mathbf{s}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} \cdot Q_{\pi}(\mathbf{s}, \mathbf{a}) \end{aligned}$$

- Chain rule: $\frac{\partial \log [\pi(\boldsymbol{\theta})]}{\partial \boldsymbol{\theta}} = \frac{1}{\pi(\boldsymbol{\theta})} \cdot \frac{\partial \pi(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}}.$
- $\rightarrow \pi(\boldsymbol{\theta}) \cdot \frac{\partial \log [\pi(\boldsymbol{\theta})]}{\partial \boldsymbol{\theta}} = \cancel{\pi(\boldsymbol{\theta})} \cdot \cancel{\frac{1}{\pi(\boldsymbol{\theta})}} \cdot \frac{\partial \pi(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}}.$

Policy Gradient

Definition: Approximate state-value function.

- $V(\mathbf{s}; \boldsymbol{\theta}) = \sum_{\mathbf{a}} \pi(\mathbf{a}|\mathbf{s}; \boldsymbol{\theta}) \cdot Q_{\pi}(\mathbf{s}, \mathbf{a}).$

Policy gradient: Derivative of $V(\mathbf{s}; \boldsymbol{\theta})$ w.r.t. $\boldsymbol{\theta}$.

- $$\frac{\partial V(\mathbf{s}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} = \sum_{\mathbf{a}} \frac{\partial \pi(\mathbf{a}|\mathbf{s}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} \cdot Q_{\pi}(\mathbf{s}, \mathbf{a})$$
$$= \sum_{\mathbf{a}} \pi(\mathbf{a}|\mathbf{s}; \boldsymbol{\theta}) \cdot \frac{\partial \log \pi(\mathbf{a}|\mathbf{s}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} \cdot Q_{\pi}(\mathbf{s}, \mathbf{a})$$

- Chain rule: $\frac{\partial \log [\pi(\boldsymbol{\theta})]}{\partial \boldsymbol{\theta}} = \frac{1}{\pi(\boldsymbol{\theta})} \cdot \frac{\partial \pi(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}}.$

- $\rightarrow \pi(\boldsymbol{\theta}) \cdot \frac{\partial \log [\pi(\boldsymbol{\theta})]}{\partial \boldsymbol{\theta}} = \cancel{\pi(\boldsymbol{\theta})} \cdot \frac{1}{\cancel{\pi(\boldsymbol{\theta})}} \cdot \frac{\partial \pi(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}}.$

Policy Gradient

Definition: Approximate state-value function.

$$\bullet V(\mathbf{s}; \boldsymbol{\theta}) = \sum_{\mathbf{a}} \pi(\mathbf{a}|\mathbf{s}; \boldsymbol{\theta}) \cdot Q_{\pi}(\mathbf{s}, \mathbf{a}).$$

Policy gradient: Derivative of $V(\mathbf{s}; \boldsymbol{\theta})$ w.r.t. $\boldsymbol{\theta}$.

$$\begin{aligned} \bullet \frac{\partial V(\mathbf{s}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} &= \sum_{\mathbf{a}} \frac{\partial \pi(\mathbf{a}|\mathbf{s}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} \cdot Q_{\pi}(\mathbf{s}, \mathbf{a}) \\ &= \sum_{\mathbf{a}} \pi(\mathbf{a}|\mathbf{s}; \boldsymbol{\theta}) \cdot \frac{\partial \log \pi(\mathbf{a}|\mathbf{s}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} \cdot Q_{\pi}(\mathbf{s}, \mathbf{a}) \\ &= \mathbb{E}_{\mathbf{A}} \left[\frac{\partial \log \pi(\mathbf{A}|\mathbf{s}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} \cdot Q_{\pi}(\mathbf{s}, \mathbf{A}) \right]. \end{aligned}$$

The expectation is taken w.r.t. the random variable $\mathbf{A} \sim \pi(\cdot | \mathbf{s}; \boldsymbol{\theta})$.

Policy gradient:

$$\frac{\partial V(\mathbf{s}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} = \mathbb{E}_{\mathbf{A} \sim \pi(\cdot | \mathbf{s}; \boldsymbol{\theta})} \left[\frac{\partial \log \pi(\mathbf{A} | \mathbf{s}, \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} \cdot Q_{\pi}(\mathbf{s}, \mathbf{A}) \right].$$

Calculate Policy Gradient

Policy Gradient:
$$\frac{\partial V(\mathbf{s}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} = \mathbb{E}_{\mathbf{A} \sim \pi(\cdot | \mathbf{s}; \boldsymbol{\theta})} \left[\frac{\partial \log \pi(\mathbf{A} | \mathbf{s}, \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} \cdot Q_{\pi}(\mathbf{s}, \mathbf{A}) \right].$$

Calculate Policy Gradient

Policy Gradient:
$$\frac{\partial V(\mathbf{s}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} = \mathbb{E}_{\mathbf{A} \sim \pi(\cdot | \mathbf{s}; \boldsymbol{\theta})} \left[\frac{\partial \log \pi(\mathbf{A} | \mathbf{s}, \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} \cdot Q_{\pi}(\mathbf{s}, \mathbf{A}) \right].$$

1. Randomly sample an action $\hat{\mathbf{a}}$ according to $\pi(\cdot | \mathbf{s}; \boldsymbol{\theta})$.

Calculate Policy Gradient

Policy Gradient: $\frac{\partial V(\mathbf{s}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} = \mathbb{E}_{A \sim \pi(\cdot | \mathbf{s}; \boldsymbol{\theta})} \left[\frac{\partial \log \pi(A | \mathbf{s}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} \cdot Q_{\pi}(\mathbf{s}, A) \right].$

1. Randomly sample an action $\hat{a}$ according to $\pi(\cdot | \mathbf{s}; \boldsymbol{\theta})$.

2. Calculate $\mathbf{g}(\hat{a}, \boldsymbol{\theta}) = \frac{\partial \log \pi(\hat{a} | \mathbf{s}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} \cdot Q_{\pi}(\mathbf{s}, \hat{a})$.

- By the definition of $\mathbf{g}$, $\mathbb{E}_A[\mathbf{g}(A, \boldsymbol{\theta})] = \frac{\partial V(\mathbf{s}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}}$.
- $\mathbf{g}(\hat{a}, \boldsymbol{\theta})$ is an unbiased estimate of $\frac{\partial V(\mathbf{s}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}}$.

Calculate Policy Gradient

Policy Gradient:
$$\frac{\partial V(\mathbf{s}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} = \mathbb{E}_{A \sim \pi(\cdot | \mathbf{s}; \boldsymbol{\theta})} \left[\frac{\partial \log \pi(A | \mathbf{s}, \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} \cdot Q_{\pi}(\mathbf{s}, A) \right].$$

1. Randomly sample an action $\hat{a}$ according to $\pi(\cdot | \mathbf{s}; \boldsymbol{\theta})$.
2. Calculate $\mathbf{g}(\hat{a}, \boldsymbol{\theta}) = \frac{\partial \log \pi(\hat{a} | \mathbf{s}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} \cdot Q_{\pi}(\mathbf{s}, \hat{a})$.
3. Use $\mathbf{g}(\hat{a}, \boldsymbol{\theta})$ as an approximation to the policy gradient $\frac{\partial V(\mathbf{s}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}}$.

Update policy network using policy gradient

Algorithm

1. Observe the state s_t .
2. Randomly sample action a_t according to $\pi(\cdot | s_t; \theta_t)$.

Algorithm

1. Observe the state s_t .
2. Randomly sample action a_t according to $\pi(\cdot|s_t; \theta_t)$.
3. Compute $q_t \approx Q_\pi(s_t, a_t)$ (some estimate).
4. Differentiate policy network: $\mathbf{d}_{\theta,t} = \frac{\partial \log \pi(a_t|s_t, \theta)}{\partial \theta} \Big|_{\theta=\theta_t}$.

1. Observe the state s_t .
2. Randomly sample action a_t according to $\pi(\cdot|s_t; \theta_t)$.
3. Compute $q_t \approx Q_\pi(s_t, a_t)$ (some estimate).
4. Differentiate policy network: $\mathbf{d}_{\theta,t} = \frac{\partial \log \pi(a_t|s_t, \theta)}{\partial \theta} \Big|_{\theta=\theta_t}$.
5. (Approximate) policy gradient: $\mathbf{g}(a_t, \theta_t) = q_t \cdot \mathbf{d}_{\theta,t}$.
6. Update policy network: $\theta_{t+1} = \theta_t + \beta \cdot \mathbf{g}(a_t, \theta_t)$.

1. Observe the state s_t .
2. Randomly sample action a_t according to $\pi(\cdot|s_t; \theta_t)$.
3. Compute $q_t \approx Q(s_t, a_t)$ (some estimate). **How?**
4. Differentiate policy network: $d_{\theta,t} = \frac{\partial \log \pi(a_t|s_t, \theta)}{\partial \theta} \Big|_{\theta=\theta_t}$.
5. (Approximate) policy gradient: $g(a_t, \theta_t) = q_t \cdot d_{\theta,t}$.
6. Update policy network: $\theta_{t+1} = \theta_t + \beta \cdot g(a_t, \theta_t)$.

1. Observe the state s_t .
2. Randomly sample action a_t according to $\pi(\cdot|s_t; \theta_t)$.
3. Compute $Q_t \approx Q(s_t, a_t)$ (some estimator) **How?**

Option 1: REINFORCE.

- Play the game to the end and generate the trajectory:

$$s_1, a_1, r_1, s_2, a_2, r_2, \dots, s_T, a_T, r_T.$$

- Compute the discounted return $u_t = \sum_{k=t}^T \gamma^{k-t} r_k$, for all t .
- Since $Q_\pi(s_t, a_t) = \mathbb{E}[U_t]$, we can use u_t to approximate $Q_\pi(s_t, a_t)$.

1. Observe the state s_t .
2. Randomly sample action a_t according to $\pi(\cdot|s_t; \theta_t)$.
3. Compute $q_t \approx Q(s_t, a_t)$ (some estimator).

How?

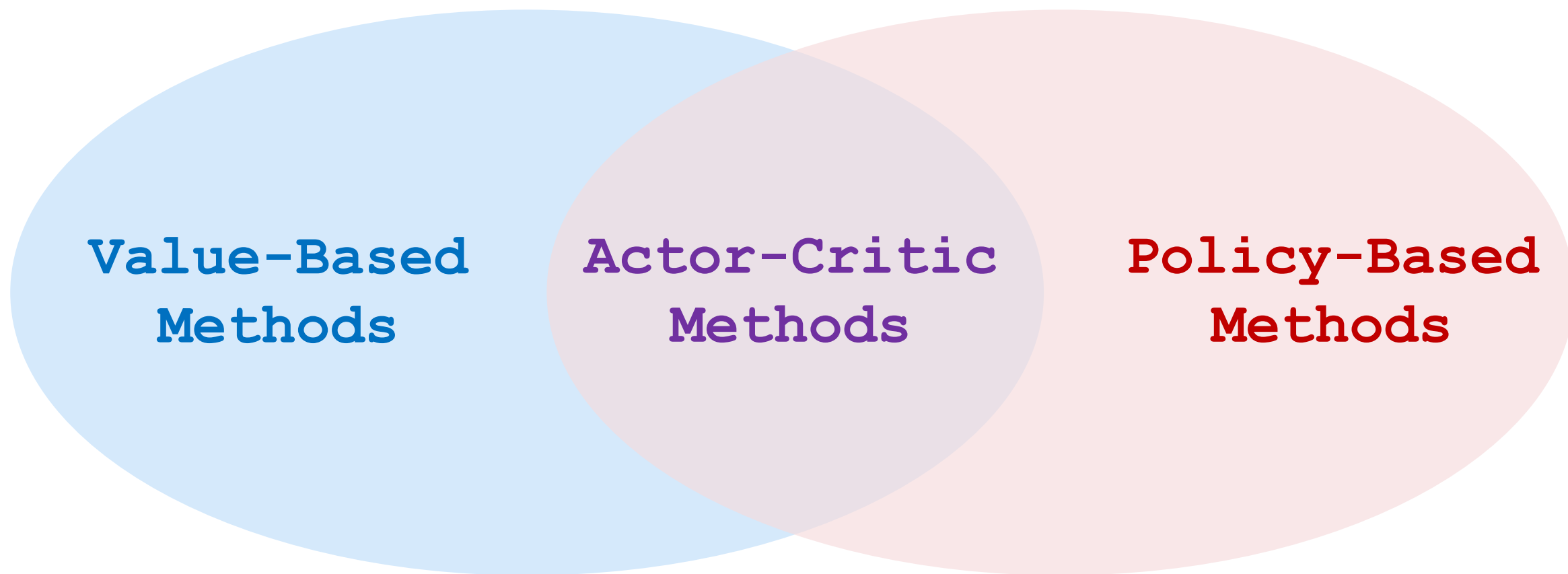
Option 2: Approximate Q_π using a neural network.

5. (Stochastic) policy gradient: $\tilde{g}(\theta_t) \approx q_t \cdot d_{\theta,t}$.
6. Update policy network: $\theta_{t+1} = \theta_t + \beta \cdot \tilde{g}(\theta_t)$.

Summary

- If a good policy function π is known, the agent can be controlled by the policy: randomly sample $a_t \sim \pi(\cdot | s_t)$.
- Approximate policy function $\pi(a|s)$ by policy network $\pi(a|s; \theta)$.
- Learn the policy network by policy gradient algorithm.
- Policy gradient algorithm learn θ that maximizes $\mathbb{E}_s[V(s; \theta)]$.

Actor-Critic Methods



Value Network and Policy Network

State-Value Function Approximation

Definition: State-value function.

- $V_{\pi}(s) = \sum_a \pi(a|s) \cdot Q_{\pi}(s, a).$

State-Value Function Approximation

Definition: State-value function.

- $V_{\pi}(s) = \sum_a \pi(a|s) \cdot Q_{\pi}(s, a).$

Policy network (actor):

- Use neural net $\pi(a|s; \theta)$ to approximate $\pi(a|s).$
- θ : trainable parameters of the neural net.

State-Value Function Approximation

Definition: State-value function.

- $V_{\pi}(s) = \sum_a \pi(a|s) \cdot Q_{\pi}(s, a).$

Policy network (actor):

- Use neural net $\pi(a|s; \theta)$ to approximate $\pi(a|s).$
- θ : trainable parameters of the neural net.

Value network (critic):

- Use neural net $q(s, a; w)$ to approximate $Q_{\pi}(s, a).$
- w : trainable parameters of the neural net.

State-Value Function Approximation

Definition: State-value function.

$$\bullet V_{\pi}(s) = \sum_a \pi(a|s) \cdot Q_{\pi}(s, a) \approx \sum_a \pi(a|s; \theta) \cdot q(s, a; \mathbf{w}).$$

Policy network (actor):

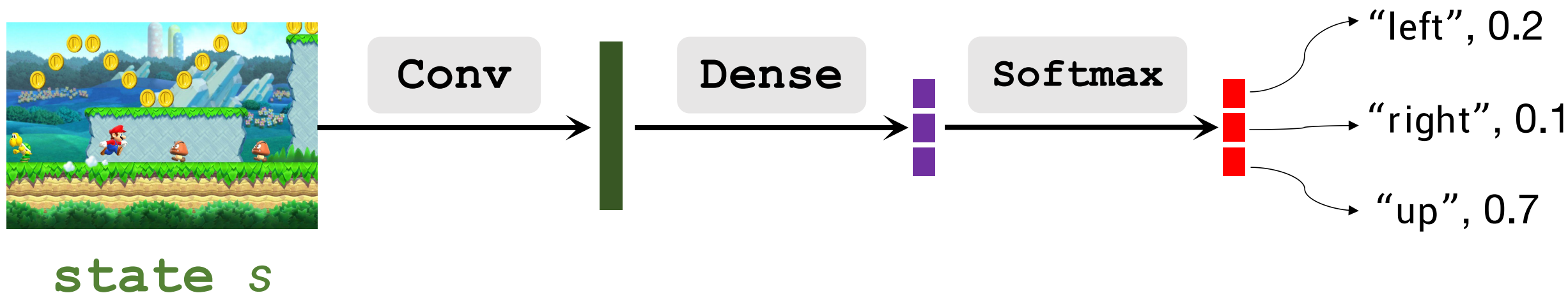
- Use neural net $\pi(a|s; \theta)$ to approximate $\pi(a|s)$.
- θ : trainable parameters of the neural net.

Value network (critic):

- Use neural net $q(s, a; \mathbf{w})$ to approximate $Q_{\pi}(s, a)$.
- $\mathbf{w}$: trainable parameters of the neural net.

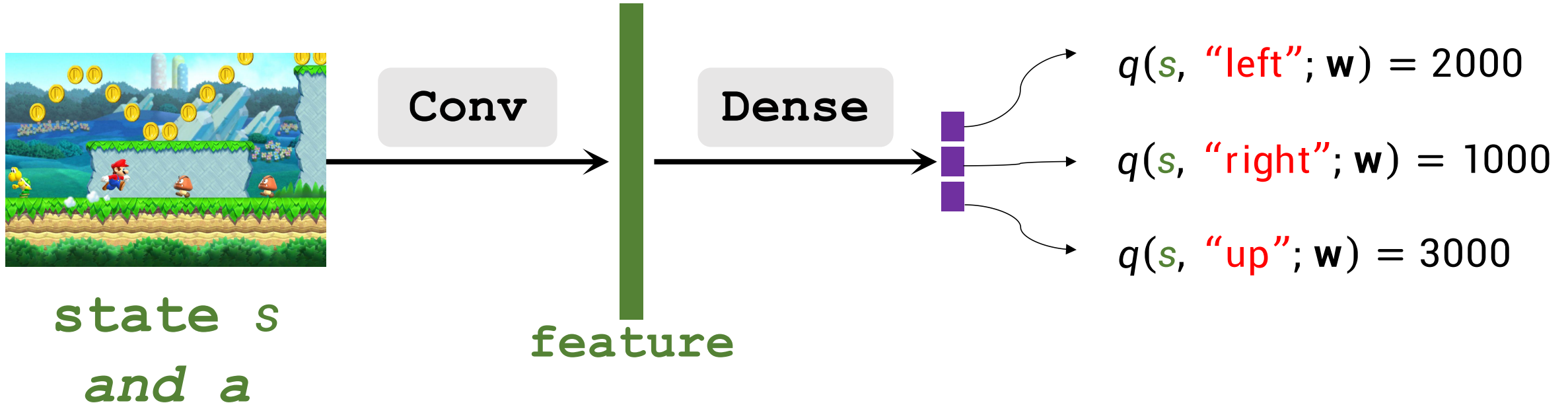
Policy Network (Actor): $\pi(a|s; \theta)$

- Input: **state** s , e.g., a screenshot of Super Mario.
- Output: probability distribution over the **actions**.
- Let $\mathcal{A}$ be the set all actions, e.g., $\mathcal{A} = \{\text{"left"}, \text{"right"}, \text{"up"}\}$.
- $\sum_{a \in \mathcal{A}} \pi(a|s; \theta) = 1$. (That is why we use softmax activation.)



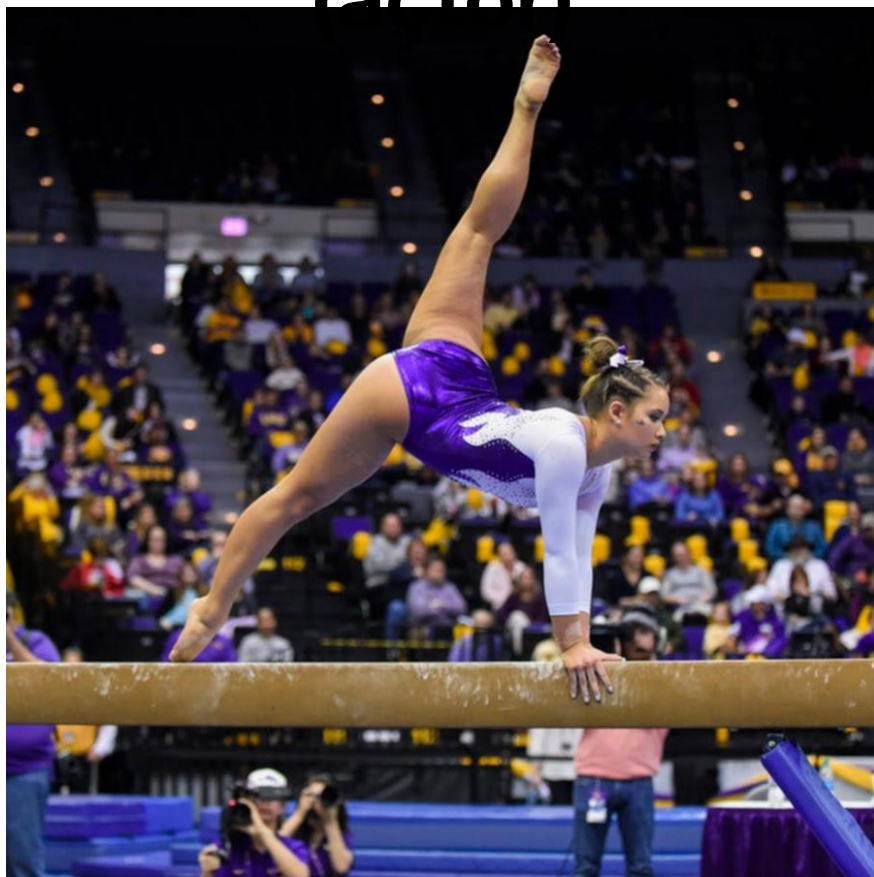
Value Network (Critic): $q(s, a; w)$

- Inputs: state s .
- Output: action-values of all the actions.



Actor-Critic Method

**policy network
(actor)**



value network (critic)



Train the Neural Networks

Train the networks

Definition: State-value function approximated using neural

networks

$$V(s; \theta, w) = \sum_a \pi(a|s; \theta) \cdot q(s, a; w).$$

Training: Update the parameters θ and w .

Train the networks

Definition: State-value function approximated using neural

networks

$$V(\mathbf{s}; \boldsymbol{\theta}, \mathbf{w}) = \sum_a \pi(a|\mathbf{s}; \boldsymbol{\theta}) \cdot q(\mathbf{s}, a; \mathbf{w}).$$

Training: Update the parameters $\boldsymbol{\theta}$ and $\mathbf{w}$.

- Update policy network $\pi(a|\mathbf{s}; \boldsymbol{\theta})$ to increase the state-value $V(\mathbf{s}; \boldsymbol{\theta}, \mathbf{w})$.
 - Actor gradually performs better.
 - Supervision is purely from the value network (critic).

Definition: State-value function approximated using neural

networks

$$V(\mathbf{s}; \boldsymbol{\theta}, \mathbf{w}) = \sum_a \pi(a|\mathbf{s}; \boldsymbol{\theta}) \cdot q(\mathbf{s}, a; \mathbf{w}).$$

Training: Update the parameters $\boldsymbol{\theta}$ and $\mathbf{w}$.

- Update policy network $\pi(a|\mathbf{s}; \boldsymbol{\theta})$ to increase the state-value $V(\mathbf{s}; \boldsymbol{\theta}, \mathbf{w})$.
 - Actor gradually performs better.
 - Supervision is purely from the value network (critic).
- Update value network $q(\mathbf{s}, a; \mathbf{w})$ to better estimate the return.
 - Critic's judgement becomes more accurate.
 - Supervision is purely from the rewards.

Definition: State-value function approximated using neural

networks

$$V(s; \theta, w) = \sum_a \pi(a|s; \theta) \cdot q(s, a; w).$$

Training: Update the parameters θ and w .

1. Observe the state s_t .
2. Randomly sample action a_t according to $\pi(\cdot|s_t; \theta_t)$.
3. Perform a_t and observe new state s_{t+1} and reward r_t .
4. Update w (in value network) using temporal difference (TD).
5. Update θ (in policy network) using policy gradient.

Update value network q using TD

- Compute $q(s_t, a_t; \mathbf{w}_t)$ and $q(s_{t+1}, a_{t+1}; \mathbf{w}_t)$.
- TD target: $y_t = r_t + \gamma \cdot q(s_{t+1}, a_{t+1}; \mathbf{w}_t)$.

Update value network q using TD

- Compute $q(s_t, a_t; \mathbf{w}_t)$ and $q(s_{t+1}, a_{t+1}; \mathbf{w}_t)$.
- TD target: $y_t = r_t + \gamma \cdot q(s_{t+1}, a_{t+1}; \mathbf{w}_t)$.
- Loss: $L(\mathbf{w}) = \frac{1}{2} [q(s_t, a_t; \mathbf{w}) - y_t]^2$.
- Gradient descent: $\mathbf{w}_{t+1} = \mathbf{w}_t - a \cdot \frac{\partial L(\mathbf{w})}{\partial \mathbf{w}} \Big|_{\mathbf{w}=\mathbf{w}_t}$.

Update policy network π using policy gradient

Definition: State-value function approximated using neural

networks

$$V(\mathbf{s}; \boldsymbol{\theta}, \mathbf{w}) = \sum_a \pi(a|\mathbf{s}; \boldsymbol{\theta}) \cdot q(\mathbf{s}, a; \mathbf{w}).$$

Policy gradient: Derivative of $V(\mathbf{s}_t; \boldsymbol{\theta}, \mathbf{w})$ w.r.t. $\boldsymbol{\theta}$.

- Let $\mathbf{g}(a, \boldsymbol{\theta}) = \frac{\partial \log \pi(a|\mathbf{s}, \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} \cdot q(\mathbf{s}_t, a; \mathbf{w})$.
- $\frac{\partial V(\mathbf{s}; \boldsymbol{\theta}, \mathbf{w}_t)}{\partial \boldsymbol{\theta}} = \mathbb{E}_A[\mathbf{g}(A, \boldsymbol{\theta})]$.

Update policy network π using policy gradient

Definition: State-value function approximated using neural

networks

$$V(\mathbf{s}; \boldsymbol{\theta}, \mathbf{w}) = \sum_a \pi(a|\mathbf{s}; \boldsymbol{\theta}) \cdot q(\mathbf{s}, a; \mathbf{w}).$$

Policy gradient: Derivative of $V(\mathbf{s}_t; \boldsymbol{\theta}, \mathbf{w})$ w.r.t. $\boldsymbol{\theta}$.

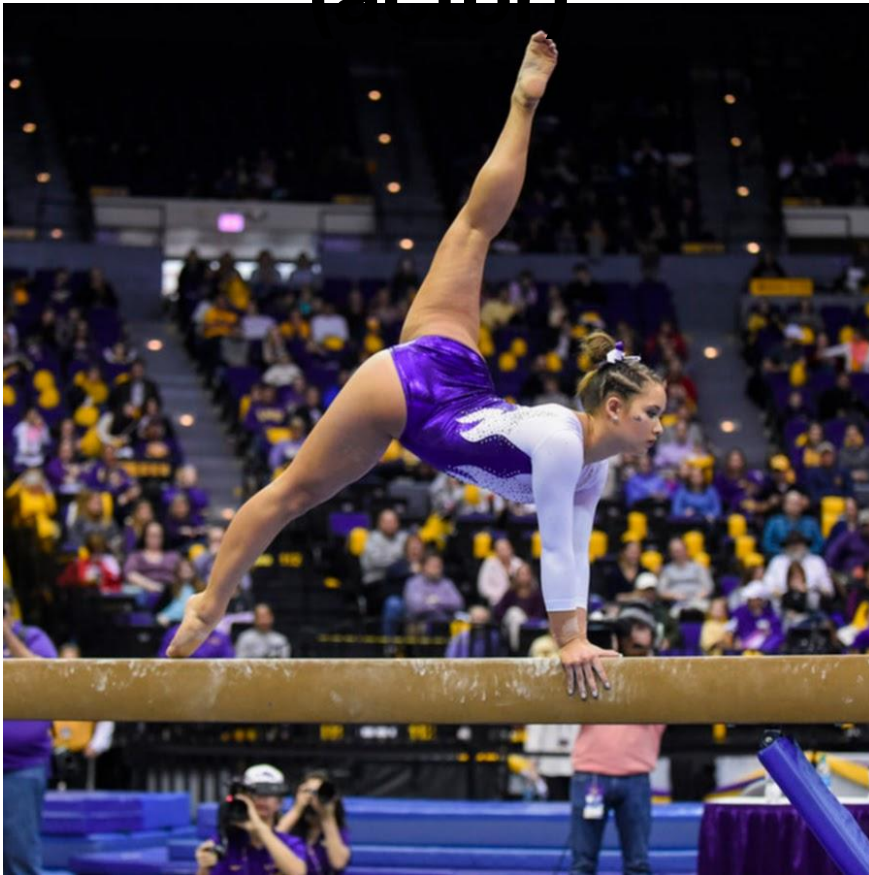
- Let $\mathbf{g}(a, \boldsymbol{\theta}) = \frac{\partial \log \pi(a|\mathbf{s}_t, \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} \cdot q(\mathbf{s}_t, a; \mathbf{w})$.
- $\frac{\partial V(\mathbf{s}; \boldsymbol{\theta}, \mathbf{w}_t)}{\partial \boldsymbol{\theta}} = \mathbb{E}_A[\mathbf{g}(A, \boldsymbol{\theta})]$.

Algorithm: Update policy network using stochastic policy gradient.

- Random sampling: $a \sim \pi(\cdot | \mathbf{s}_t; \boldsymbol{\theta}_t)$. (Thus $\mathbf{g}(a, \boldsymbol{\theta})$ is unbiased.)
- Stochastic gradient ascent: $\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \beta \cdot \mathbf{g}(a, \boldsymbol{\theta}_t)$.

Actor-Critic Method

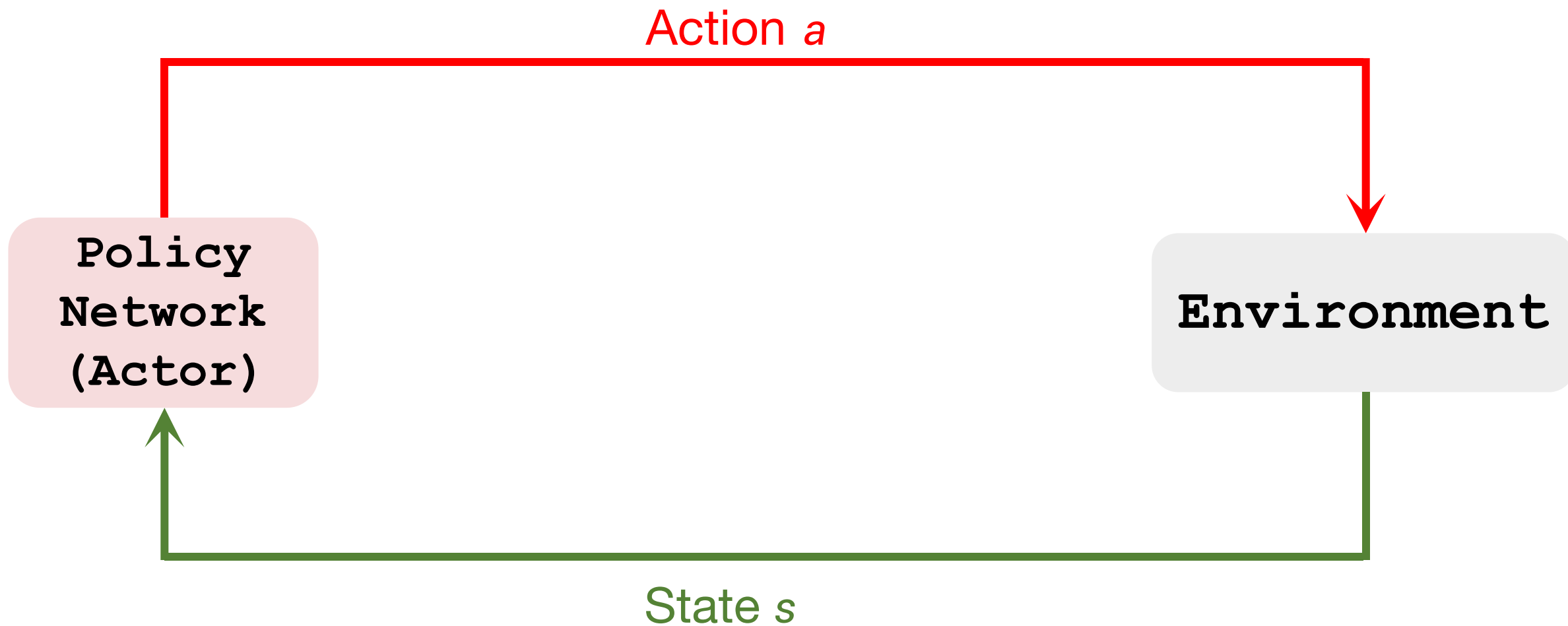
**policy network
(actor)**



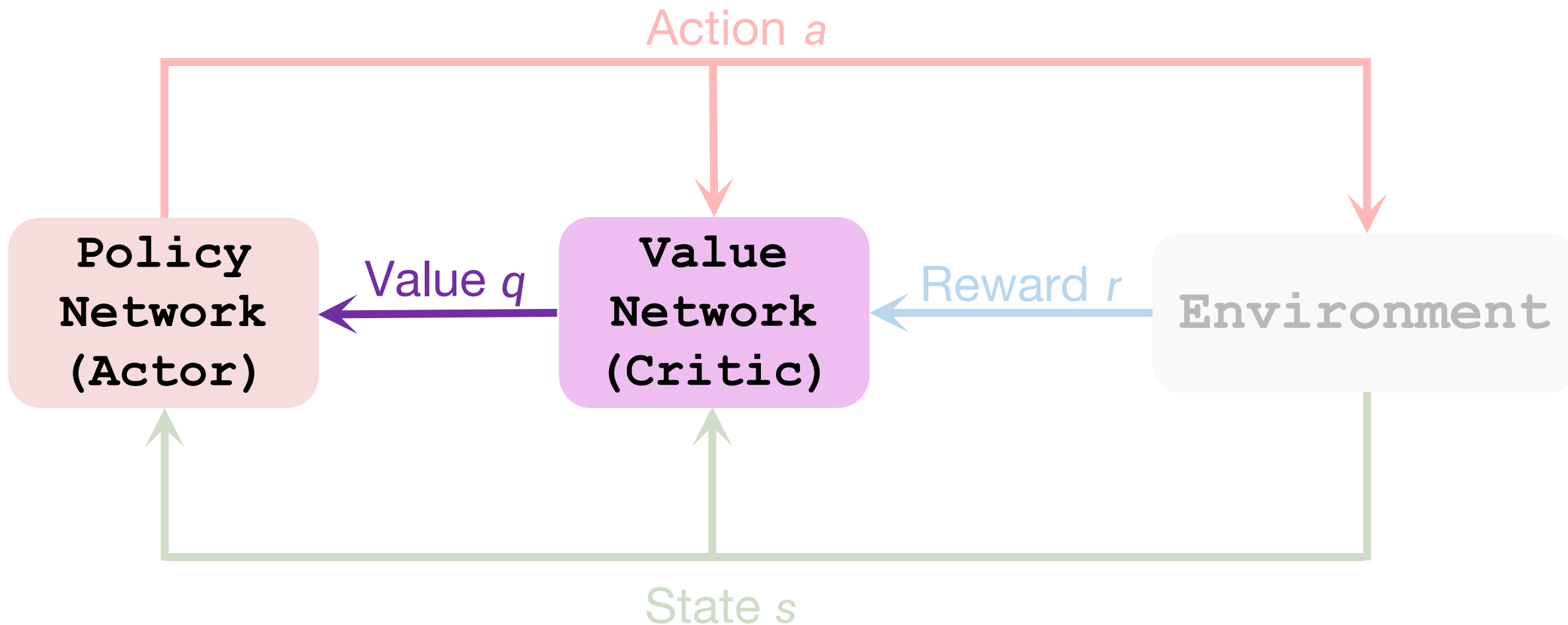
value network (critic)



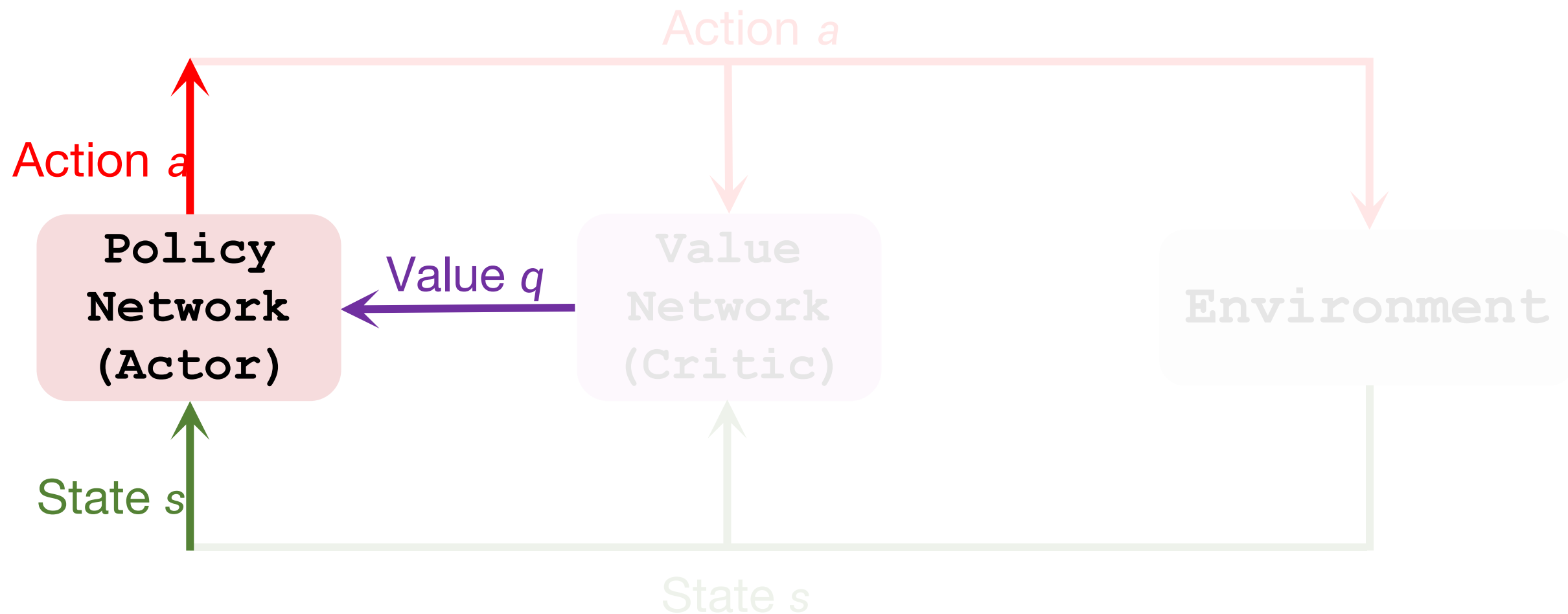
Actor-Critic Method



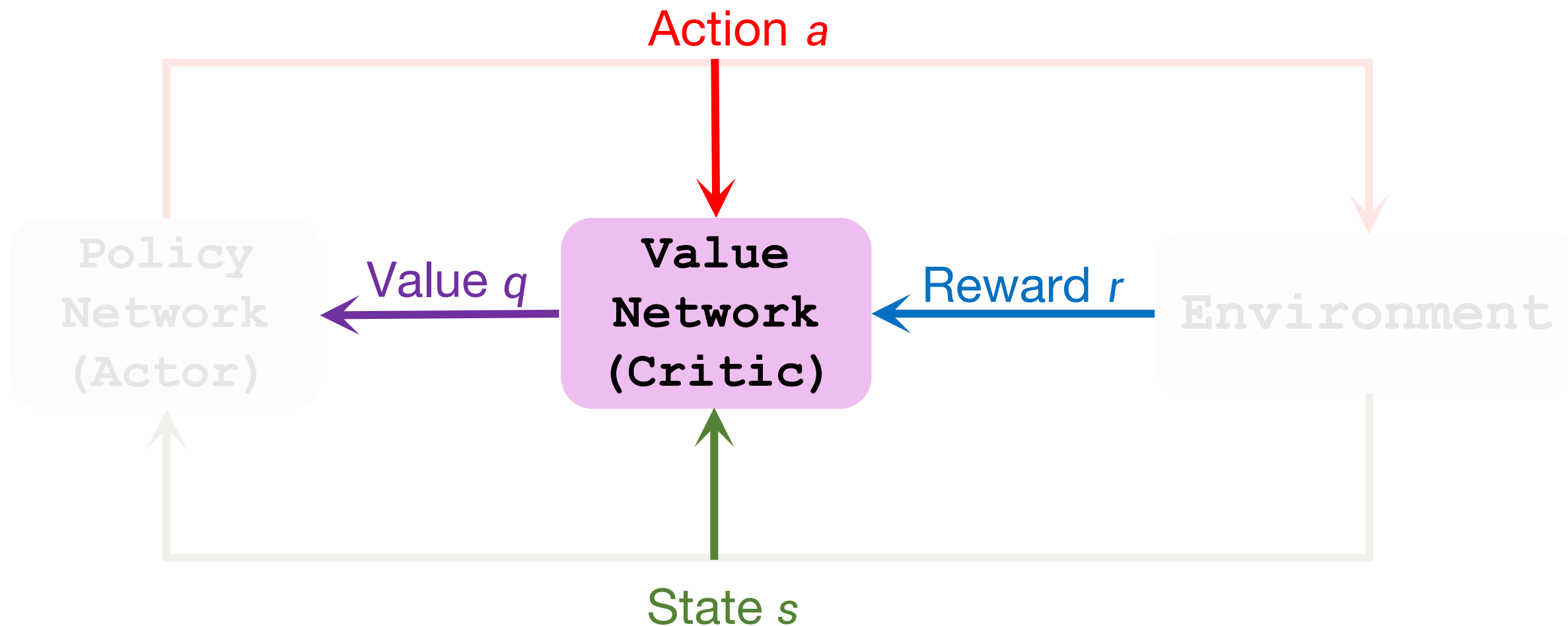
Actor-Critic Method



Actor-Critic Method: Update Actor



Actor-Critic Method: Update Critic



Summary of Algorithm

1. Observe state s_t and randomly sample $a_t \sim \pi(\cdot | s_t; \theta_t)$.
2. Perform a_t ; then environment gives new state s_{t+1} and reward r_t .
3. Randomly sample $\tilde{a}_{t+1} \sim \pi(\cdot | s_{t+1}; \theta_t)$. (Do not perform $\tilde{a}_{t+1}$!)

Summary of Algorithm

1. Observe state s_t and randomly sample $a_t \sim \pi(\cdot | s_t; \theta_t)$.
2. Perform a_t ; then environment gives new state s_{t+1} and reward r_t .
3. Randomly sample $\tilde{a}_{t+1} \sim \pi(\cdot | s_{t+1}; \theta_t)$. (Do not perform $\tilde{a}_{t+1}$!)
4. Evaluate value network: $q_t = q(s_t, a_t; w_t)$ and $q_{t+1} = q(s_{t+1}, \tilde{a}_{t+1}; w_t)$.
5. Compute TD error: $\delta_t = q_t - \underbrace{(\tilde{r}_t + q_{t+1})}_{\text{TD Target}}$.

Summary of Algorithm

1. Observe state s_t and randomly sample $a_t \sim \pi(\cdot | s_t; \theta_t)$.
2. Perform a_t ; then environment gives new state s_{t+1} and reward r_t .
3. Randomly sample $\tilde{a}_{t+1} \sim \pi(\cdot | s_{t+1}; \theta_t)$. (Do not perform $\tilde{a}_{t+1}$!)
4. Evaluate value network: $q_t = q(s_t, a_t; \mathbf{w}_t)$ and $q_{t+1} = q(s_{t+1}, \tilde{a}_{t+1}; \mathbf{w}_t)$.
5. Compute TD error: $\delta_t = q_t - (r_t + \gamma \cdot q_{t+1})$.
6. Differentiate value network: $\mathbf{d}_{w,t} = \frac{\partial q(s_t, a_t; \mathbf{w})}{\partial \mathbf{w}} \Big|_{\mathbf{w}=\mathbf{w}_t}$.
7. Update value network: $\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \cdot \delta_t \cdot \mathbf{d}_{w,t}$.

Summary of Algorithm

1. Observe state s_t and randomly sample $a_t \sim \pi(\cdot | s_t; \theta_t)$.
2. Perform a_t ; then environment gives new state s_{t+1} and reward r_t .
3. Randomly sample $\tilde{a}_{t+1} \sim \pi(\cdot | s_{t+1}; \theta_t)$. (Do not perform $\tilde{a}_{t+1}$!)
4. Evaluate value network: $q_t = q(s_t, a_t; w_t)$ and $q_{t+1} = q(s_{t+1}, \tilde{a}_{t+1}; w_t)$.
5. Compute TD error: $\delta_t = q_t - (r_t + \gamma \cdot q_{t+1})$.
6. Differentiate value network: $d_{w,t} = \frac{\partial q(s_t, a_t; w)}{\partial w} \Big|_{w=w_t}$.
7. Update value network: $w_{t+1} = w_t - \alpha \cdot \delta_t \cdot d_{w,t}$.
8. Differentiate policy network: $d_{\theta,t} = \frac{\partial \log \pi(a_t | s_t, \theta)}{\partial \theta} \Big|_{\theta=\theta_t}$.
9. Update policy network: $\theta_{t+1} = \theta_t + \beta \cdot q_t \cdot d_{\theta,t}$.

Summary

Policy Network and Value Network

Definition: State-value function.

- $V_{\pi}(s) = \sum_a \pi(a|s) \cdot Q_{\pi}(s, a).$

Definition: function approximation using neural networks.

- Approximate policy function $\pi(a|s)$ by $\pi(a|s; \theta)$ (actor).
- Approximate value function $Q_{\pi}(s, a)$ by $q(s, a; w)$ (critic).

Roles of Actor and Critic

During training

- Agent is controlled by policy network (**actor**): $a_t \sim \pi(\cdot | s_t; \theta)$.
- Value network q (**critic**) provides the **actor** with supervision.

Roles of Actor and Critic

During training

- Agent is controlled by policy network (**actor**): $a_t \sim \pi(\cdot | s_t; \theta)$.
- Value network q (**critic**) provides the **actor** with supervision.

After training

- Agent is controlled by policy network (**actor**): $a_t \sim \pi(\cdot | s_t; \theta)$.
- Value network q (**critic**) will not be used.

Update the **policy network (actor)** by **policy gradient**.

- Seek to increase state-value: $V(\mathbf{s}; \boldsymbol{\theta}, \mathbf{w}) = \sum_a \pi(a|\mathbf{s}; \boldsymbol{\theta}) \cdot q(\mathbf{s}, a; \mathbf{w})$.
- Compute policy gradient: $\frac{\partial V(\mathbf{s}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} = \mathbb{E}_A \left[\frac{\partial \log \pi(A|\mathbf{s}, \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} \cdot q(\mathbf{s}, A; \mathbf{w}) \right]$.
- Perform gradient ascent.

Update the **policy network (actor)** by **policy gradient**.

- Seek to increase state-value: $V(\mathbf{s}; \boldsymbol{\theta}, \mathbf{w}) = \sum_a \pi(a|\mathbf{s}; \boldsymbol{\theta}) \cdot q(\mathbf{s}, a; \mathbf{w})$.
- Compute policy gradient: $\frac{\partial V(\mathbf{s}; \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} = \mathbb{E}_A \left[\frac{\partial \log \pi(A|\mathbf{s}, \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} \cdot q(\mathbf{s}, A; \mathbf{w}) \right]$.
- Perform gradient ascent.

Update the **value network (critic)** by **TD learning**.

- Predicted action-value: $q_t = q(\mathbf{s}_t, \mathbf{a}_t; \mathbf{w})$.
- TD target: $y_t = r_t + \gamma \cdot q(\mathbf{s}_{t+1}, \mathbf{a}_{t+1}; \mathbf{w})$
- Gradient: $\frac{\partial (q_t - y_t)^2 / 2}{\partial \mathbf{w}} = (q_t - y_t) \cdot \frac{\partial q(\mathbf{s}_t, \mathbf{a}_t; \mathbf{w})}{\partial \mathbf{w}}$.
- Perform gradient descent.

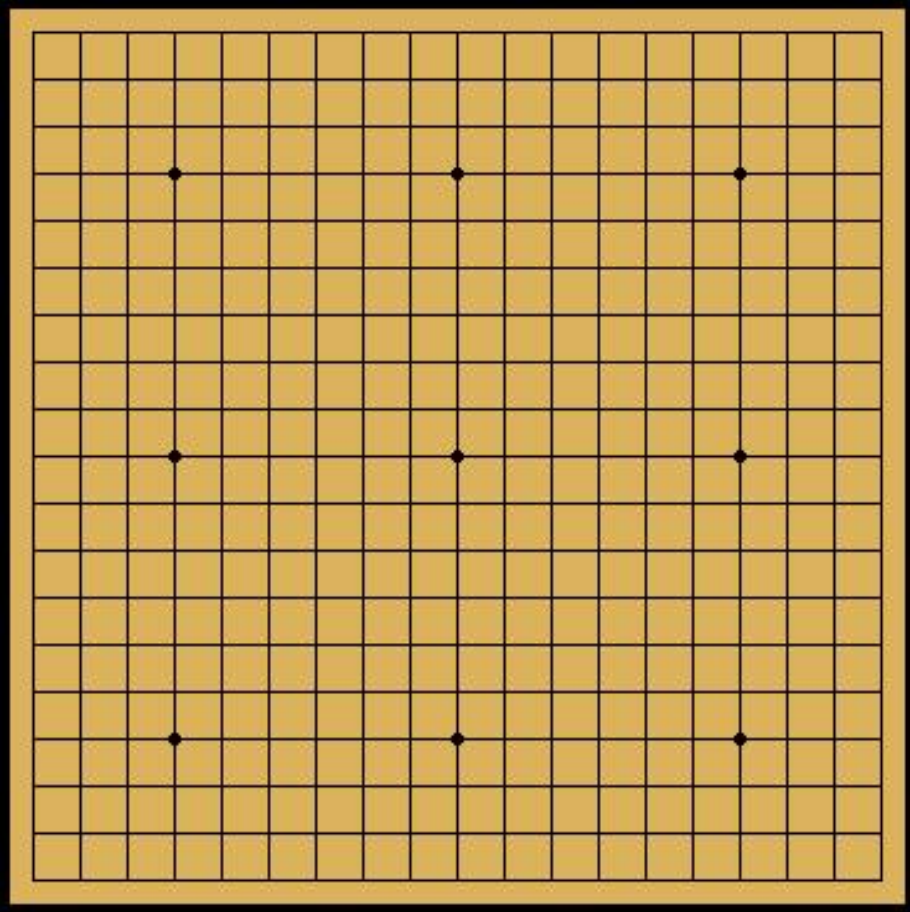


- What taught in this lecture is not exactly the same to the original AlphaGo papers [1,2].
- There are simplifications here.

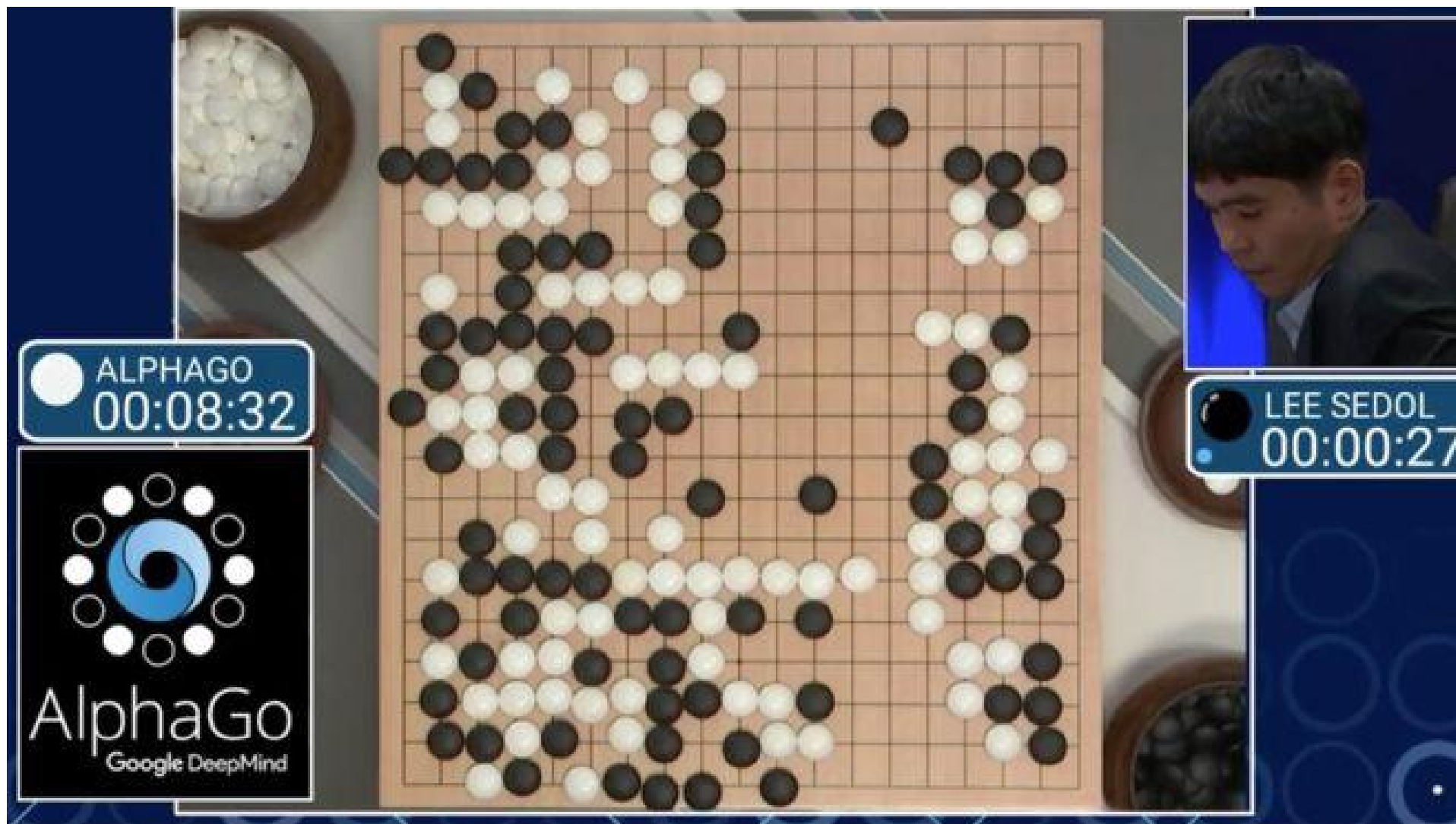
Reference

1. Silver and others: [Mastering the game of Go with deep neural networks and tree search](#). *Nature*, 2016.
2. Silver and others: [Mastering the game of Go without human knowledge](#). *Nature*, 2017.

Go Game



- The standard Go board has a 19×19 grid of lines, containing 361 points.
- **State**: arrangement of black, white, and space.
 - State s can be a $19 \times 19 \times 2$ tensor of 0 or 1.
 - (AlphaGo actually uses a $19 \times 19 \times 17$ or a $19 \times 19 \times 48$ tensor to store other information.)
- **Action**: place a stone on a vacant point.
 - Action space: $\mathcal{A} \subset \{1, 2, 3, \dots, 361\}$.
- Go is very complex.
 - Number of possible sequence of **actions** is



High-Level Ideas

Training in 3 steps:

1. Initialize policy network using behavior cloning.
(Supervised learning from human experience.)
2. Train the policy network using policy gradient. (Two policy networks play against each other.)
3. After training the policy network, use it to train a value network.

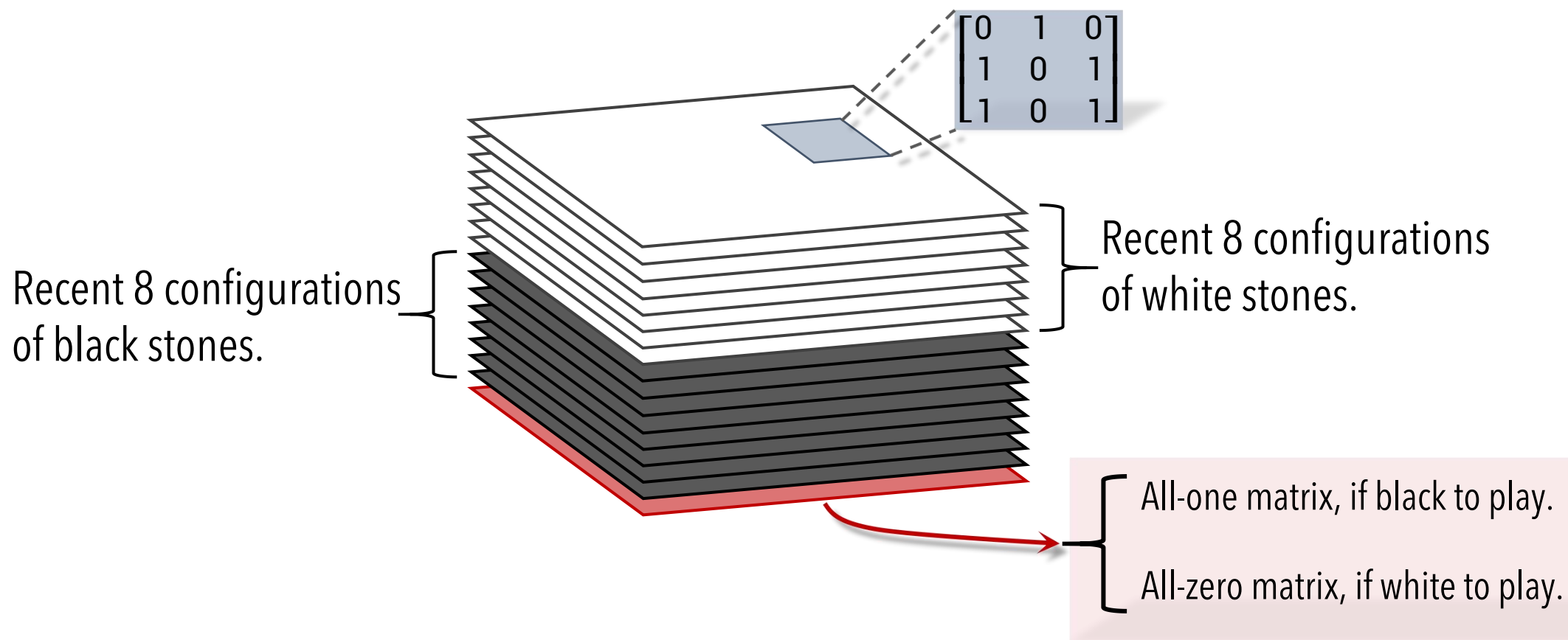
Training in 3 steps:

1. Initialize policy network using behavior cloning.
(Supervised learning from human experience.)
2. Train the policy network using policy gradient. (Two policy networks play against each other.)
3. After training the policy network, use it to train a value network.

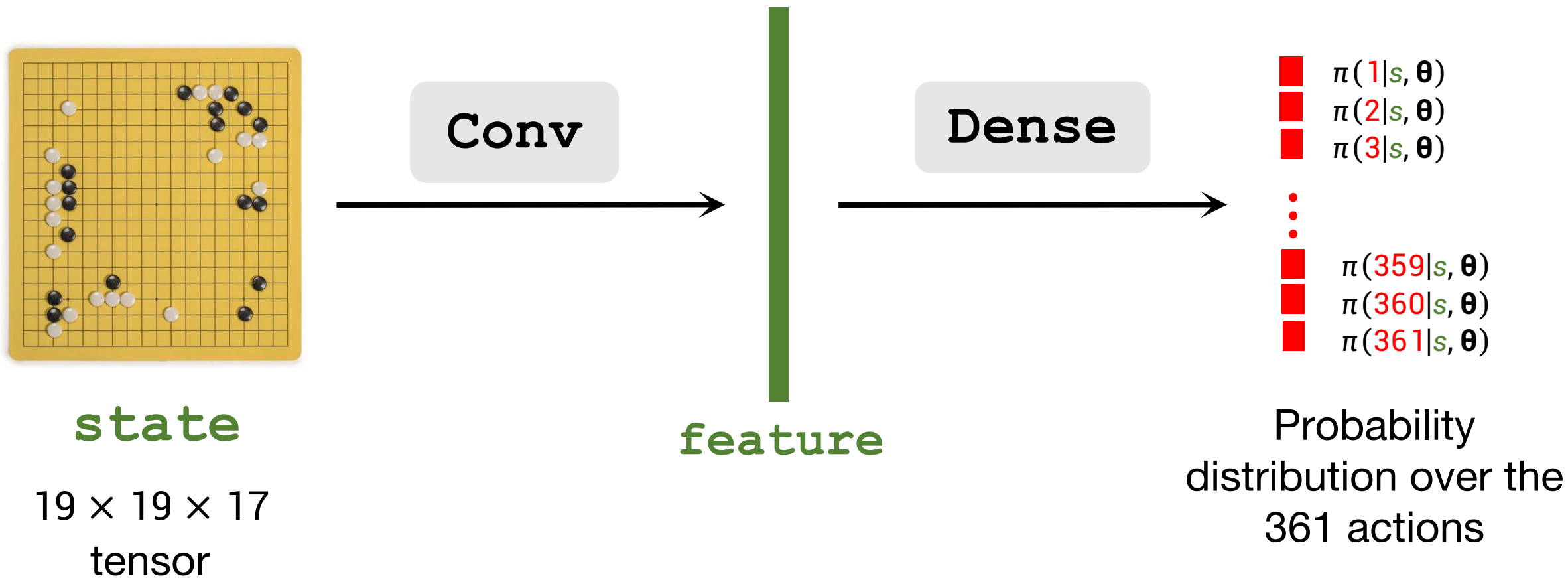
Execution (actually play Go games):

- Do Monte Carlo Tree Search (MCTS) using the policy and value networks.

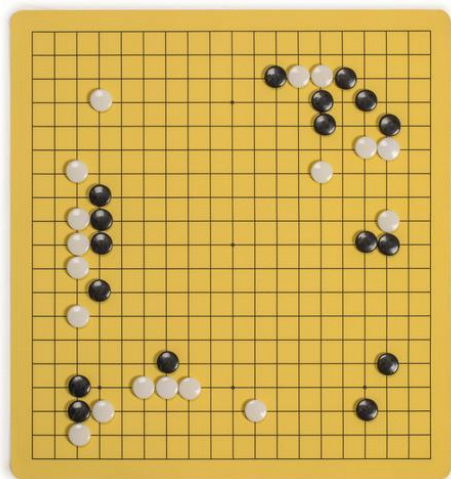
State (of AlphaGo Zero)



Policy Network (of AlphaGo Zero)



Policy Network (of AlphaGo)



state

$19 \times 19 \times 48$
tensor

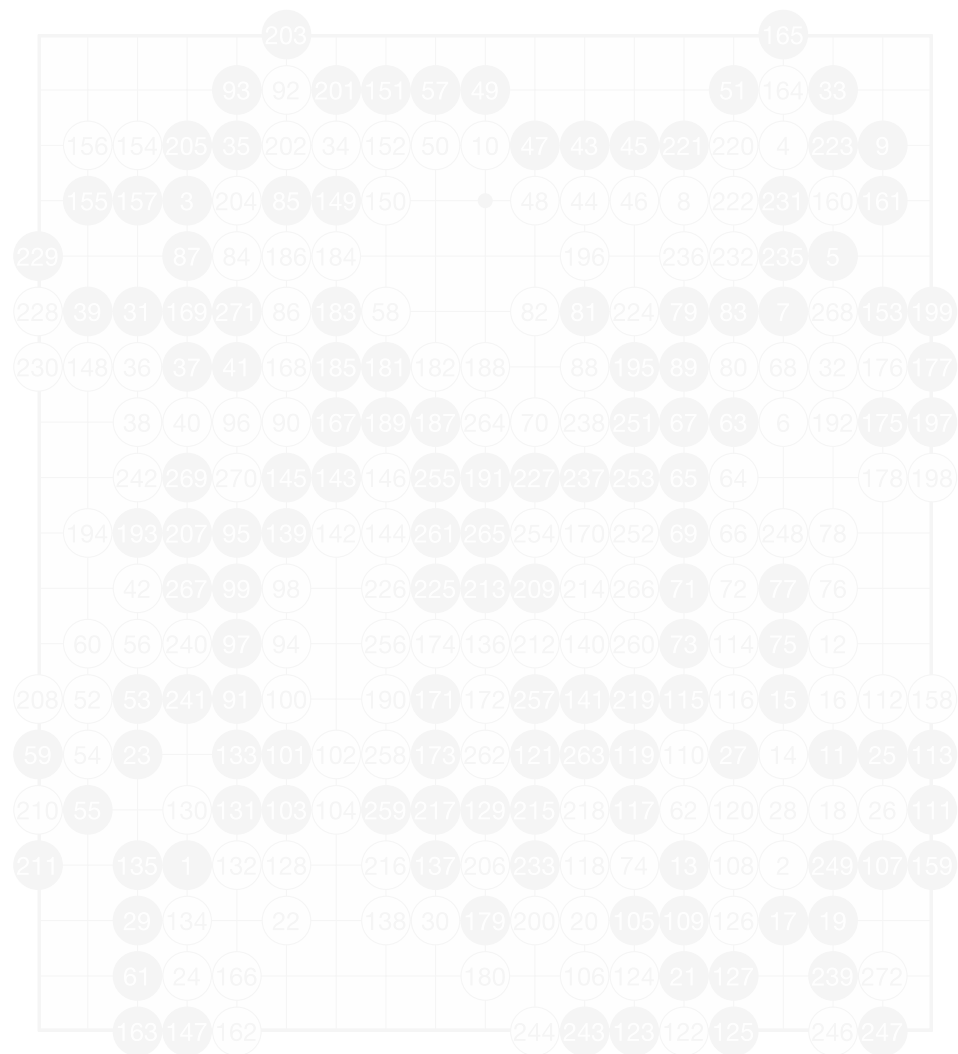
Conv

$\pi(1|s, \theta)$
 $\pi(2|s, \theta)$
 $\pi(3|s, \theta)$
 $\vdots$
 $\pi(359|s, \theta)$
 $\pi(360|s, \theta)$
 $\pi(361|s, \theta)$

Probability
distribution over the
361 actions

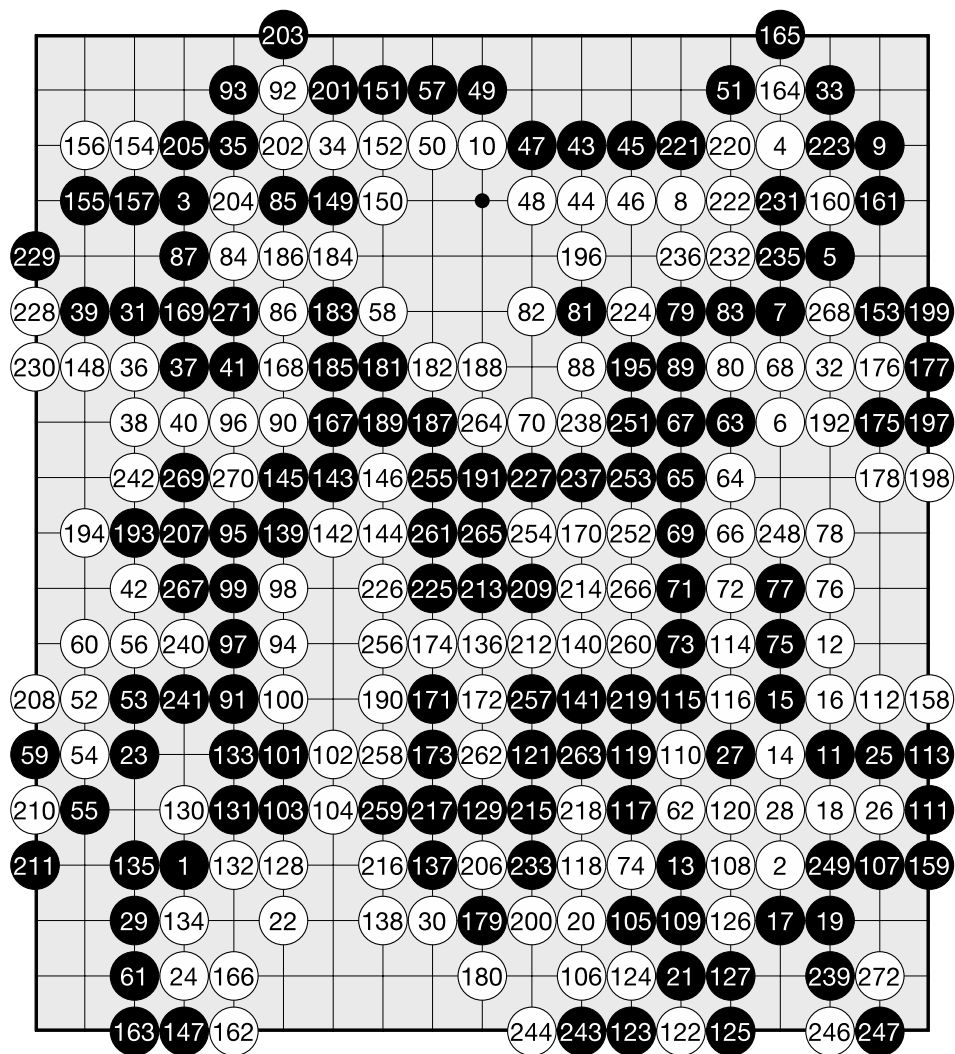
Initialize Policy Network by Behavior Cloning

Learning from human's record



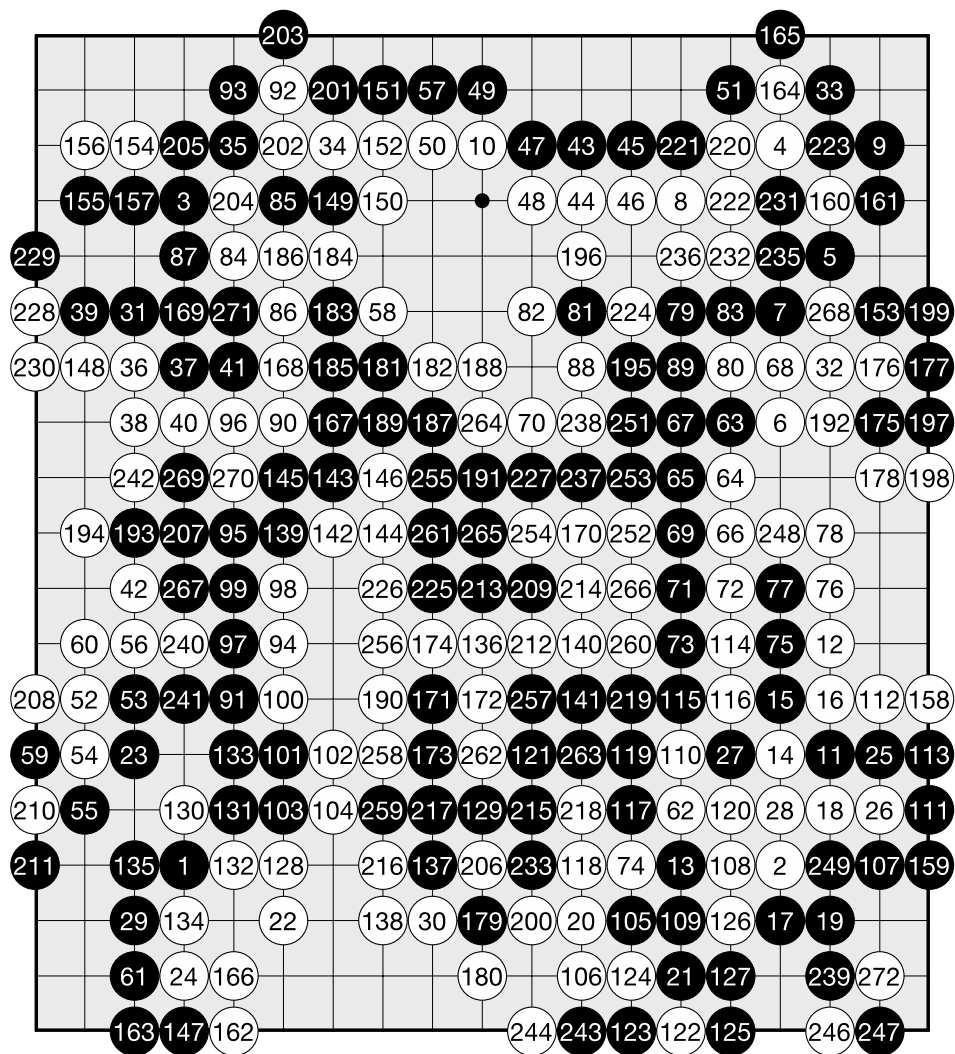
- Initially, the network's parameters are random.
 - If two policy networks play against each other, they would do random actions.
 - It would take very long before they make reasonable actions.

Learning from human's record



- Initially, the network's parameters are random.
- Human' sequences of actions have been recorded. (KGS dataset has 160K games' records.)

Learning from human's record



- Initially, the network's parameters are random.
- Human' sequences of actions have been recorded. (KGS dataset has 160K games' records.)
- Behavior cloning: Let the policy network imitate human players.
- After behavior cloning, the policy network beats advanced amateurs.

Behavior cloning is not reinforcement learning!

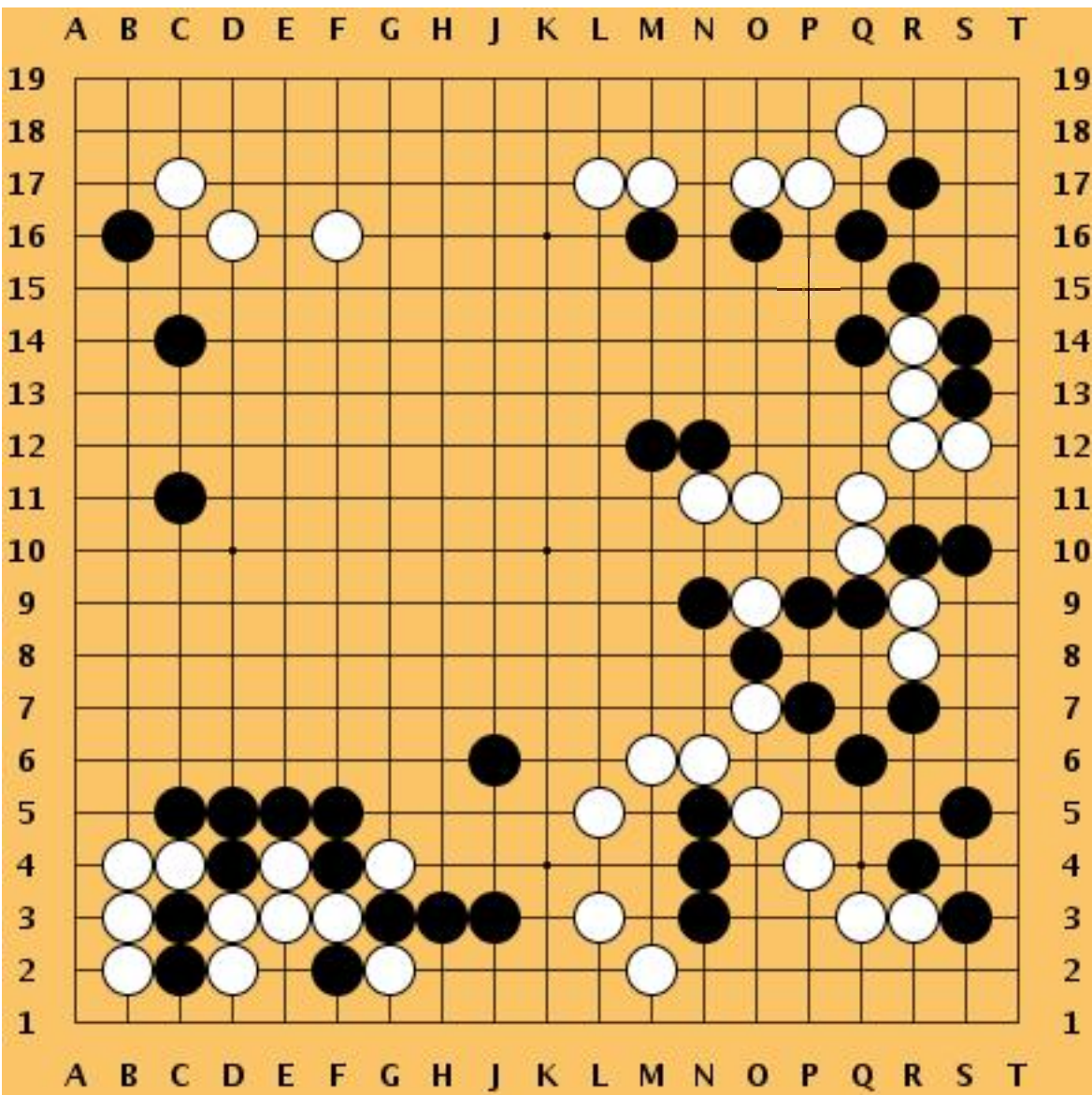
- Reinforcement learning: Supervision is from rewards given by the environment.
- **Imitation learning**: Supervision is from experts' actions.
 - Agent does not see rewards.
 - Agent simply imitates experts' actions.

Behavior cloning is not reinforcement learning!

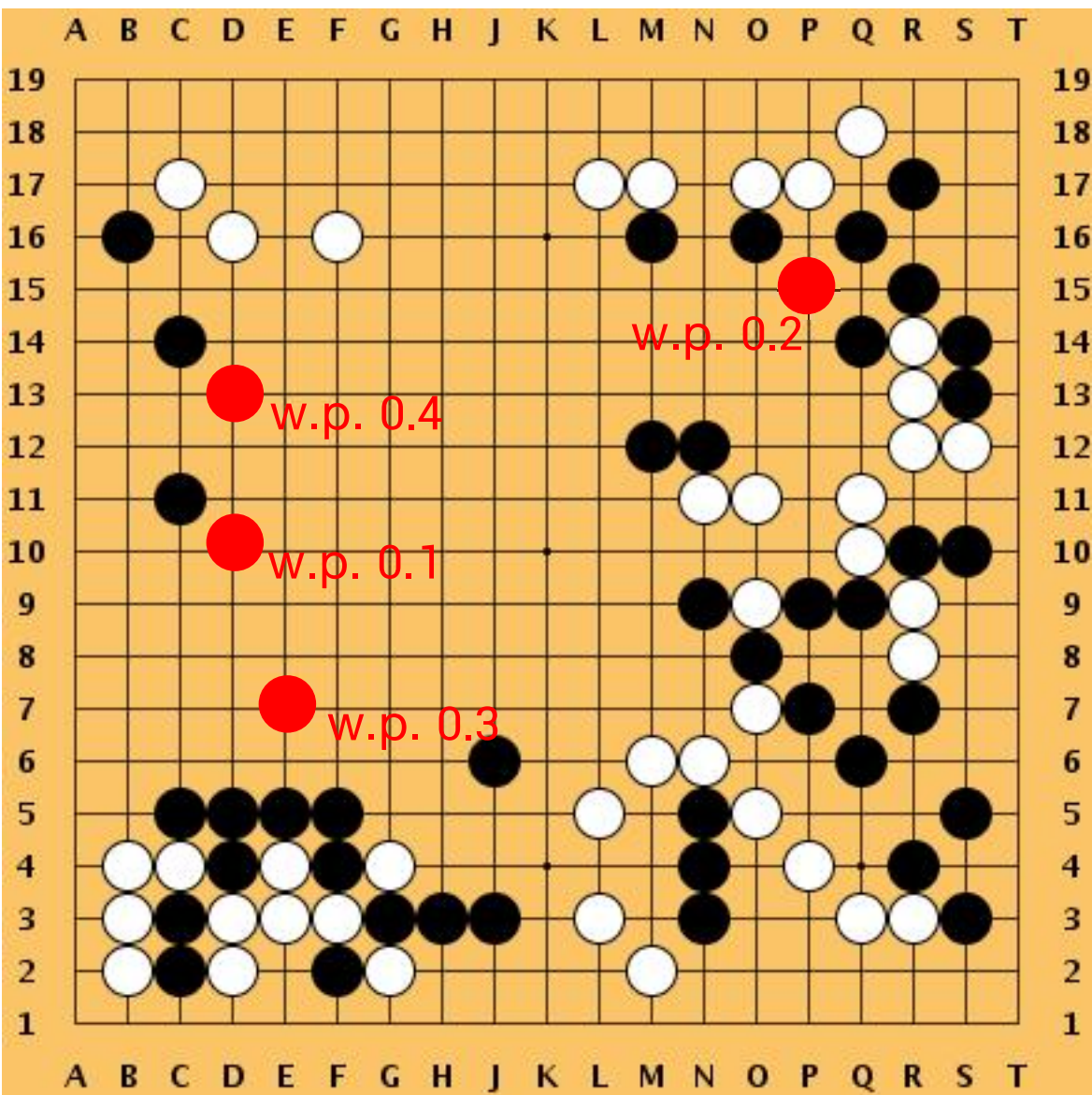
- Reinforcement learning: Supervision is from rewards given by the environment.
- **Imitation learning**: Supervision is from experts' actions.
- **Behavior cloning** is one of the **imitation learning** methods.
- **Behavior cloning** is simply classification or regression.

Behavior Cloning

- Observe this state s_t .

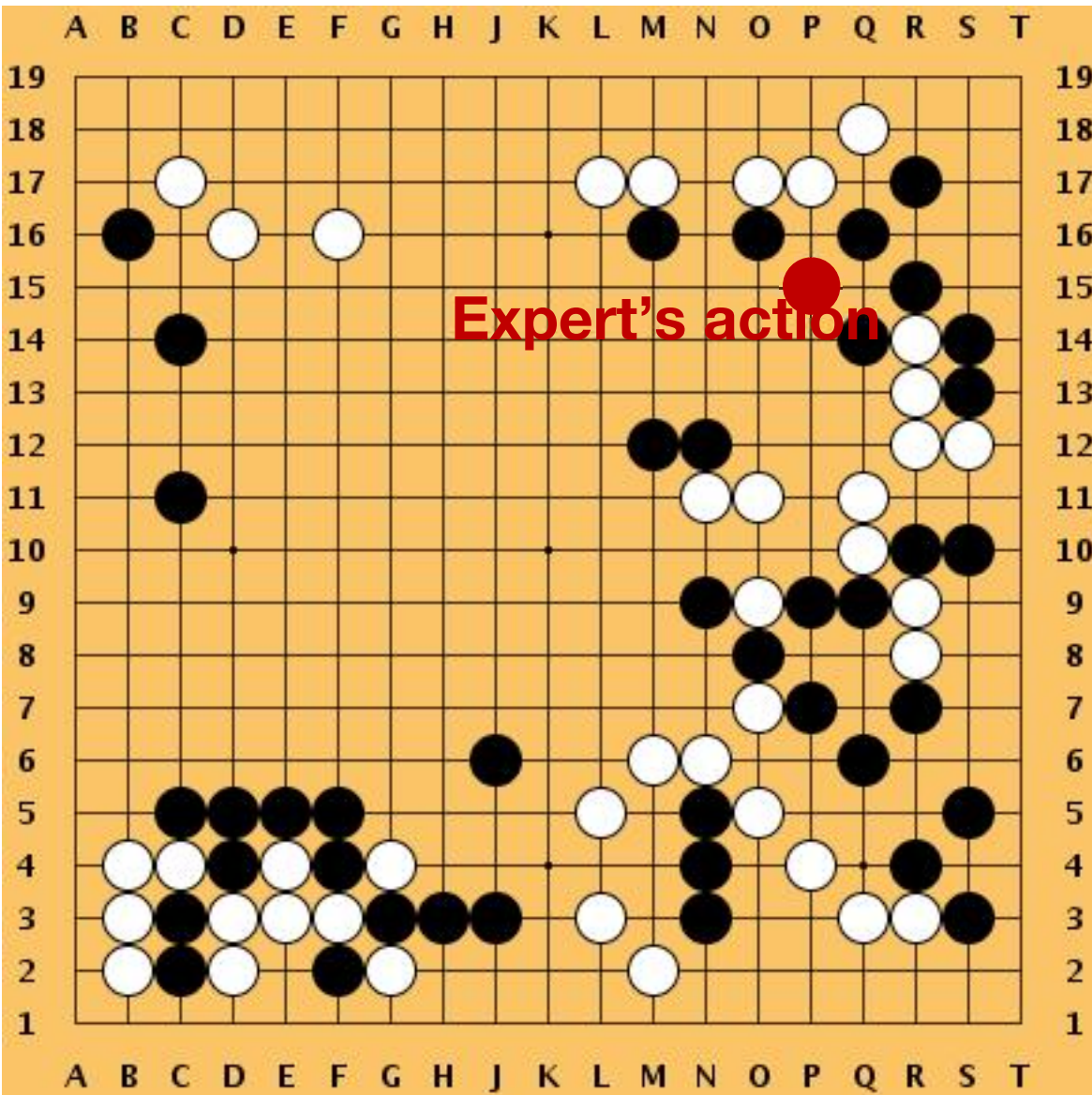


Behavior Cloning



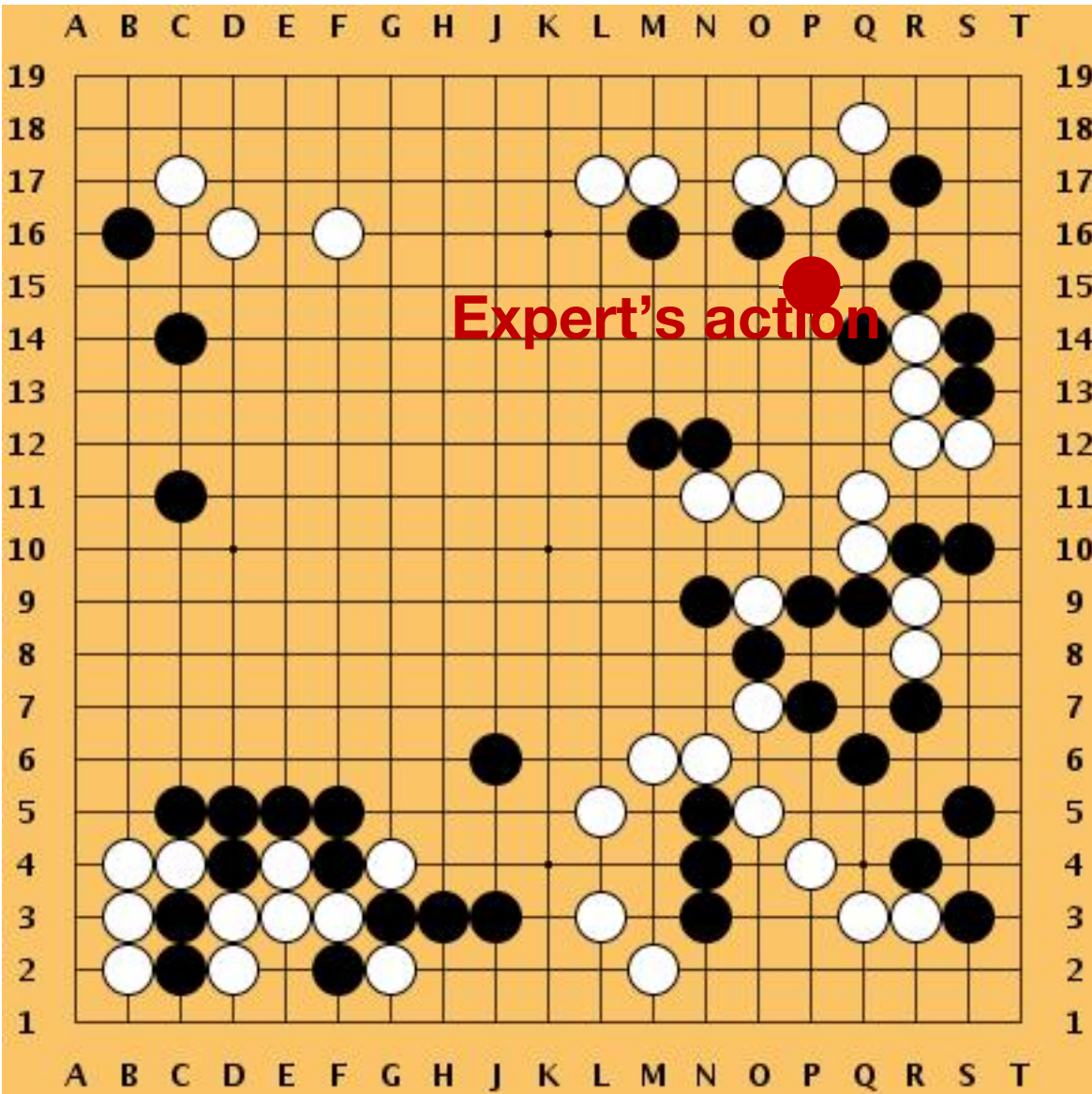
- Observe this state s_t .
- Policy network makes a prediction:
 $\mathbf{p}_t = [\pi(1|s_t, \boldsymbol{\theta}), \dots, \pi(361|s_t, \boldsymbol{\theta})] \in (0,1)^{361}.$

Behavior Cloning



- Observe this state s_t .
- Policy network makes a prediction:
 $\mathbf{p}_t = [\pi(1|s_t, \boldsymbol{\theta}), \dots, \pi(361|s_t, \boldsymbol{\theta})] \in (0,1)^{361}$.
- The expert's action is $a_t^* = 281$.
- Let $\mathbf{y}_t \in \{0,1\}^{361}$ be the one-hot vector of $a_t^* = 281$.

Behavior Cloning



- Observe this state s_t .
- Policy network makes a prediction:
 $\mathbf{p}_t = [\pi(1|s_t, \theta), \dots, \pi(361|s_t, \theta)] \in (0,1)^{361}$.
- The expert's action is $a_t^* = 281$.
- Let $\mathbf{y}_t \in \{0,1\}^{361}$ be the one-hot vector of $a_t^* = 281$.
- Loss = CrossEntropy($\mathbf{y}_t$, $\mathbf{p}_t$).
- Use gradient descent to update policy network.

After behavior cloning...

- Suppose the current state s_t has appeared in training data.
- The policy network imitates expert's action a_t . (Which is a good action!)

After behavior cloning...

- Suppose the current state s_t has appeared in training data.
- The policy network imitates expert's action a_t . (Which is a good

Question: Why bother doing RL after behavior cloning?

- What if the current state s_t has not appeared in training data?
- Then the policy network's action a_t can be bad.

After behavior cloning...

- Suppose the current state s_t has appeared in training data.
- The policy network imitates expert's action a_t . (Which is a good

Question: Why bother doing RL after behavior cloning?

- What if the current state s_t has not appeared in training data?
- Then the policy network's action a_t can be bad.
- Number of possible states is too big.
- There is a big chance that s_t has not appeared in training data.

Behavior cloning + RL beats behavior cloning with 80% chance.

Train Policy Network Using Policy Gradient

Reinforcement learning of policy network

- Player's parameters are the latest parameters of the policy network.
- Opponent's parameters are randomly selected from previous iterations.

Player
(Agent)

policy network with
latest param

V.S.

Opponent
(Environment)

policy network with
old param

Reinforcement learning of policy network

Reinforcement learning is guided by rewards.

- Suppose a game ends at step T .
- Rewards:
 - $r_1 = r_2 = r_3 = \dots = r_{T-1} = 0$. (When the game has not ended.)
 - $r_T = +1$ (winner).
 - $r_T = -1$ (loser).

Reinforcement learning of policy network

Reinforcement learning is guided by rewards.

- Suppose a game ends at step T .
- Rewards:
 - $r_1 = r_2 = r_3 = \dots = r_{T-1} = 0$. (When the game has not ended.)
 - $r_T = +1$ (winner).
 - $r_T = -1$ (loser).
- Recall that return is defined by $u_t = \sum_{i=t}^T r_i$. (No discount here.)
- Winner's returns: $u_1 = u_2 = \dots = u_T = +1$.
- Loser's returns: $u_1 = u_2 = \dots = u_T = -1$.

Policy gradient: Derivative of state-value function $V(\mathbf{s}; \boldsymbol{\theta})$ w.r.t. $\boldsymbol{\theta}$.

- Recall that $\frac{\partial \log \pi(\mathbf{a}_t | \mathbf{s}_t, \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} \cdot Q_{\pi}(\mathbf{s}_t, \mathbf{a}_t)$ is approximate policy gradient.

Policy gradient: Derivative of state-value function $V(\mathbf{s}; \boldsymbol{\theta})$ w.r.t. $\boldsymbol{\theta}$.

- Recall that $\frac{\partial \log \pi(\mathbf{a}_t | \mathbf{s}_t, \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} \cdot Q_{\pi}(\mathbf{s}_t, \mathbf{a}_t)$ is approximate policy gradient.
- By definition, the action value is $Q_{\pi}(\mathbf{s}_t, \mathbf{a}_t) = \mathbb{E}[U_t | \mathbf{s}_t, \mathbf{a}_t]$.
- Thus, we can replace $Q_{\pi}(\mathbf{s}_t, \mathbf{a}_t)$ by the observed return u_t .
- Approximate policy gradient: $\frac{\partial \log \pi(\mathbf{a}_t | \mathbf{s}_t, \boldsymbol{\theta})}{\partial \boldsymbol{\theta}} \cdot u_t$.

Train policy network using policy gradient

Repeat the followings:

- Two policy networks play a game to the end. (Player v.s. Opponent.)
- Get a trajectory: $s_1, a_1, s_2, a_2, \dots, s_T, a_T$.
- After the game ends, update the player's policy network.
 - The player's returns: $u_1 = u_2 = \dots = u_T$. (Either +1 or -1.)
 - Sum of approximate policy gradients: $\mathbf{g}_\theta = \sum_{t=1}^T \frac{\partial \log \pi(a_t | s_t, \theta)}{\partial \theta} \cdot u_t$.
 - Update policy network: $\theta \leftarrow \theta + \beta \cdot \mathbf{g}_\theta$.

Play Go using the policy network

- The policy network π has been learned.
- Observing the current state s_t , randomly sample action

$$a_t \sim \pi(\cdot | s_t, \theta).$$

Play Go using the policy network

- The policy network π has been learned.
- Observing the current state s_t , randomly sample action

$$a_t \sim \pi(\cdot | s_t, \theta).$$

- The learned policy network π is strong, but not strong enough.
- A small mistake may change the game result.

Train the Value Network

Definition: State-value function.

- $V_{\pi}(s) = \mathbb{E}[U_t | S_t = s]$, where $U_t = +1$ (if win) and -1 (if lose) .
- The expectation is taken with respect to
 - The future actions $A_t, A_{t+1}, \dots, A_T$.
 - The future states $S_{t+1}, S_{t+2}, \dots, S_T$

State-Value Function

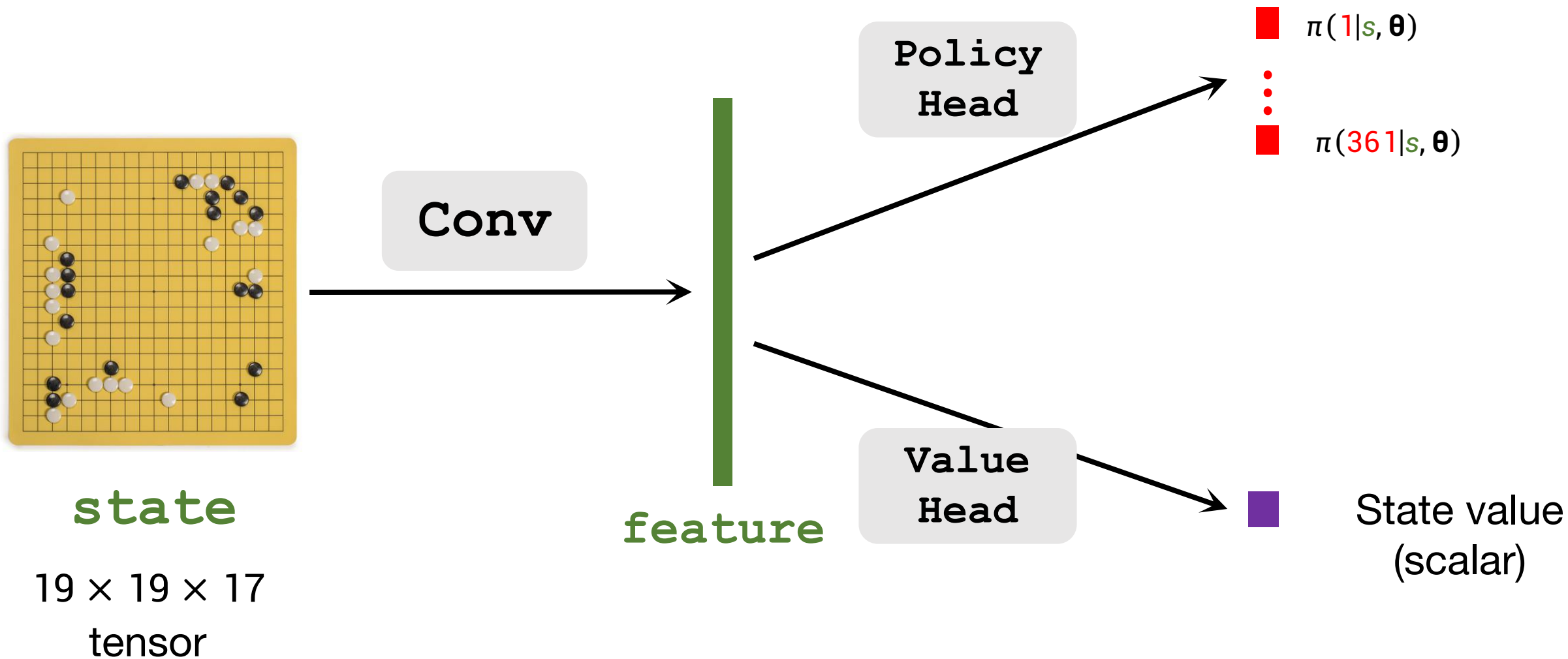
Definition: State-value function.

- $V_{\pi}(s) = \mathbb{E}[U_t | S_t = s]$, where $U_t = +1$ (if win) and -1 (if lose) .
- The expectation is taken with respect to
 - The future actions $A_t, A_{t+1}, \dots, A_T$.
 - The future states $S_{t+1}, S_{t+2}, \dots, S_T$

Approximate state-value function using a value network.

- Use a neural network $v(s; \mathbf{w})$ to approximate $V_{\pi}(s)$.
- It evaluate how good the current situation is.

Policy Value Networks (AlphaGo Zero)



Train the value network

After finishing training the policy network, train the value network.

Train the value network

After finishing training the policy network, train the value network.

Repeat the following:

1. Play a game to the end.
 - If win, let $u_1 = u_2 = \dots = u_T = +1$.
 - If lose, let $u_1 = u_2 = \dots = u_T = -1$.
2. Loss:
$$L = \sum_{t=1}^T \frac{1}{2} [v(\mathbf{s}_t; \mathbf{w}) - u_t]^2.$$
3. Update:
$$\mathbf{w} \leftarrow \mathbf{w} - a \cdot \frac{\partial L}{\partial \mathbf{w}}.$$

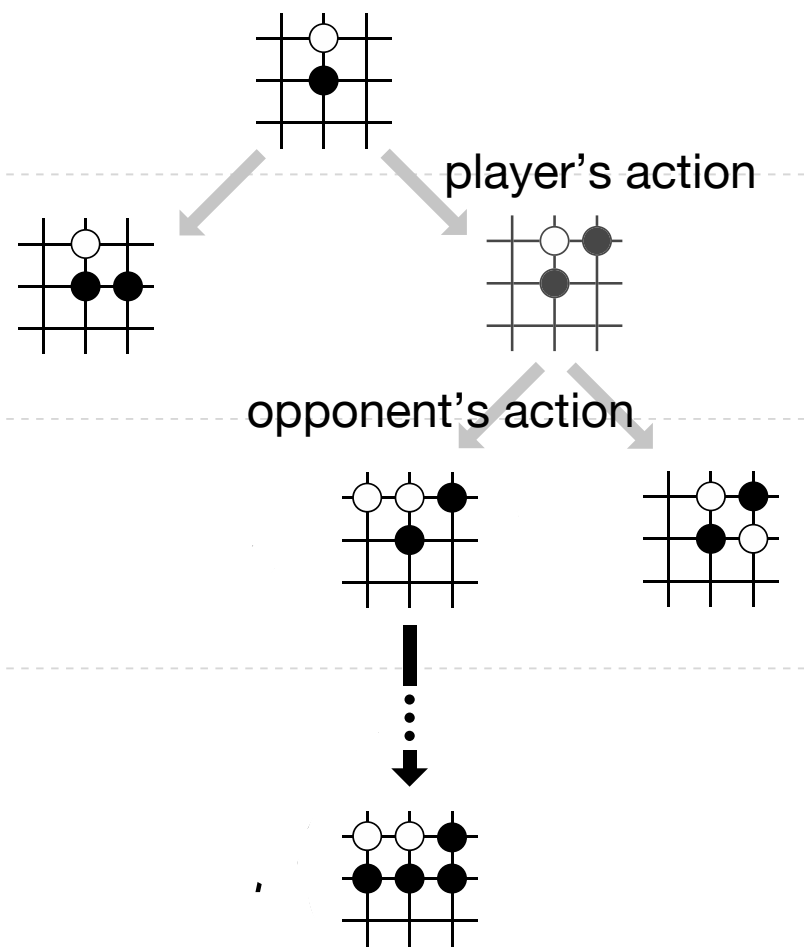
Monte Carlo Tree Search

How do human play Go?

Players must look ahead two or more steps.

- Suppose I choose action a_t .
- What will be my opponent's action? (His action leads to state s_{t+1} .)
- What will I be my action a_{t+1} upon observing s_{t+1} ?
- What will be my opponent's action? (His action leads to state s_{t+2} .)
- What will I be my action a_{t+2} upon observing s_{t+3} ?
- If I can exhaustively foresee all the possibilities, I will win.

Select actions by look-ahead search



Main idea

- Randomly select an action ***a***.
- Look ahead and see whether ***a*** leads to win or lose.
- Repeat this procedure many times.
- Choose the action ***a*** that has the highest score.

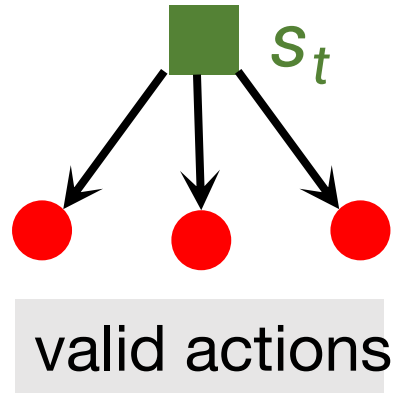
Monte Carlo Tree Search (MCTS)

Every simulation of Monte Carlo Tree Search (MCTS) has 4 steps:

1. **Selection:** The player makes an action a . (Imaginary action; not actual move.)
2. **Expansion:** The opponent makes an action; the state updates. (Also imaginary action; made by the policy network.)
3. **Evaluation:** Evaluate the state-value and get score v . Play the game to the end to receive reward r .
4. **Backup:** Use the score $\frac{v+r}{2}$ to update action-values of a .

Step 1: Selection

Question: Observing s_t , which action shall we explore?

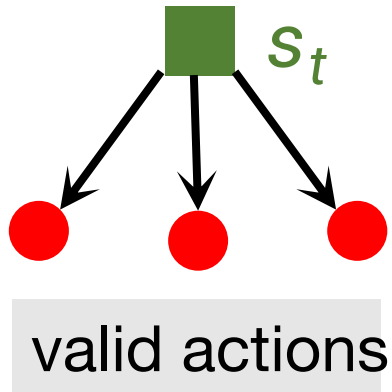


Step 1: Selection

Question: Observing s_t , which action shall we explore?

First, for all the valid actions a , calculate the score:

$$\text{score}(a) = Q(a) + \eta \cdot \frac{\pi(a | s_t; \theta)}{1 + N(a)}.$$



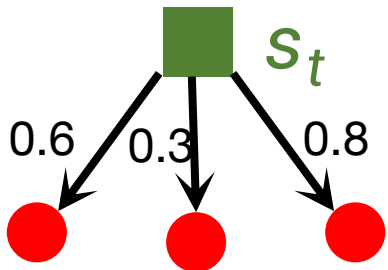
- $Q(a)$: Action-value computed by MCTS. (To be defined.)
- $\pi(a | s_t; \theta)$: The learned policy network.
- $N(a)$: Given s_t , how many times we have selected a so far.

Step 1: Selection

Question: Observing s_t , which action shall we explore?

First, for all the valid actions a , calculate the score:

$$\text{score}(a) = Q(a) + \eta \cdot \frac{\pi(a | s_t; \theta)}{1 + N(a)}.$$



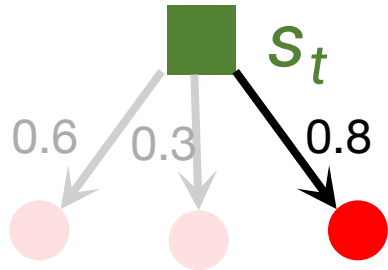
- $Q(a)$: Action-value computed by MCTS. (To be defined.)
- $\pi(a | s_t; \theta)$: The learned policy network.
- $N(a)$: Given s_t , how many times we have selected a so far.

Step 1: Selection

Question: Observing s_t , which action shall we explore?

First, for all the valid actions a , calculate the score:

$$\text{score}(a) = Q(a) + \eta \cdot \frac{\pi(a | s_t; \theta)}{1 + N(a)}.$$



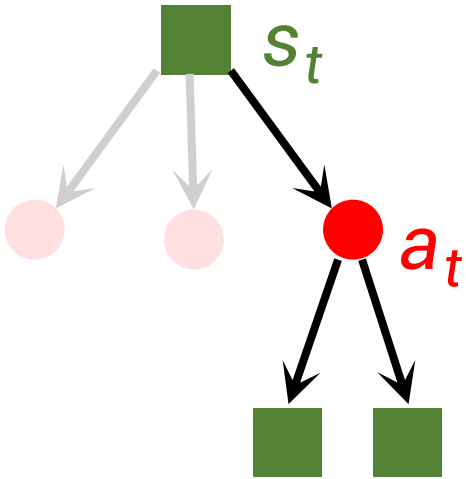
- $Q(a)$: Action-value computed by MCTS. (To be defined.)
- $\pi(a | s_t; \theta)$: The learned policy network.
- $N(a)$: Given s_t , how many times we have selected a so far.

Second, the action with the biggest $\text{score}(a)$ is selected.

Step 2: Expansion

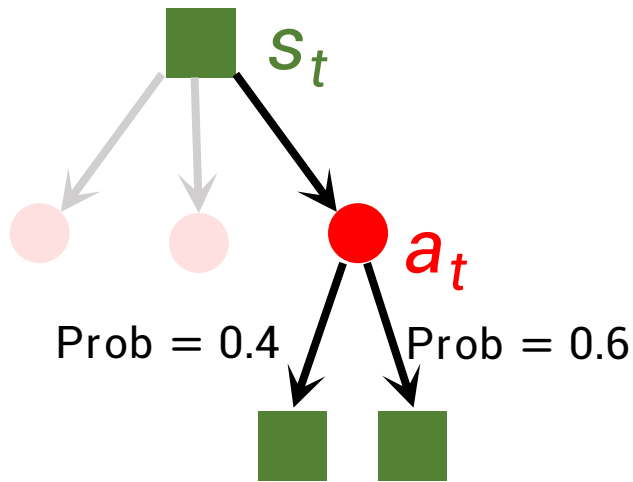
Question: What will be the opponent's action?

- Given a_t , the opponent's action a'_t will lead to the new state s_{t+1} .



Step 2: Expansion

Question: What will be the opponent's action?



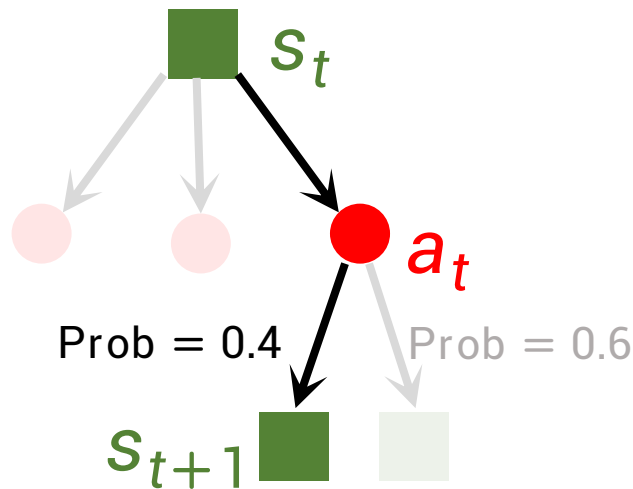
- Given a_t , the opponent's action a'_t will lead to the new state s_{t+1} .
- The opponent's action is randomly sampled from

$$a'_t \sim \pi(\cdot \mid s'_t; \theta).$$

Here, s'_t is the state observed by the opponent.

Step 2: Expansion

Question: What will be the opponent's action?



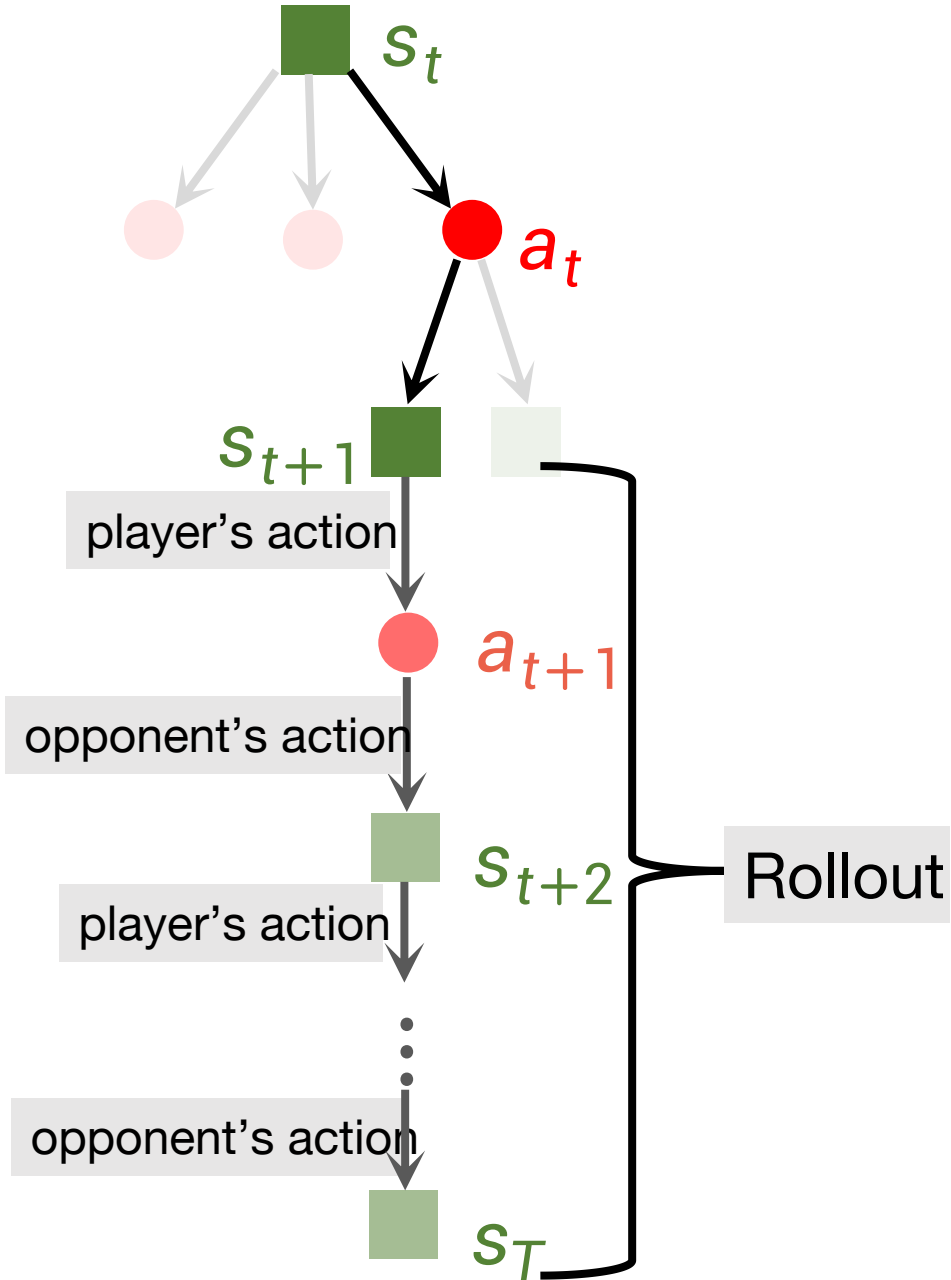
- Given a_t , the opponent's action a'_t will lead to the new state s_{t+1} .
- The opponent's action is randomly sampled from

$$a'_t \sim \pi(\cdot \mid s'_t; \theta).$$

Here, s'_t is the state observed by the opponent.

- The state-transition probability $p(s_{t+1} \mid s_t, a_t)$ is unknown.
- Use the policy function π as the state-transition function p .

Step 3: Evaluation

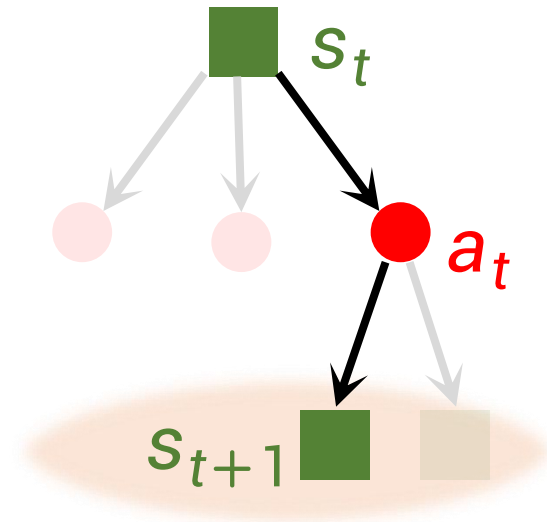


Run a rollout to the end of the game (step

T) Player's action: $a_k \sim \pi(\cdot \mid s_k; \theta)$.

• Opponent's action: $a'_k \sim \pi(\cdot \mid s'_k; \theta)$.

Step 3: Evaluation



Run a rollout to the end of the game (step

T) Player's action: $a_k \sim \pi(\cdot \mid s_k; \theta)$.

• Opponent's action: $a'_k \sim \pi(\cdot \mid s'_k; \theta)$.

• Receive reward r_T at the end.

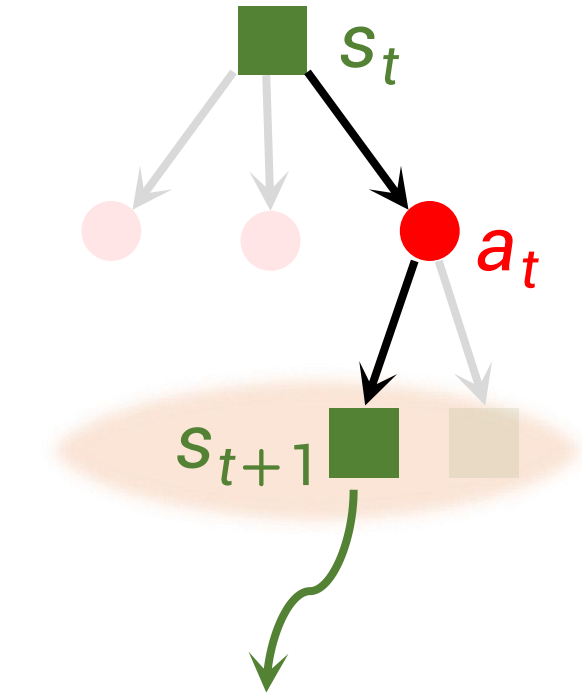
• Win: $r_T = +1$.

• Lose: $r_T = -1$.

Evaluate the state s_{t+1} .

• $v(s_{t+1}; \mathbf{w})$: output of the value network.

Step 3: Evaluation



Record $V(s_{t+1})$.

$$V(s_{t+1}) = \frac{1}{2} v(s_{t+1}; \mathbf{w}) + \frac{1}{2} r_T.$$

Run a rollout to the end of the game (step

T) Player's action: $a_k \sim \pi(\cdot | s_k; \theta)$.

• Opponent's action: $a'_k \sim \pi(\cdot | s'_k; \theta)$.

• Receive reward r_T at the end.

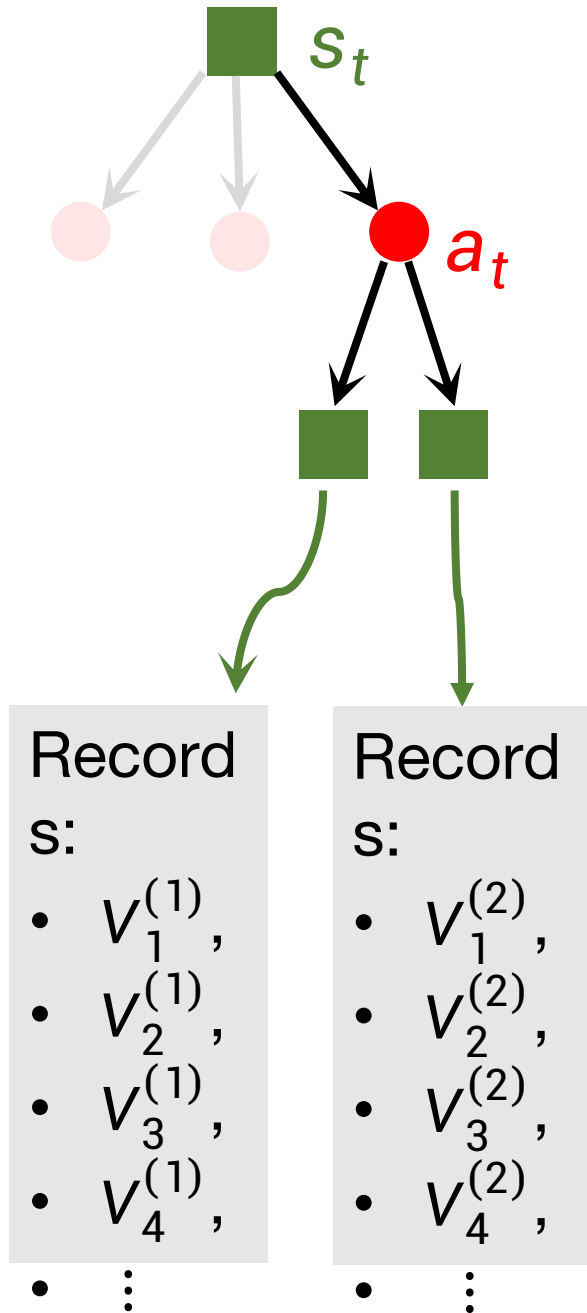
• Win: $r_T = +1$.

• Lose: $r_T = -1$.

Evaluate the state s_{t+1} .

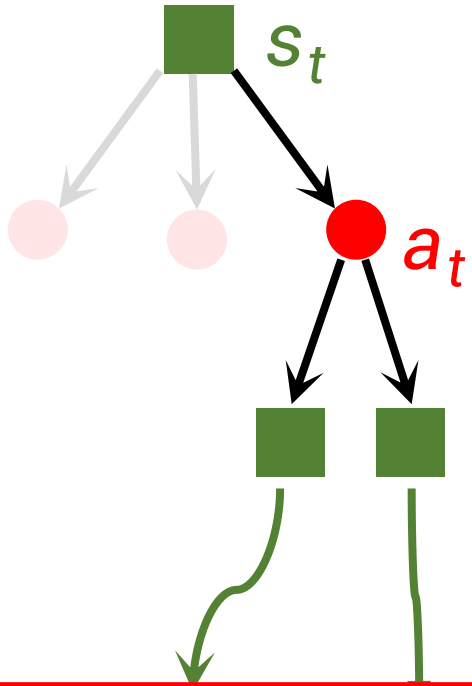
• $v(s_{t+1}; \mathbf{w})$: output of the value network.

Step 4: Backup



- MCTS repeats such a simulation many times.
- Each child of a_t has multiple recorded $V(s_{t+1})$.

Step 4: Backup



- MCTS repeats such a simulation many times.
- Each child of a_t has multiple recorded $V(s_{t+1})$.
- Update action-value:

$$Q(a_t) = \text{mean}(\text{the recorded } V\text{'s}).$$

- The Q values will be used in Step 1 (selection).

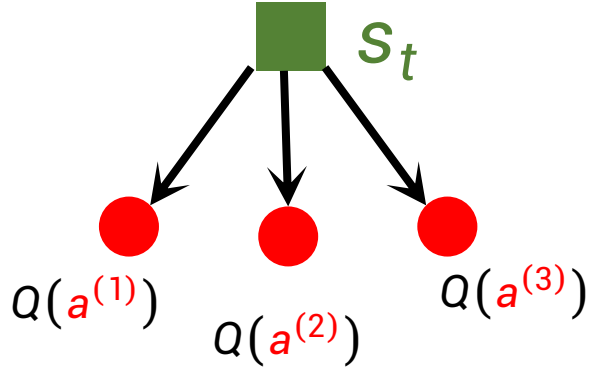
Record
s:

- $V_1^{(1)}$,
- $V_2^{(1)}$,
- $V_3^{(1)}$,
- $V_4^{(1)}$,

Record
s:

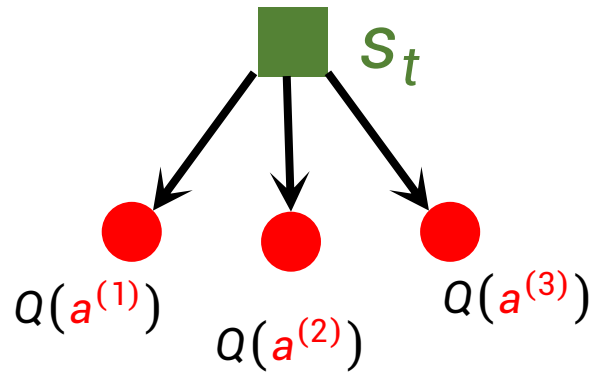
- $V_1^{(2)}$,
- $V_2^{(2)}$,
- $V_3^{(2)}$,
- $V_4^{(2)}$,

Step 4: Backup



- MCTS repeats such a simulation many times.
- Each child of a_t has multiple recorded $V(s_{t+1})$.
- Update action-value:
$$Q(a_t) = \text{mean}(\text{the recorded } V\text{'s}).$$
- The Q values will be used in Step 1 (selection).

Revisit Step 1 Selection

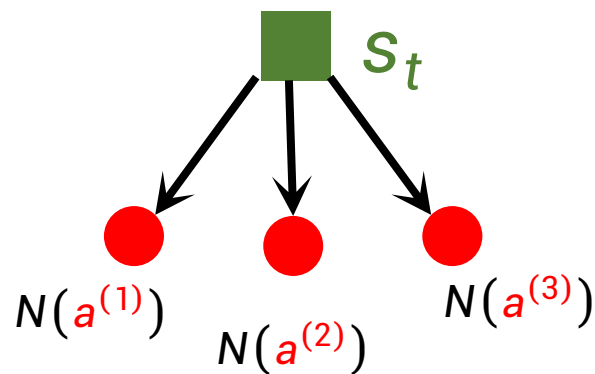


First, for all the valid actions a , calculate the score:

$$\text{score}(a) = Q(a) + \eta \cdot \frac{\pi(a | s_t; \theta)}{1 + N(a)}.$$

Second, the action with the biggest score(a) is selected.

Decision Making after MCTS



- $N(a)$: How many times a has been selected so far.
- After MCTS, the player makes **actual** decision:
$$a_t = \operatorname{argmax}_a N(a)$$

- MCTS has 4 steps: **selection**, **expansion**, **evaluation**, and **backup**.
- To perform one action, AlphaGo repeats the 4 steps for many times to calculate $Q(a)$ and $N(a)$ (for every action a .)
- AlphaGo executes the action a with the highest $N(a)$ value.
- To perform the next action, AlphaGo does MCTS all over again. (Initialize $Q(a)$ and $N(a)$ to zeros.)

Summary

Training in 3 steps:

1. Train a policy network using behavior cloning.
2. Train the policy network using policy gradient algorithm.
3. Train a value network.

Training in 3 steps:

1. Train a policy network using behavior cloning.
2. Train the policy network using policy gradient algorithm.
3. Train a value network.

Execution (actually play Go games):

- Select the “best” action by Monte Carlo Tree Search.
- To perform one action, AlphaGo repeats simulations many times.

AlphaGo Zero

AlphaGo Zero v.s. AlphaGo

- AlphaGo Zero is stronger than AlphaGo. (100-0 against AlphaGo.)
- Differences:
 - AlphaGo Zero does not use human experience. (No behavior cloning.)
 - MCTS is used to train the policy network.

Is behavior cloning useless?

- AlphaGo Zero does not use human experience. (No behavior cloning.)
- For the Go game, human experience is harmful.
- **In general, is behavior cloning useless?**
- What if a surgery robot (randomly initialized) is learned purely by performing surgery? (Human experience is not used.)

Training of policy network

AlphaGo Zero uses MCTS in training. (AlphaGo does not.)

1. Observe state s_t .
2. Predictions made by policy network:

$$\mathbf{p} = [\pi(a = 1|s_t; \boldsymbol{\theta}), \dots, \pi(a = 361|s_t; \boldsymbol{\theta})] \in \mathbb{R}^{361}.$$

Training of policy network

AlphaGo Zero uses MCTS in training. (AlphaGo does not.)

1. Observe state s_t .

2. Predictions made by policy network:

$$\mathbf{p} = [\pi(a = 1|s_t; \boldsymbol{\theta}), \dots, \pi(a = 361|s_t; \boldsymbol{\theta})] \in \mathbb{R}^{361}.$$

3. Predictions made by MCTS:

$$\mathbf{n} = \text{normalize}[N(a = 1), N(a = 2), \dots, N(a = 361)] \in \mathbb{R}^{361}.$$

4. Loss: $L = \text{CrossEntropy}(\mathbf{n}, \mathbf{p})$.

5. Use $\frac{\partial L}{\partial \boldsymbol{\theta}}$ to update $\boldsymbol{\theta}$.

- AlphaGo:
 - Silver and others: Mastering the game of Go with deep neural networks and tree search. *Nature*, 2016.
- AlphaGo Zero:
 - Silver and others: Mastering the game of Go without human knowledge. *Nature*, 2017.